

The Data Analyst Who Pretends to Work

The Book Your Boss Shouldn't Know Exists



Andrea Bruno

The Data Analyst Who Pretends to Work

The Book Your Boss Shouldn't Know Exists

Andrea Bruno

Graphene Lab

2026

A Note About the Tool Behind This Book	14
What are AgentBridge and PowerBITool?	14
How to get it (it is free)	15
Stuck? Real people answer in 24 hours — for free	16
How to read this book	17
About the images in this book	18
1. What a Data Analyst Actually Does	20
The job in one line	20
A day in the life	20
The four cousins (and how they differ)	21
Where the analyst works	22
A curiosity: the oldest analyst	23
Try it: see a model in one sentence	23
What you'll carry from this chapter	24
2. A Short History: From Tally Men to Data Analysts	25
The first analysts: the tally men	25
Counting people: the census	25
1662: a man finds a pattern in death records	26
1801: a Scotsman makes numbers visible	26
1858: the chart that saved thousands of lives	26
1890: the machine that became IBM	27
1979: the spreadsheet comes home	28
1989: someone gives the job a name	28
2010s: everyone gets a dashboard	28
Now: the assistant that does it for you	28
What you'll carry from this chapter	29
3. Thinking in Data	30
The golden rule: compared to what?	30
The trap of the missing bullet holes	30

Evidence over opinion	31
KPIs: the few numbers that matter	31
Goals worth chasing: the SMART idea	31
The dashboard is a cockpit	32
A curiosity: the base rate	32
What you'll carry from this chapter	32
4. Statistics Without the Pain	34
The three averages: mean, median, mode	34
Spread: are things steady or wild?	34
The bell curve (and why it shows up everywhere)	35
Outliers: the one weird number	35
The big trap: correlation is not causation	35
A curiosity: the correlation coefficient	36
What you'll carry from this chapter	36
5. The Analytical Mind: Turning Questions into Numbers	38
From "something is wrong" to a real question	38
The four-step method	38
The Five Whys	39
Don't boil the ocean	39
Beware the question you can't answer	39
A curiosity: the cobra effect	40
A worked example, start to finish	40
What you'll carry from this chapter	40
6. Where Data Comes From	42
Three kinds of data	42
The usual suspects: where business data hides	42
Taking the temperature: profiling	43
Seeing the columns clearly	44
A curiosity: the "fifth kind" of data	45

Try it yourself	45
What you'll carry from this chapter	46
7. Dirty Data and How to Clean It	47
The usual mess	47
Cleaning with calculated columns	47
Check before you commit	50
Hunting the blanks	51
A curiosity: 80/20 of the job	52
When cleaning is never enough	52
What you'll carry from this chapter	53
8. Models, Tables, and Relationships	54
Tables, rows, and keys	54
The relationship: how two tables talk	54
Wiring it up, live	55
Seeing the whole wiring	57
The star schema: the shape you want	57
A calculated table: summarising on the fly	58
A curiosity: the many-to-many trap	58
Try it yourself	59
What you'll carry from this chapter	59
9. Asking Questions with SQL and DAX	61
SQL: the language of databases	61
DAX: the language of Power BI	61
Counting and summing	62
Ranking and filtering	63
Grouping across a relationship	65
Pulling a related value	65
A few more, because they're easy	66
A curiosity: DAX's scary reputation	68

What you'll carry from this chapter	68
10. Spreadsheets, Excel, and Hitting the Wall	69
Why spreadsheets won	69
The pivot table: analysis in a box	69
The wall	69
The spreadsheet vs. the model	70
The same question, two ways	70
A curiosity: the spreadsheet that lost a billion	71
When to stay, when to go	72
What you'll carry from this chapter	72
11. The Metrics That Matter	73
What makes a metric worth watching	73
The core sales metrics	73
Money metrics: margin and share	77
A few more quick ones	79
A curiosity: the metric that backfired	80
What you'll carry from this chapter	80
12. Four Kinds of Analysis	82
1. Descriptive — what happened?	82
2. Diagnostic — why did it happen?	83
3. Predictive — what will happen?	85
4. Prescriptive — what should we do?	86
The ladder in one picture	87
Ranking: the analyst's favourite move	87
A curiosity: the analytics maturity myth	89
What you'll carry from this chapter	89
13. Knowing Your Customers	90
The customer questions	90
A per-customer measure: the foundation	90

Who are the best customers?	91
Segmenting: slicing customers into groups	93
A real framework: RFM	95
Finding the high-value ones	96
A couple more cuts	96
A curiosity: the churn you can't see	98
What you'll carry from this chapter	98
14. Trends, Time, and Seasonality	99
Why time is special	99
Getting the date into usable pieces	99
The year-over-year view	101
The monthly trend	102
Filtering a period	103
Finding the slow period	105
Moving averages: smoothing the noise	105
A curiosity: the "January effect" that isn't	106
What you'll carry from this chapter	106
15. Testing, Chance, and Being Right	107
The problem: was it the change or the luck?	107
The idea of a sample	107
A/B testing: the honest experiment	107
Significance: is the difference real?	108
The two ways to be wrong	108
Comparing two groups, live	109
A curiosity: the cookie that fooled everyone	109
When you don't need a formal test	110
What you'll carry from this chapter	110
16. Power BI in Plain English	111
What Power BI actually is	111

The three layers inside	111
Seeing the model, live	112
The data dictionary: documentation for free	113
Seeing the measures	114
A curiosity: Power BI's improbable rise	115
Why the assistant matters here	115
What you'll carry from this chapter	116
17. Connecting to Your Data	117
The local connection	117
Finding what's open	117
Confirming the connection	118
What "live" really means	119
A curiosity: the port is a secret door	120
Reconnecting and safety	120
What you'll carry from this chapter	120
18. Modelling Data in Power BI	121
What a good model looks like	121
Documenting as you go	121
The data dictionary, table by table	123
The health check: best practices	124
Re-checking after changes	125
A curiosity: the bus factor	126
Why the assistant is a good modeller	126
What you'll carry from this chapter	127
19. DAX: The Language Behind the Numbers	128
What DAX is for	128
The assistant writes it; you read it	128
Validate before you trust	129
Linting: the style cop for DAX	132

Editing measures	134
A curiosity: the filter context beast	135
What you'll carry from this chapter	135
20. Choosing the Right Chart	137
The one rule	137
Comparing categories: use bars	137
Change over time: use a line	139
Part of a whole: use a pie (carefully)	140
The charts to avoid	142
A curiosity: the chart that lied to a nation	142
Colour with purpose	142
A dashboard is a story, not a paint palette	143
What you'll carry from this chapter	143
21. Reports and Dashboards People Actually Use	144
Dashboard vs. report — they're not the same	144
The KPI row: the top of every good dashboard	144
The single big number	146
The leaderboard	147
What makes a dashboard usable	149
The drill-down principle	149
A curiosity: the cockpit rule	149
The assistant's role in the dashboard	149
What you'll carry from this chapter	150
22. The Rise of the AI Assistant	151
The old way: a chain of manual steps	151
The new way: describe the answer	151
Why this is a bigger deal than "autocomplete"	153
The agentic shift	154
What it means for the analyst	154

A curiosity: the Jevons paradox of analysis	155
The honest caveat	155
What you'll carry from this chapter	155
23. Meet AgentBridge and PowerBITool	156
AgentBridge: the assistant	156
PowerBITool: the hands inside Power BI	156
How they fit together	158
The safety guard	158
Free, and supported by real people	158
A curiosity: the plugin model	159
What you can do with them, together	159
What you'll carry from this chapter	159
24. Automating the Analyst's Day	160
Morning: build the reporting pieces	160
Before the meeting: validate everything	161
Mid-morning: catch the gaps	162
Late morning: model behaviour	163
Afternoon: targeting	165
End of day: document and check	166
Close: the finished model	168
What the day shows	168
What you'll carry from this chapter	169
25. Storytelling, Ethics, and Governance	170
Storytelling: the number is not the point	170
The honesty of a chart	171
Ethics: three questions to ask before you publish	171
Governance: the model is an asset	172
The human edge, restated	172
A short checklist for the human	173

What you'll carry from this chapter	173
26. Your Career as a Data Analyst in the Age of the Assistant	174
The job description is changing	174
What the assistant does <i>not</i> take away	174
The skills to double down on	175
A curiosity: the calculator analyst	175
The portfolio beats the certificate	176
The honest warning	176
Where this leaves you	176
What you'll carry from this chapter	177
Conclusion: The Analyst Who Stopped Pretending	178
The pretending is over	178
What this book tried to do	178
The promise behind every example	179
A final curiosity	179
Appendix A — Glossary of Terms	180
Appendix B — Data Quality Checklist	183
Completeness	183
Accuracy	183
Consistency	183
Uniqueness	184
Validity	184
Timeliness	184
Integrity	184
How the assistant helps	184
Appendix C — Dashboard Checklist	186
Purpose	186
Layout	186
Metrics	186

Visuals	186
Interactivity	187
Trust	187
Honesty	187
Appendix D — Real Cases by Sector	188
Retail	188
E-commerce	188
Manufacturing	189
Finance	189
Healthcare	189
The common thread	190
Appendix E — Guided Exercises	191
Exercise 1 — Read the model	191
Exercise 2 — Profile a table	191
Exercise 3 — Clean text	191
Exercise 4 — Bucket a number	191
Exercise 5 — Wire a relationship	192
Exercise 6 — Build a measure	192
Exercise 7 — Filter a measure	192
Exercise 8 — Share of total	192
Exercise 9 — Rank	192
Exercise 10 — Validate before you save	193
Exercise 11 — Lint	193
Exercise 12 — Document	193
Appendix F — Exercise Solutions	194
Exercise 1 — Read the model	194
Exercise 2 — Profile a table	194
Exercise 3 — Clean text	194
Exercise 4 — Bucket a number	194

Exercise 5 — Wire a relationship	195
Exercise 6 — Build a measure	195
Exercise 7 — Filter a measure	195
Exercise 8 — Share of total	195
Exercise 9 — Rank	196
Exercise 10 — Validate before you save	196
Exercise 11 — Lint	196
Exercise 12 — Document	196
The pattern in every solution	196
Appendix G — A Portfolio Project Model	197
The project: a sales analysis dashboard	197
What to show in the portfolio	198
Why this works	199
Make it yours	199
Appendix H — Interview FAQs	200
“What does a data analyst actually do?”	200
“SQL or DAX?”	200
“What’s the difference between a calculated column and a measure?”	
“Explain CALCULATE.”	200200
“How do you handle divide-by-zero?”	201
“How do you check data quality?”	201
“What’s a star schema?”	201
“How do you deal with a misleading chart?”	201
“What do you do when the data contradicts the expected answer?”	
“How do you use AI tools in your workflow?”	202201
“Tell me about a project you built.”	202
The meta-answer	202

A Note About the Tool Behind This Book

This book is about a job: the data analyst. It is about what that job really is, where it came from, and where it is going. It uses plain words. You do not need a degree in mathematics or computer science to follow it. If you run a small business, keep your own numbers, or just like understanding how things work, this book is for you.

Here is the honest part. Every single example you will see in this book — every table, every measure, every chart, every “watch this” moment — was made with a real tool, not typed by hand. That tool is **PowerBITool**, running inside **AgentBridge**.

What are AgentBridge and PowerBITool?

AgentBridge is an AI assistant that runs on your own computer. You talk to it the way you would talk to a colleague: in ordinary sentences. It listens, it thinks, and it does the work.

PowerBITool is a plugin that gives AgentBridge hands inside **Microsoft Power BI Desktop** — the popular program people use to build dashboards and reports. With PowerBITool, the assistant can open your data model, add tables, create measures, connect tables together, run queries, check your work, and take a screenshot of what it made — all while you watch it happen on your own screen.

No cloud. No uploading your company’s data to a stranger’s server. It works with the Power BI Desktop already on your machine.

You → AgentBridge → PowerBITool → your Power BI Desktop (on your PC)

How to get it (it is free)

PowerBITool is free and open. To try it yourself:

1. Install **AgentBridge** (free) from the GitHub page below.
2. Add the **PowerBITool** plugin to it.
3. Open a report in **Power BI Desktop**.
4. Start talking to your assistant.

Scan this code with your phone camera to open the PowerBITool page, where you will find the download and simple, step-by-step install instructions:



PowerBITool on GitHub

github.com/Graphene-Lab/PowerBITool

You can also just type that address into a browser.

Stuck? Real people answer in 24 hours — for free

Here is something we are proud of. PowerBITool is free, and so is the help that comes with it. If something does not work, or you want a feature, or you simply found a bug, you open an **issue** on the same GitHub page and our technicians answer — usually within **24 hours**, and with a real fix, not a canned reply.

Scan this code to reach the issues page and see how it works:



Report an issue to PowerBITool

github.com/Graphene-Lab/PowerBITool/issues

That is the whole promise: a free tool, free support, real humans, fast answers.

How to read this book

You do not need to install anything to enjoy this book. Read it like a story if you like. But if you want to try things as you go — and we hope you will — each hands-on example shows two things:

- **What a person typed** to the assistant (one or two ordinary sentences).
- **What came back** (the real result, from the real tool).

The pictures in this book show that exchange: the question on the right, the answer from PowerBITool on the left, exactly as it appears in AgentBridge.

About the images in this book

You will see two kinds of pictures.

Chat panels show the exchange itself: what a person typed, and the real result PowerBITool returned from the live model.

Chart images show that same real data *visualised* — bar, line, and donut charts drawn from the actual numbers the tool returned (sales by category, top customers, the monthly trend, and so on). They are rendered visualisations of the real captured output, so you can see the data as a picture, not only as text.

One honest note about Power BI Desktop itself. Power BI renders this same data on its own report canvas, and the tool can capture that canvas as a PNG (`CaptureReportScreenshot`, over the Power BI Desktop Bridge). That capture needs a report with visuals already built in the Power BI Desktop window. This book was produced in an environment without a GUI-built report, so the chart images here are rendered from the real data rather than screen-captured from Power BI. The procedure to capture authentic Power BI Desktop screenshots ships with the tool, and you can drop those captures straight into these same spots.

Let's begin with the job itself.

— *Graphene Lab*

1. What a Data Analyst Actually Does

Ask ten people what a data analyst does and you will get ten different answers. Some picture a person in a dark room typing green code. Some picture someone making pretty pie charts. Both are wrong, and neither is wrong. Let's clear it up.

A data analyst is a person who answers questions using numbers. That is the whole job. The questions come from real life: *Why did sales drop last month? Which product should we push? Which customers are about to leave?* The answers come from data: the records a business leaves behind every day — sales, clicks, invoices, support tickets, delivery times.

The analyst's skill is not math. It is **translation**. They turn a fuzzy human question into a precise question about data, find the answer, and turn it back into a sentence a busy manager can act on.

The job in one line

A data analyst turns a messy real-world question into a clear, honest answer backed by numbers.

Everything else — the tools, the charts, the code — is just how they get there.

A day in the life

Picture a Tuesday. At 9:00 the head of sales drops a question into the analyst's inbox: *"Why is our Milan store down 12% this month?"*

That single sentence is actually five questions:

- Is the 12% real, or a fluke of how the numbers were counted?
- Is it just Milan, or are other stores down too?
- What changed this month — price, stock, weather, a competitor opening next door?
- Is the drop in the number of customers, or in what each customer spends?
- What could we actually do about it?

The analyst digs through sales records, customer data, and whatever else might explain it. By the afternoon they have an answer: *foot traffic fell because a road was closed for construction for two weeks; the customers who did come spent the same as always*. Not a mystery. A road.

That is the job. Curiosity, a little method, and the data.

The four cousins (and how they differ)

People mix up four nearby jobs. Here is the simple version.

Role	What they mostly do	The question they answer
Data analyst	Looks at what already happened, explains it	“What happened and why?”
Data scientist	Builds models to predict or guess	“What will happen next?”
Data engineer	Builds the pipes that move and store data	“How do we get clean data here?”
BI analyst	Builds dashboards and reports people look at	“How do we see it every day?”

There is a lot of overlap. In a small company, one person is all four. But the data analyst’s home ground is the first column: understanding the present and the recent past.

Where the analyst works

Everywhere. A few real shapes of the job:

- **Retail** — why did a store’s sales fall; which products sell together.
- **Banking** — watching transactions for patterns that look like fraud.
- **E-commerce** — where do shoppers abandon their cart, and why.
- **Manufacturing** — which shift or line throws off more defective parts.
- **Software (SaaS)** — how many subscribers stay, and what makes them leave.

Different buildings, same job: ask a question, find the numbers, tell the truth.

A curiosity: the oldest analyst

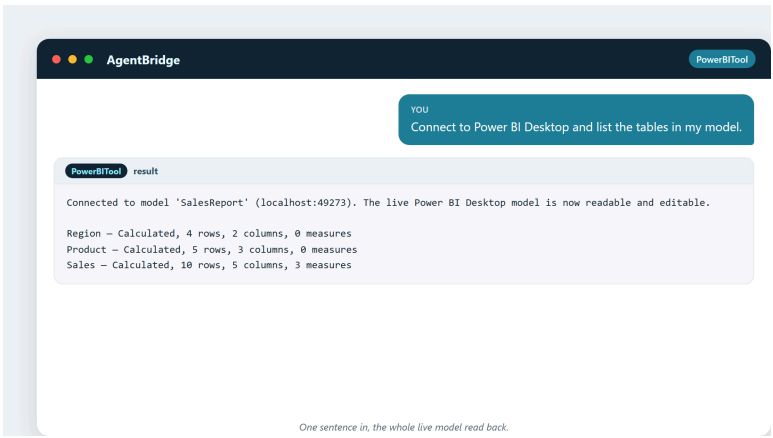
Data analysis is older than computers by thousands of years. The ancient Egyptians sent scribes to count the harvest and the cattle so they could plan taxes and grain stores before a famine. Those scribes were data analysts. The spreadsheet is new; the job is ancient.

Try it: see a model in one sentence

Enough theory. Here is your first taste of the tool this book is built on. In AgentBridge, with PowerBITool, a person typed one sentence about a Power BI report that was already open on their screen:

“Connect to Power BI Desktop and list the tables in my model.”

Here is what came back:



Connect and list tables

In one line, the assistant read the live model and reported every table, how many rows it holds, and how many measures it has. No menus clicked, no code written. That is the shape of everything in this book: a plain sentence in, a real result out.

What you'll carry from this chapter

- A data analyst answers real questions with numbers.
- The core skill is translation, not math.
- The job is ancient; the tools are new — and getting faster.

In the next chapter we travel back in time to see how counting turned into a career.

2. A Short History: From Tally Men to Data Analysts

The data analyst is not a new job. It is one of the oldest jobs there is, wearing a very new costume. Let's walk through time and meet the ancestors of everyone who today opens a dashboard before their coffee.

The first analysts: the tally men

Around 8,000 years ago, in the land of Sumer (modern Iraq), people used small clay tokens to count things: one token for one sheep, another for a jar of grain. To “run a report” you pressed the tokens into a clay ball and wrote on the outside. That clay ball was the world's first spreadsheet, and the person who made it was the world's first data analyst.

The ancient Egyptians did the same on a grander scale. Every year they sent scribes to measure the Nile's flood, count the cattle, and estimate the harvest. Why? So the pharaoh could plan taxes and fill the granaries before a bad year. Sound familiar? It is the same reason a modern company forecasts sales: *look at the numbers now so you are not surprised later.*

Counting people: the census

Almost every empire kept a census — a headcount of its people, for taxes and for armies. The Romans took a census every few years (the word *census* is simply Latin for “counting”). Han China counted its population centuries before Europe caught up. These were not idle

curiosities. A ruler who did not know how many people he had could not tax them, feed them, or defend them.

1662: a man finds a pattern in death records

In London, plague deaths were published every week in sheets called the *Bills of Mortality*. Most people read them and felt sad. One man, a haberdasher named **John Graunt**, read them and saw a *pattern*. He counted, grouped, and compared years of death records and published a small book in 1662. He noticed that deaths rose and fell in predictable ways, that more boys were born than girls, and that the city could be understood as a system, not a chaos.

Historians call Graunt the father of demography and of statistics. He did it with nothing but a pile of public records and a curious mind. That is the whole method, right there, 360 years ago.

1801: a Scotsman makes numbers visible

A Scottish engineer named **William Playfair** was frustrated. Tables of numbers bored people, and bored people made no decisions. So he invented three things we now take for granted: the **line chart**, the **bar chart**, and the **pie chart**. Playfair turned numbers into pictures, and pictures into understanding. Every chart you have ever seen descends from him.

1858: the chart that saved thousands of lives

This is the most important story in the history of data, and most people have never heard it.

In the Crimean War, **Florence Nightingale** ran a military hospital at Scutari. During her first winter there, **4,077 British soldiers died** — and **ten times more died from disease than from battle wounds**. The hospital was filthy: bad sewers, no ventilation, overcrowding.

Nightingale was also a sharp statistician. She drew a chart — she called it a “coxcomb”, a set of colored wedges — showing that almost all deaths came from preventable infection, not from enemy fire. She put it in front of Members of Parliament who would never have read a table of numbers but could not look away from the picture.

The result: a Sanitary Commission was sent, the sewers were flushed, the wards were ventilated, and the death rate fell from around **42% to 2%**. A chart changed a policy and saved thousands of lives. That is the power of what you do when you analyse data well. In 1859 Nightingale became the **first woman elected to the Royal Statistical Society** — not for nursing, but for statistics.

1890: the machine that became IBM

Counting a whole country by hand was painfully slow. The 1880 U.S. census took **eight years** to process — the data was stale before it was finished. A young statistician named **Herman Hollerith** invented a machine that read **punched cards**: a hole in a position meant “yes”, no hole meant “no”. Electrical contacts counted the holes automatically.

The 1890 census, with more people and more questions than ever, finished in about **six years — two years faster** than the smaller one before it. Hollerith started a company to sell his machines. In 1911 it merged into a larger firm, and in 1924 that firm was renamed

International Business Machines — IBM. The punched card ruled data processing for nearly a century.

1979: the spreadsheet comes home

For a long time, only big institutions could analyse data. Then came the personal computer, and with it **VisiCalc** (1979) — the first spreadsheet. Suddenly an ordinary person could put numbers in boxes and see them add up, sort, and update. The power to analyse moved from the institution to the individual. Excel, born in 1985, carried that torch for decades.

1989: someone gives the job a name

In 1989 an analyst named **Howard Dresner** coined the term “**business intelligence**” — using data to make better business decisions. Tools appeared to pull company data into dashboards that updated themselves. The analyst’s job got a title, a toolkit, and a seat at the table.

2010s: everyone gets a dashboard

Tools like **Power BI** made powerful analysis cheap and visual. A small business could now have the kind of live dashboards that once cost a fortune. The barrier fell again.

Now: the assistant that does it for you

And here we are. The latest step is not a faster chart or a bigger spreadsheet. It is an **assistant** — an AI that listens to a plain-English question and does the whole chain of work: finds the data, writes the query, builds the measure, checks the answer. That is what this book is

about, and it is the natural end of a very long line that starts with a clay token and a curious Sumerian.

Every ancestor above had the same goal you have: turn raw numbers into a decision. The tools changed. The job — and the curiosity behind it — never did.

What you'll carry from this chapter

- Data analysis is ancient: counting the harvest *is* data analysis.
- Charts (Playfair) and honest statistics (Graunt) turned numbers into insight.
- A good chart can literally save lives (Nightingale).
- The tools keep getting faster and cheaper — now an AI does the chain for you.

Next: how to actually *think* in data, and the famous trap of the missing bullet holes.

3. Thinking in Data

There is a way of thinking that separates a good analyst from a person who just makes charts. It is not math. It is a habit of asking, “*How do I know that? And compared to what?*” Let’s build that habit.

The golden rule: compared to what?

A number on its own means almost nothing. “We made 1,000 sales this month.” Is that good? You cannot tell. You need to compare:

- **To last month** — did we grow or shrink?
- **To last year** — is this normal for the season?
- **To the target** — did we hit the goal?
- **To a similar thing** — is this store better than the others?

The moment someone says a number, a trained analyst asks “*compared to what?*” This single question kills more bad decisions than any software ever will.

The trap of the missing bullet holes

During World War II, the U.S. military studied which parts of returning bombers had the most bullet holes, planning to add armour there. A statistician named **Abraham Wald** stopped them. He said: *you are only looking at the planes that came back. The planes hit in the critical spots — the cockpit, the engines — never returned, so you see no holes there.* The armour belonged exactly where the holes were **missing**.

This is called **survivorship bias**, and it is everywhere. You study the successful companies and copy what they did, ignoring the failures that

did the same thing and died. You look at the customers who stayed and miss why the others left. The lesson: always ask what is *not* in your data.

Evidence over opinion

In most meetings, the loudest voice wins. In a data-driven team, the best evidence wins. That does not mean numbers replace judgement — it means opinions get checked against facts. “I feel like the new website is better” becomes “the new site converts 3.1% versus 2.4%, and the difference is real.” Feelings get a test.

KPIs: the few numbers that matter

A **KPI** — Key Performance Indicator — is a number you watch because it tells you whether you are winning. The art is choosing the *few* that matter and ignoring the hundreds that do not.

A good KPI is: - **Clear** — everyone agrees what it means. -

Measurable — you can actually compute it. - **Tied to a goal** — it moves when the business moves. - **Actionable** — if it’s bad, you can do something.

A bad KPI is a number that looks impressive but changes nothing. Vanity metrics — “total website visits ever” — feel good and decide nothing.

Goals worth chasing: the SMART idea

A vague goal (“sell more”) produces vague work. A **SMART** goal gives the analyst something to measure:

- **Specific** — “increase online sales”

- **Measurable** — “by 10%”
- **Achievable** — realistic, not a wish
- **Relevant** — it matters to the business
- **Time-bound** — “by the end of the quarter”

Now the analyst can answer: are we there? By how much? On track? A good goal is a question with a deadline.

The dashboard is a cockpit

Think of a dashboard like an airplane cockpit. The pilot does not want 500 dials. They want the handful that tell them: altitude, speed, fuel, heading. A good dashboard is the same — a few honest numbers, visible at a glance, that let you act before something goes wrong. If a dashboard needs a manual to read, it is not a dashboard; it is a report.

A curiosity: the base rate

Imagine a disease affects 1 in 1,000 people. A test is 99% accurate. You test positive. What are the chances you actually have it? Most people say 99%. The real answer is closer to **9%**. Because the disease is so rare, the handful of false positives from 999 healthy people roughly equals the one true positive. This is the **base rate**, and ignoring it fools doctors, lawyers, and CEOs alike. Thinking in data means always asking: *how common is this to begin with?*

What you’ll carry from this chapter

- Always ask “*compared to what?*”
- Watch for what is missing (survivorship bias).
- Let evidence, not volume, win arguments.

- Choose a few actionable KPIs; set SMART goals.
- A dashboard is a cockpit, not a filing cabinet.

Next: the small amount of statistics you actually need — and the one mistake about correlation that trips up everyone.

4. Statistics Without the Pain

You do not need a lot of statistics to be a good analyst. You need a handful of ideas, understood deeply, and the wisdom to know when they fool you. Here is the whole kit, in plain words.

The three averages: mean, median, mode

People say “average” as if there were one. There are three, and choosing the wrong one can lie without technically being wrong.

- **Mean** — add everything, divide by the count. The classic average.
- **Median** — the middle value when you line them all up. Half are above, half below.
- **Mode** — the most common value.

Why does it matter? Picture a small company. Ten staff earn €30,000, and the boss earns €500,000.

- The **mean** salary is €72,727 — “we pay well!”
- The **median** salary is €30,000 — the typical worker’s reality.

One number is “correct” and the other is “correct”, and they tell completely different stories. When a few extreme values (outliers) are in the mix, the **median** is usually the honest one. When someone quotes an average, ask: *mean or median?*

Spread: are things steady or wild?

An average hides how spread out the numbers are. Two delivery services both average 3 days. One always takes 3 days. The other takes 1

day or 5 days at random. Same average, totally different experience.

The measure of spread you will use most is the **standard deviation** — roughly, “how far from the average things usually are.” Small standard deviation = steady, predictable. Large = wild, unreliable. Averages tell you the centre; spread tells you the risk.

The bell curve (and why it shows up everywhere)

Many real things — heights, test scores, measurement errors — pile up around the middle and thin out at the ends, forming a bell shape. This is the **normal distribution**, and it is everywhere because of a beautiful fact: when many small random influences add up, the result tends toward a bell. You do not need the math. You need the instinct: most cases are near the middle, extremes are rare, and a value far out in the tail is worth investigating.

Outliers: the one weird number

An **outlier** is a value far from the rest. One customer buys €50,000 while everyone else buys €50. One delivery takes 30 days while the rest take 3. Outliers can be: - **Errors** — a typo, a test record, a misplaced decimal. - **Real but rare** — a whale customer, a genuine disaster.

Always look at outliers before you trust an average. A single fat client can make a whole month look great and hide that the other 200 customers are leaving.

The big trap: correlation is not causation

This is the single most important sentence in this book.

Correlation means two things move together. **Causation** means one thing *causes* the other. They are not the same, and confusing them causes expensive nonsense.

Classic example: **ice cream sales and drowning deaths rise together** every summer. Does ice cream cause drowning? No. A third thing — hot weather — drives both. When you see two things move together, always ask: - Does A cause B? - Does B cause A? - Does some hidden C cause both? - Is it just coincidence?

“Customers who use our app more are happier” might mean the app makes them happy — or that already-happy customers use it more. Correlation points you at a clue. It does not hand you the answer.

A curiosity: the correlation coefficient

Statisticians squeeze “how strongly two things move together” into one number from **-1 to +1**. +1 means they rise in perfect lockstep; -1 means one rises as the other falls; 0 means no relationship. It is a useful thermometer for a relationship — but remember, even a perfect +1 is still not proof of cause.

What you’ll carry from this chapter

- Know which average you are using; the median often tells the truth.
- Averages hide spread — watch the standard deviation.
- Outliers can fake a whole story; look at them first.
- Correlation is a clue, never proof. Always hunt for the hidden third thing.

Next: how to turn a fuzzy business worry into a sharp question you can actually answer with data.

5. The Analytical Mind: Turning Questions into Numbers

The heart of the job is a single move: taking a fuzzy worry and turning it into a question that data can answer. Master that move and you can work with any tool. Let's learn it.

From “something is wrong” to a real question

A manager storms in: “*Sales are bad!*” That is not a question. It is a mood. The analyst's first job is to turn the mood into a question with an answer.

- “Bad compared to what?” → *compared to last quarter.*
- “Bad everywhere, or somewhere?” → *only in the north region.*
- “Bad in what?” → *in units, not in price.*
- “Since when?” → *starting in March.*

Now it is a real question: “*Why did unit sales in the north region fall starting in March?*” That question can be answered. The original could not.

The four-step method

Almost every analysis follows the same four steps. Learn this loop and you have the job.

1. **Question** — write the real, answerable question.
2. **Hypothesis** — guess the likely answer *before* you look. (e.g. “a competitor opened nearby.”)

3. **Test** — pull the data that would confirm or kill the guess.
4. **Conclusion** — what does the data say? Was the guess right? What now?

The hypothesis step is the secret. Guessing first stops you from wandering through data until you find something that confirms whatever you stumbled on. It keeps you honest.

The Five Whys

A trick from Toyota for getting to a root cause: ask “why?” five times.

- Sales are down. *Why?* Fewer customers came in.
- *Why?* Foot traffic dropped.
- *Why?* The bus stop moved away.
- *Why?* The city redesigned the street.
- *Why?* ...and now you see the real cause is not your marketing at all.

You often do not need five. Two or three “whys” usually burrow past the symptom to the thing you can actually fix.

Don’t boil the ocean

A rookie mistake is trying to analyse *everything*. You cannot. Pick the smallest slice of data that could answer the question, look at that, and only widen if you need to. A focused 20-minute check beats a week of drowning in spreadsheets.

Beware the question you can’t answer

Some questions have no answer in the data you have. “*Why do people not buy our product?*” may need a survey, not a database. A good

analyst knows the difference between “I need more data” and “I need a different kind of data” — and says so instead of inventing an answer.

A curiosity: the cobra effect

In colonial India, the government, troubled by cobras, offered a bounty for every dead cobra. People started **breeding cobras** to collect the bounty. When the government caught on and cancelled the reward, the breeders released the now-worthless cobras, and the wild cobra population went *up*. Solving the wrong problem — or measuring the wrong thing — can make it worse. Measure carefully. The thing you reward is the thing you get.

A worked example, start to finish

Mood: “Our new app update is a disaster.” **Question:** Did daily active users fall after the update? **Hypothesis:** The update broke the login, so people left. **Test:** Compare daily active users before and after the update date; check login error rates. **Result:** Users fell 15%, but login errors did not rise. The guess was wrong. **New hypothesis:** A feature people loved was removed. **Test:** Look at usage of the removed feature before the update — it was heavily used. **Conclusion:** The update removed a popular feature. That is the cause. Fix: bring it back.

Notice how the data killed the first guess and pointed to the real one. That is the method working. The analyst did not know the answer at the start — they knew how to *find* it.

What you’ll carry from this chapter

- Turn moods into answerable questions.

- Guess first (hypothesis), then test — it keeps you honest.
- Ask “why” a few times to reach the root cause.
- Analyse the smallest useful slice; don’t boil the ocean.
- Measure the right thing, or you breed cobras.

Part I is done — you understand the craft. Next we get our hands dirty with the raw material: where data comes from, and how to see it with a real tool.

6. Where Data Comes From

Before you can analyse anything, you need data — and you need to know what shape it is in. This chapter is about the raw material: where it comes from, the forms it takes, and how to take its temperature before you build anything.

Three kinds of data

Everything you will ever analyse falls into three buckets:

- **Structured** — tidy rows and columns. A sales table, a customer list, a bank statement. Easy for computers to read. This is your bread and butter.
- **Semi-structured** — has some order but not a neat grid. A web log, a JSON file from an app, an email with fields. Needs a little shaping.
- **Unstructured** — no built-in order. Text documents, images, videos, a customer's free-text complaint. The hardest to analyse, and where AI is getting surprisingly good.

Most business analysis lives in the structured world. That is the good news: it is the kind you can point a tool at and get answers fast.

The usual suspects: where business data hides

- **The ERP / management system** — orders, invoices, stock, customers.
- **The CRM** — leads, opportunities, contacts, sales pipeline.
- **Spreadsheets** — the universal fallback, for better and worse.
- **Databases** — SQL servers holding the company's records.

- **Web and app logs** — every click, page view, and event.
- **CSV / Excel exports** — data pulled from any system into a file.
- **APIs** — live data streamed from a service (weather, shipping, payments).
- **IoT sensors** — temperature, machine status, footfall counters.

A real analysis often stitches several of these together. The analyst’s first move is to find the data and understand its shape.

Taking the temperature: profiling

Before you trust a table, you **profile** it: how many rows, what columns, how many distinct values, how many blanks, the minimum and maximum, the most common values. Profiling is a health check that tells you what you are dealing with before you build a single chart.

Here is a real one. A person asked the assistant to profile the products table:

“Profile the Products table.”

AgentBridge

PowerBITool

YOU

Profile the Products table.

PowerBITool

result

Profile of Products – 8 rows, 5 columns

Column	Type	Distinct	Blanks	Min	Max	Top values
ProductID	Int64	8	?	1	8	1 (1), 2 (1), 3 (1), 4 (1), 5 (1), 6 (1), 7 (1), 8 (1)
Name	String	8	?			Ceramic Mug Set Wireless Kettle Espresso Machine (1), Coffee Grinder (1), Wireless Kettle (1), Office Chair (1), Standing Desk (1), Desk Lamp (1), Notebook Pack (1)
Category	String	3	?			Furniture Stationery Kitchen (4), Furniture (3), Stationery (1)
Price	Double	8	?	12	349	12 (1), 24 (1), 39 (1), 59 (1), 89 (1), 189 (1), 249 (1), 349 (1)

Sample rows:

Products[ProductID]	Products[Name]	Products[Category]	Products[Price]
1	Espresso Machine	Kitchen	249
2	Coffee Grinder	Kitchen	89
3	Ceramic Mug Set	Kitchen	24

3 row(s) returned ...

Distinct values, blanks, min/max, top values, samples.

Profile the Products table

In one shot you can see: 8 products, 3 categories (Kitchen 4, Furniture 3, Stationery 1), prices from €12 to €349, and a few sample rows. No guessing. The shape of the data is now obvious.

The same works for customers:

“Profile the Customers table.”



See who is in the data at a glance.

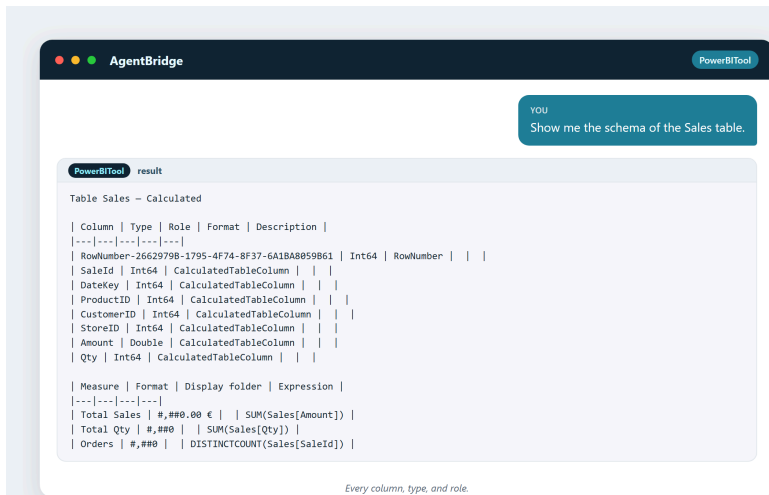
Profile the Customers table

Twelve customers across four cities and three segments. You can already see the story forming — Milan and Rome are the biggest, the segments are balanced.

Seeing the columns clearly

Sometimes you just want the structure — the columns and their types. The assistant reads the schema directly:

“Show me the schema of the Sales table.”



Schema of the Sales table

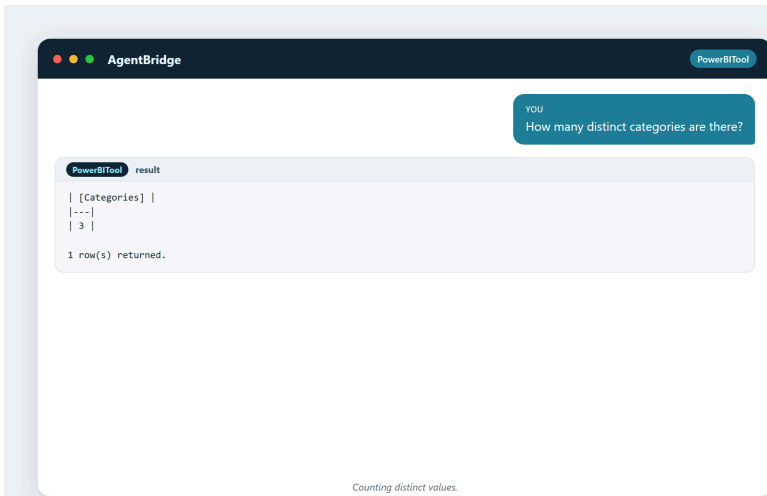
Every column, its type, and the measures already attached. This is the map you carry into every later question.

A curiosity: the “fifth kind” of data

There is a joke among analysts that the fifth kind of data is **data you did not know you had** — the metadata. When did each record change? Who touched it? How many times was a page viewed? Metadata is the data *about* your data, and it often holds the most interesting answers of all.

Try it yourself

“How many distinct categories are there?”



Counting distinct values.

Count distinct categories

A one-line question, a one-line answer, straight from the live model.

What you'll carry from this chapter

- Data comes in three shapes: structured, semi-structured, unstructured.
- Business data hides in ERP, CRM, databases, spreadsheets, logs, and APIs.
- Always **profile** before you build — know the shape and the gaps.
- Don't forget the metadata: the data about your data.

Next: the unglamorous, essential work of cleaning dirty data — and how a few calculated columns fix a mess in seconds.

7. Dirty Data and How to Clean It

Here is a truth nobody puts on the job description: **most of an analyst's time is spent cleaning data**. Real-world data is messy — misspelled, duplicated, missing, inconsistent. Garbage in, garbage out. Before you can find any truth, you have to sweep the floor.

The usual mess

Every analyst meets the same cast of problems:

- **Inconsistent text** — “Milan”, “milan”, “MILANO”, “Milano”. Four values, one city.
- **Mixed formats** — dates as 03/04/2025 and 2025-04-03 in the same column.
- **Missing values** — blank cities, empty categories, no phone number.
- **Duplicates** — the same customer twice under two emails.
- **Wrong types** — a number stored as text, so it won't add up.
- **Out-of-place values** — a negative sale that is really a refund.

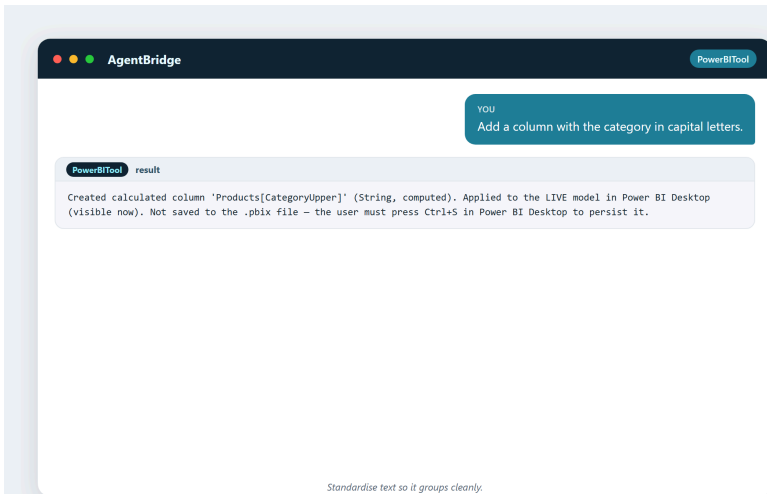
None of these are dramatic. All of them will quietly ruin an analysis if you ignore them.

Cleaning with calculated columns

In Power BI, a lot of cleaning is done with **calculated columns** — new columns you create with a formula that fixes or standardises existing data. This is exactly where the assistant shines: you describe the fix in plain words, it writes the formula and applies it live.

Standardise text. A person asked:

“Add a column with the category in capital letters.”

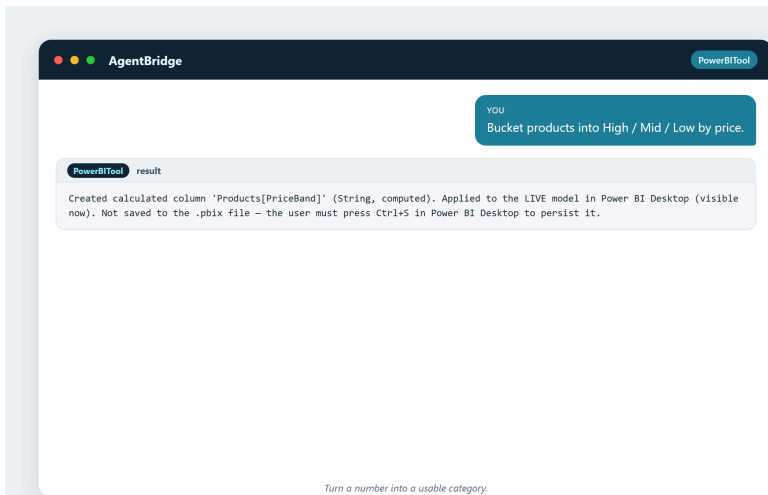


Category in capitals

Now “kitchen”, “Kitchen”, and “KITCHEN” all become “KITCHEN” and group together. One small column, one whole class of problem gone.

Turn a number into a usable band.

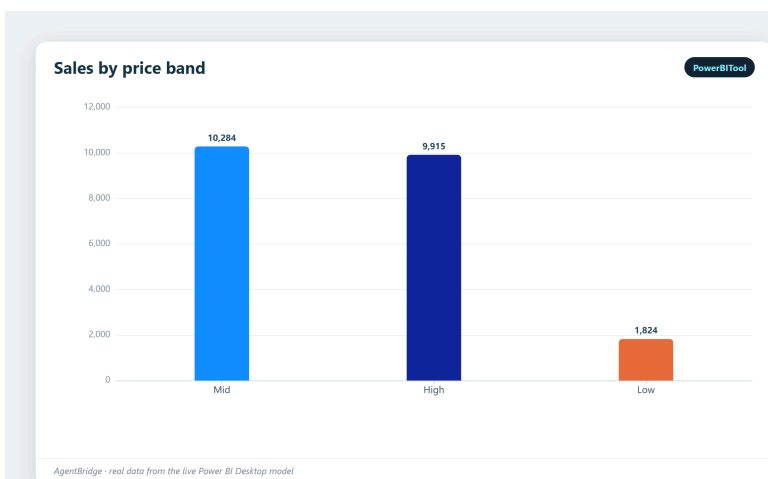
“Bucket products into High / Mid / Low by price.”



Price band

A raw price of €249 is hard to group by. A band of “High” is easy to chart and easy to talk about. This is one of the most useful tricks in analysis: turning a continuous number into a friendly category.

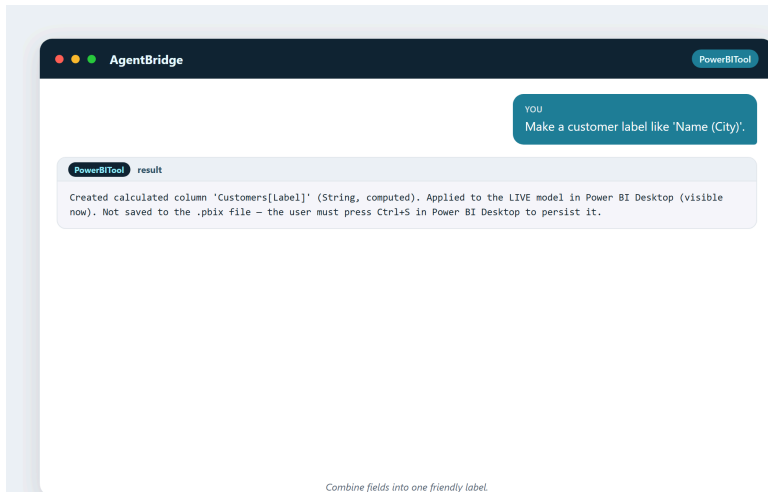
And here is what that band buys you — sales grouped and charted by price band:



Sales by price band — bar chart

Combine fields into a label.

“Make a customer label like ‘Name (City)’.”



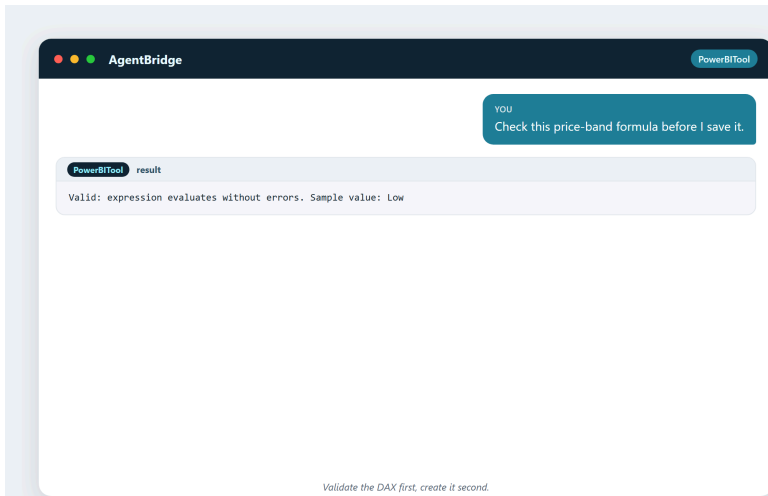
Customer label

Now every customer has one clean display label, built from two columns, without anyone typing a thing.

Check before you commit

A good habit: **validate the formula before you save it**. The assistant can test a formula and show you a sample value, so you know it works before it becomes part of the model.

“Check this price-band formula before I save it.”



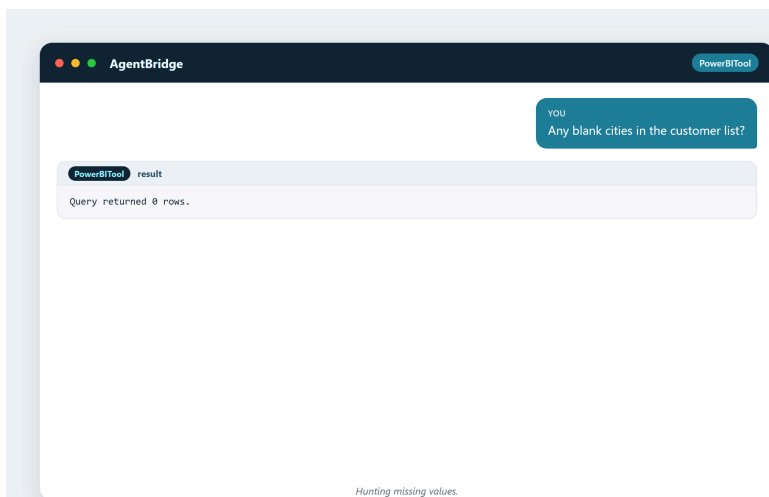
Validate the price band

It returns “Valid” with a sample value. No surprises later.

Hunting the blanks

Missing values are silent killers. A blank city means a customer vanishes from every map. The assistant can hunt them down:

“Any blank cities in the customer list?”



Blank cities check

If the result is empty, you are clean. If not, you know exactly where the holes are before they mislead a chart.

A curiosity: 80/20 of the job

Ask any experienced analyst how their time splits and you'll hear a version of the same joke: **80% of data science is cleaning data, and the other 20% is complaining about cleaning data.** It's a cliché because it's true. The analysts who are good at cleaning are worth their weight — because a beautiful model built on dirty data is a beautiful way to be wrong.

When cleaning is never enough

Sometimes the data is too far gone — missing 40% of a key field, or two systems that simply disagree. A good analyst knows when to stop cleaning and escalate: *fix this at the source*, or *collect better data next time*. Cleaning is a tool, not a religion.

What you'll carry from this chapter

- Real data is dirty; cleaning is most of the job.
- Calculated columns fix text, band numbers, and build labels in seconds.
- Validate a formula before you commit it.
- Hunt the blanks before they mislead a chart.
- Know when to stop cleaning and fix the source.

Next: how data pieces connect — tables, keys, and relationships — the wiring that makes analysis possible.

8. Models, Tables, and Relationships

A pile of tables is not a model. A **model** is what you get when you tell the computer how the tables *relate* to each other. Those connections — the wiring — are what let you ask a question in one place and get an answer that spans many tables. This chapter is about that wiring.

Tables, rows, and keys

Every table has **rows** (one record each) and **columns** (one attribute each). The magic is in the **key** — a column that uniquely identifies each row. A customer ID, a product code, an order number. Keys are how tables recognise each other.

- A **primary key** is the unique ID in a table (one row per customer).
- A **foreign key** is a column in another table that points to that ID (each sale stores the customer's ID).

The relationship: how two tables talk

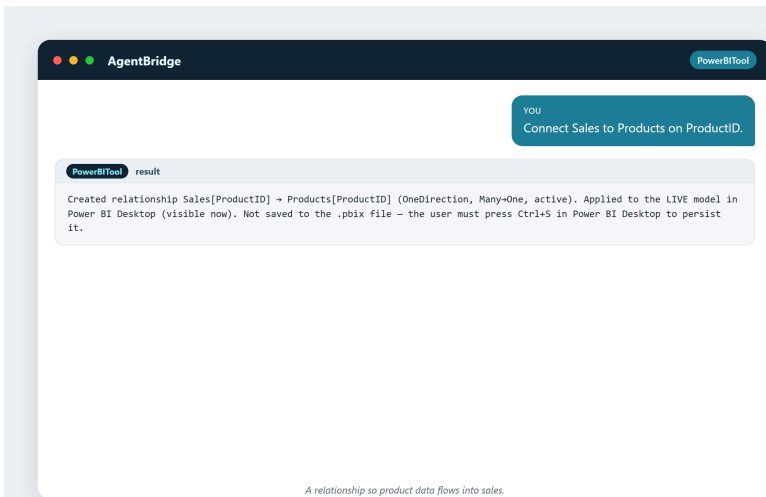
A **relationship** connects a foreign key to a primary key. Once connected, the computer can answer questions that cross tables: “which product was in this sale?” “which city did this customer live in?” — without you ever merging files by hand.

The most common kind is **many-to-one**: many sales point to one product. Each sale has a product ID; the product table has one row per product. Many sales, one product. That is the backbone of almost every business model.

Wiring it up, live

Here is the assistant creating a relationship from a plain request:

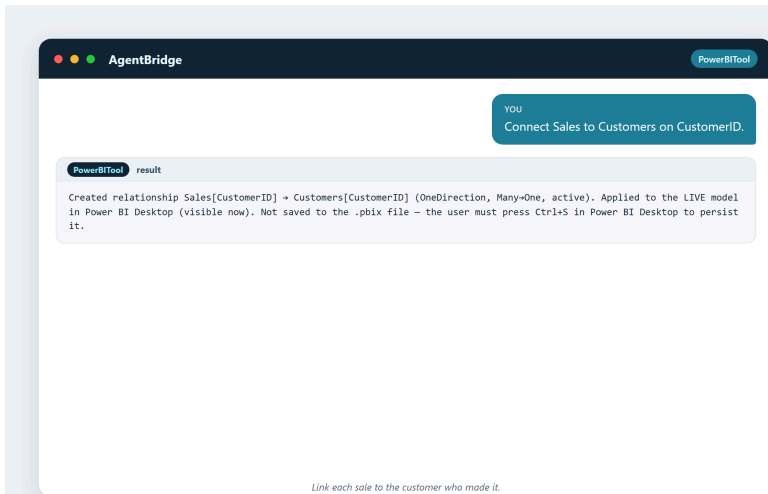
“Connect Sales to Products on ProductID.”



Sales to Products relationship

The tool reports the direction (Many→One) and confirms it’s live in Power BI Desktop. Then the customer link:

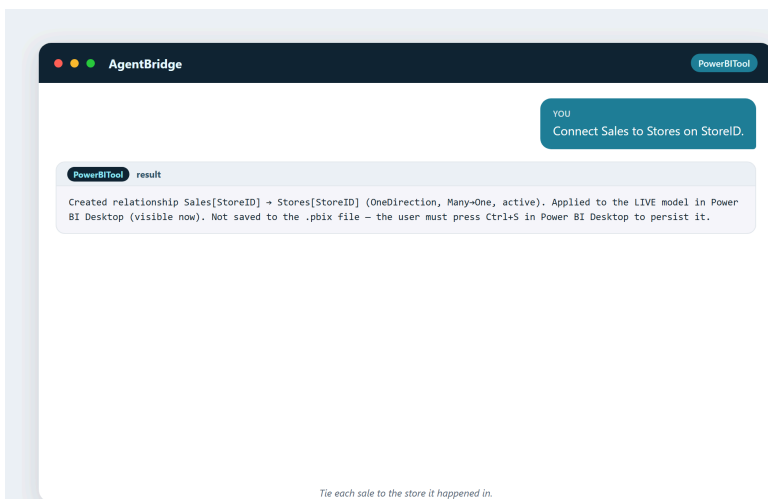
“Connect Sales to Customers on CustomerID.”



Sales to Customers relationship

And the store link:

“Connect Sales to Stores on StoreID.”

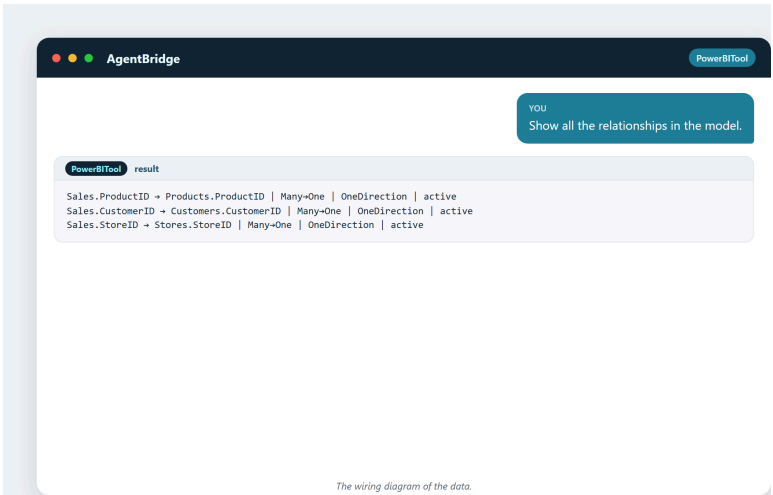


Sales to Stores relationship

Three sentences, and the model now has a spine. Every later question about “sales by product”, “sales by customer”, “sales by store” works because of these three lines.

Seeing the whole wiring

“Show all the relationships in the model.”



All relationships

Three clean Many→One relationships, all active. This is the wiring diagram — the thing you check first when a number looks wrong.

The star schema: the shape you want

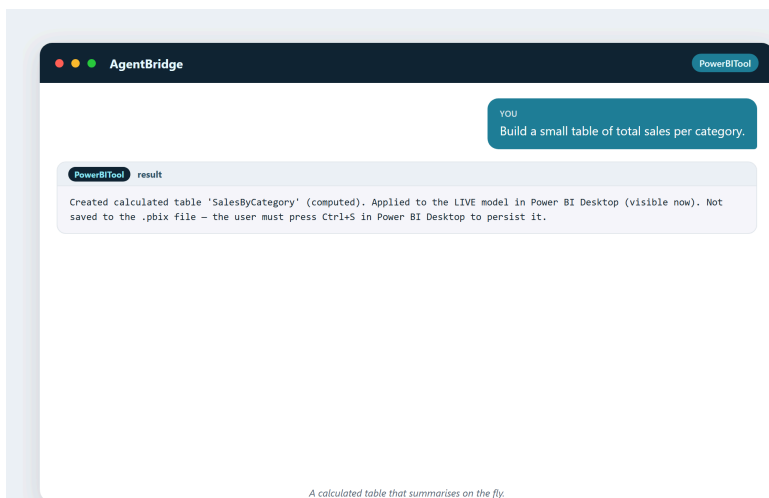
Pull it together and you get the most famous shape in business data: the **star schema**. A fact table in the middle (Sales), surrounded by dimension tables (Products, Customers, Stores, Date). The fact table holds the numbers; the dimensions hold the descriptive detail. Drawn out, it looks like a star.

Why is it so loved? Because it is simple, fast, and matches how people ask questions. “Sales by category” is just the fact table reaching over to the product dimension. Almost every good BI model is a star, or a field of stars.

A calculated table: summarising on the fly

Sometimes you want a small summary table built from the model itself:

“Build a small table of total sales per category.”



Sales by category table

A new table, computed live, that rolls the detail up into a tidy summary. Handy for a quick report or a snapshot.

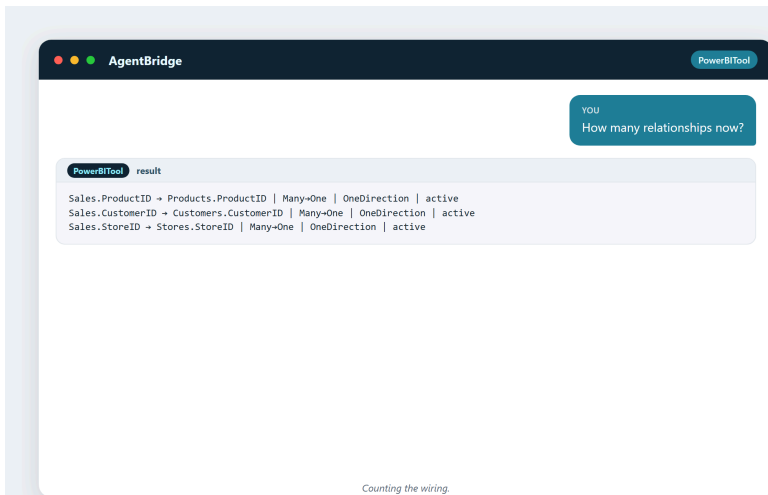
A curiosity: the many-to-many trap

The most dangerous relationship is **many-to-many** without care — many products in many promotions, many students in many classes.

Get it wrong and your totals double-count or vanish. The fix is a “bridge” table in the middle. If your numbers suddenly look inflated, a sloppy many-to-many is the first suspect.

Try it yourself

“How many relationships now?”



Count relationships

A quick check that the wiring is all there.

What you’ll carry from this chapter

- A model is tables plus the relationships between them.
- Keys (primary and foreign) are how tables recognise each other.
- Many-to-one is the backbone of business data.
- The star schema is the shape you usually want.
- Beware sloppy many-to-many — it inflates totals.

Next: asking the data questions directly — SQL and DAX, the two languages of getting answers.

9. Asking Questions with SQL and DAX

Once the data is in and wired up, you ask it questions. There are two languages you should recognise: **SQL** for databases, and **DAX** for Power BI. You do not need to write them by hand anymore — the assistant does — but you should understand what they do, so you can ask well and read the answers.

SQL: the language of databases

SQL (Structured Query Language) has been the way to talk to databases since the 1970s. It reads almost like English:

- **SELECT** — which columns you want
- **FROM** — which table
- **WHERE** — which rows to keep
- **GROUP BY** — how to bundle and total

A classic: “*total sales by region*” is a **SELECT**, a **JOIN** to the region table, and a **GROUP BY**. SQL is everywhere — if your company has a database, SQL is how you read it.

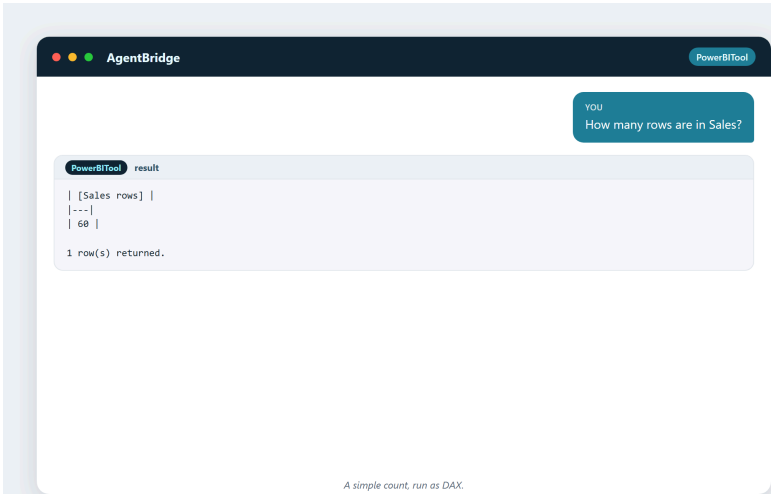
DAX: the language of Power BI

DAX (Data Analysis Expressions) is the language inside Power BI. It looks different from SQL but does the same job: ask for a number, get a number. DAX is built around **measures** — named calculations you can reuse. **SUM**, **AVERAGE**, **COUNT**, **CALCULATE** are its workhorses.

The beautiful thing: you do not have to type DAX. You describe the answer you want, and the assistant writes and runs the DAX for you. Let's watch.

Counting and summing

“How many rows are in Sales?”

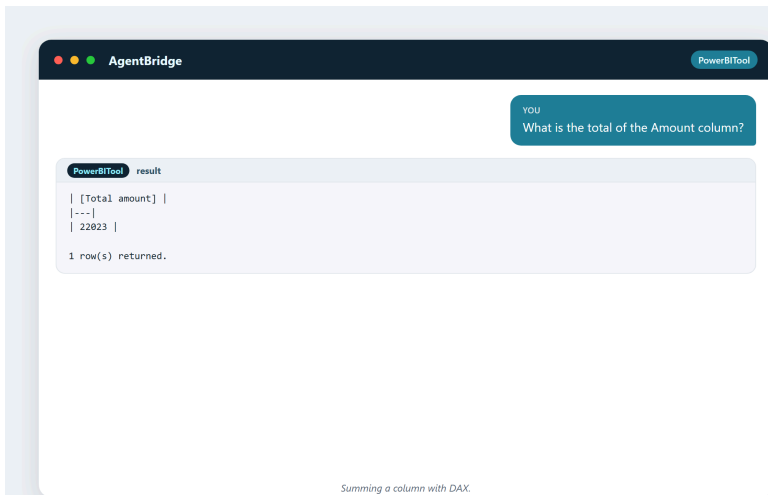


A simple count, run as DAX.

Count rows

Sixty sales records. Simple, instant.

“What is the total of the Amount column?”

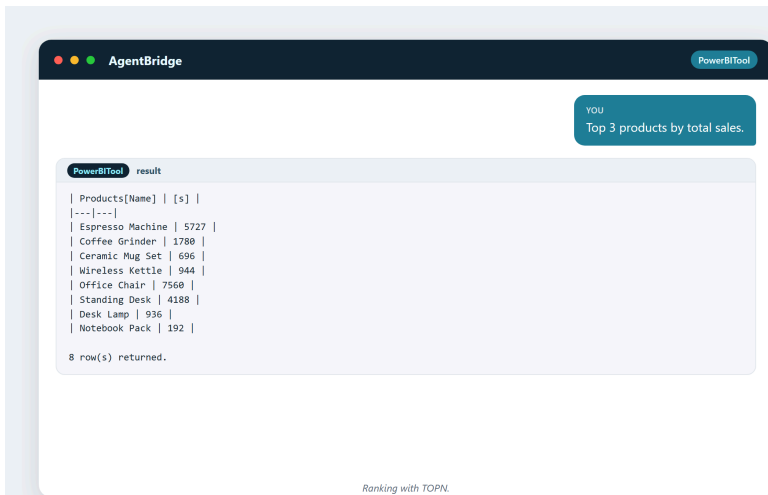


Sum amount

€22,023 in total sales. The kind of number that used to mean a pivot table and ten minutes; now it's one sentence.

Ranking and filtering

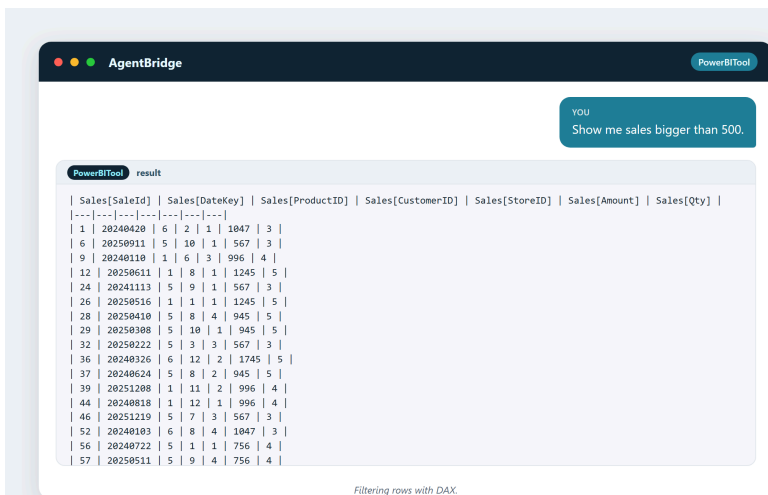
“Top 3 products by total sales.”



Top products

The assistant ranks them for you. (Note how it returns the full ranking so you can see the whole picture, not just the top slice.)

“Show me sales bigger than 500.”

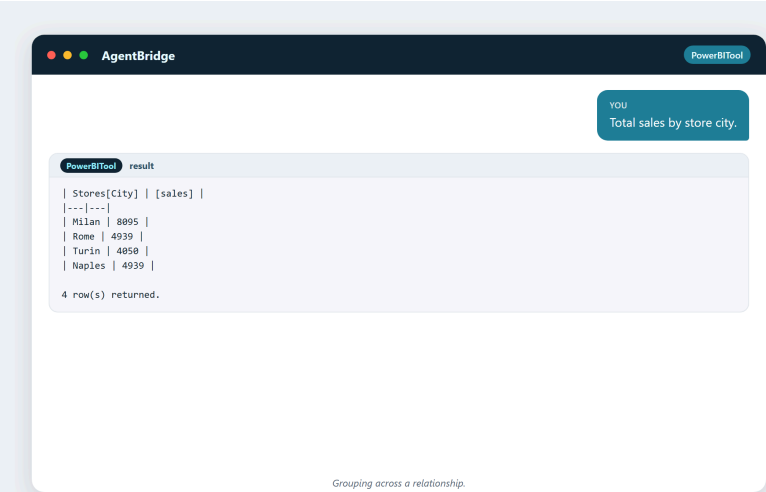


Sales over 500

A filter, run live, returning every big sale. This is how you find the outliers and the whales.

Grouping across a relationship

“Total sales by store city.”

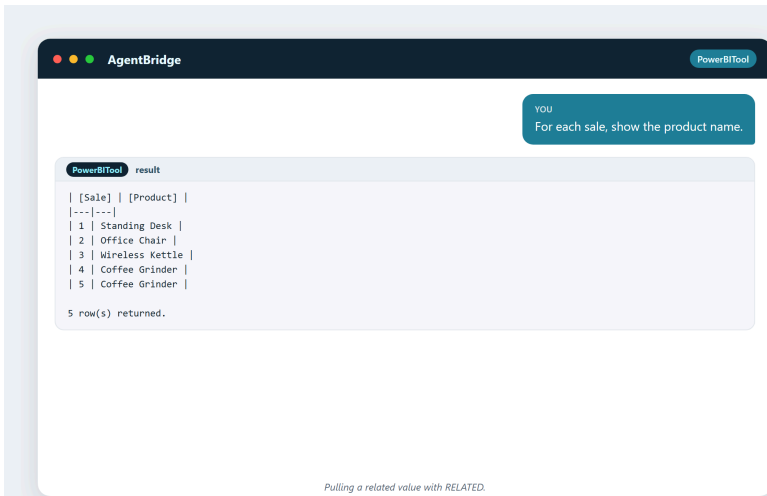


Sales by city

This is the moment the model pays off: the assistant reaches across the Sales→Stores relationship and totals by city, no manual merge needed.

Pulling a related value

“For each sale, show the product name.”

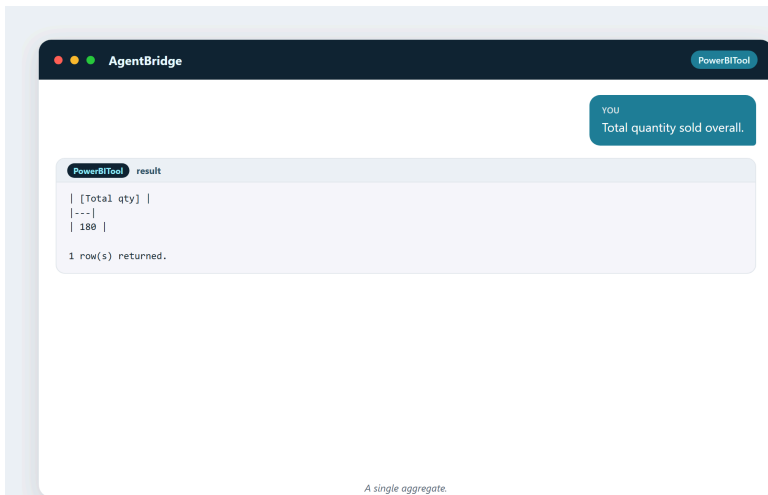


Related product name

Using `RELATED`, the assistant pulls the product name onto each sale — the kind of thing that in Excel means a `VLOOKUP` and a prayer.

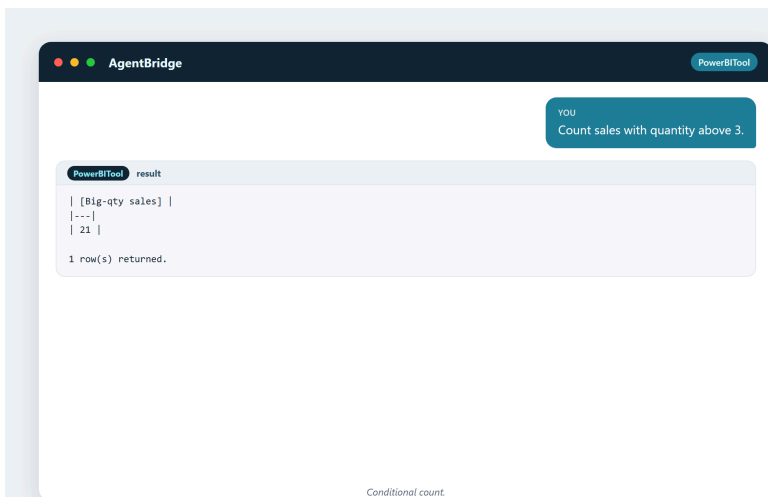
A few more, because they're easy

“Total quantity sold overall.”



Total quantity

“Count sales with quantity above 3.”



Count big-quantity sales

Each one a plain sentence, each one a real DAX query run against the live model.

A curiosity: DAX's scary reputation

DAX has a reputation for being hard. It is not hard to *use* — it is hard to *master the filter context*, the subtle rule about what data a calculation sees at any moment. But here is the secret of this book: you don't have to master it. You describe the answer; the assistant writes the DAX and handles the context. The difficulty moves from your shoulders to the tool's.

What you'll carry from this chapter

- SQL talks to databases; DAX talks to Power BI.
- Both read close to English: select, filter, group, total.
- You describe the answer; the assistant writes and runs the query.
- Relationships make cross-table questions trivial.
- The hard part of DAX is now the tool's problem, not yours.

Next: the spreadsheet — where everyone starts, and the wall where everyone needs something more.

10. Spreadsheets, Excel, and Hitting the Wall

No book about data analysis can skip the spreadsheet. It is where almost everyone starts, and for good reason — it is brilliant. But it is also where people hit a wall, and knowing where that wall is tells you when to move on.

Why spreadsheets won

The spreadsheet is one of the most successful pieces of software ever made. Its genius is that it is **direct manipulation**: you type a number in a box, and the boxes that depend on it update instantly. No code, no compile, no waiting. You see your work and your result side by side.

Spreadsheets gave ordinary people the power to model: budgets, forecasts, schedules, price lists. Before the spreadsheet, that power lived only in mainframes and only with programmers. After it, anyone with a PC could do it.

The pivot table: analysis in a box

The **pivot table** is the spreadsheet's superpower. Drag a few fields and it summarises thousands of rows: sales by month, by product, by region. For a huge share of business analysis, a pivot table is the whole job. If you can pivot, you can analyse.

The wall

But spreadsheets have a ceiling, and every analyst eventually hits it:

- **Size** — past a million rows, Excel groans, slows, and crashes.
- **Fragility** — one deleted cell, one broken formula, and the whole workbook is quietly wrong. There is no safety net.
- **No relationships** — joining two tables means VLOOKUP, and VLOOKUP breaks the moment data moves.
- **Version chaos** — “Budget_FINAL_v3_really_final.xlsx” edited by five people, all disagreeing.
- **No refresh** — a report that gets updated by copy-paste every Monday is a report that is wrong every Tuesday.
- **No sharing story** — a spreadsheet is a file, not a live dashboard others can trust and explore.

If your Monday morning is “open the file, paste the new data, drag the formulas, re-save, email it” — you are doing by hand what a proper model does by itself.

The spreadsheet vs. the model

Here is the difference in one line:

A spreadsheet stores numbers in cells. A model stores *logic* and computes the numbers fresh every time.

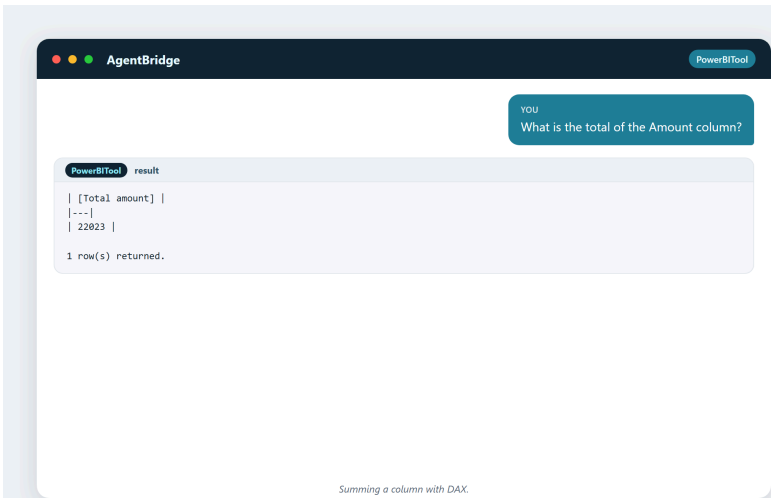
In a spreadsheet, the number *is* the answer, sitting in a cell, rotting. In a model, the answer is recomputed from the data and the rules, every time you look, always current.

The same question, two ways

In Excel, “total sales” means a SUM formula over a column, correct only as long as nobody touches the rows. In a model, it is a measure —

SUM(Sales[Amount]) — that recomputes on demand and can be sliced by any dimension without a single new formula:

“What is the total of the Amount column?”



Total amount as a measure

Same number, but now it lives in a model that can answer “by region”, “by month”, “by customer” without any extra work — because the logic is stored, not the result.

A curiosity: the spreadsheet that lost a billion

In 1998, a spreadsheet error helped cause a \$1.2 billion loss at a major financial fund (LTCM), and countless companies have been burned by a single wrong cell. In 2008, a famous research paper on debt and growth was found to have a spreadsheet error — an accidentally excluded set of rows — that flipped its conclusion and had influenced real policy for years. Spreadsheets are powerful, and that is exactly why their mistakes are dangerous. A model with tested logic is safer than a spreadsheet with a hidden + where a - should be.

When to stay, when to go

Stay in the spreadsheet when: the data is small, the job is one-off, you are sketching. Move to a model when: the data is big, the report repeats, more than one person touches it, or you need it to be *right* and *current*. The assistant and Power BI are how you cross that bridge without pain.

What you'll carry from this chapter

- Spreadsheets are brilliant for small, direct, one-off work.
- The pivot table is a genuine superpower.
- The wall: size, fragility, no relationships, version chaos, no refresh.
- A model stores logic, not frozen results.
- Move on when the report repeats or the data grows.

Part II is done — you know where data comes from, how to clean it, how to wire it, and how to ask it questions. Next we turn data into meaning: the metrics and kinds of analysis that drive decisions.

11. The Metrics That Matter

A metric is a number you watch to know how the business is doing. Choose the right ones and you can steer. Choose the wrong ones and you can drive straight off a cliff while the dashboard glows green. This chapter is about picking the numbers that actually matter — and building them with the assistant.

What makes a metric worth watching

A good metric passes three tests:

1. **It moves when the business moves.** If the business gets worse, the number should get worse.
2. **You can act on it.** A number you can only admire is decoration.
3. **It's honest.** It can't be gamed into looking good while things rot.

A **vanity metric** fails these. “Total registered users since 2010” only ever goes up. It feels great and means nothing. Watch rates and changes, not ever-growing totals.

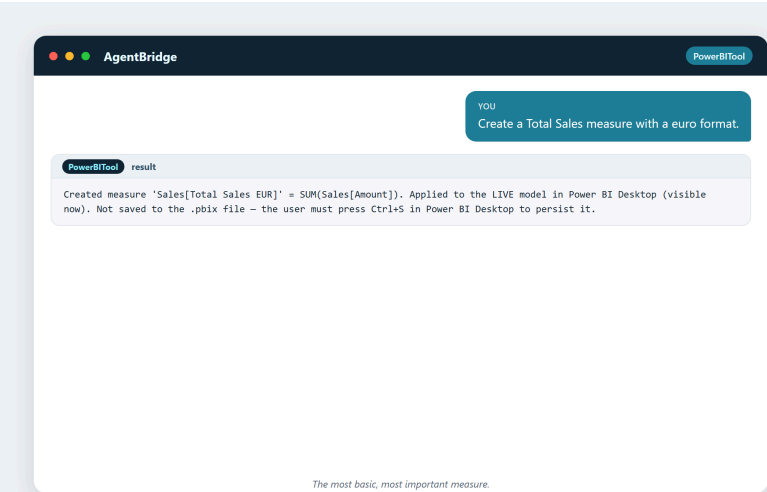
The core sales metrics

Every business that sells things watches a similar set:

- **Total sales** — the headline revenue.
- **Units sold** — how much stuff moved.
- **Orders** — how many transactions.
- **Average order value** — revenue per order.
- **Active customers** — how many people actually bought.
- **Largest sale** — the biggest single line (for spotting whales).

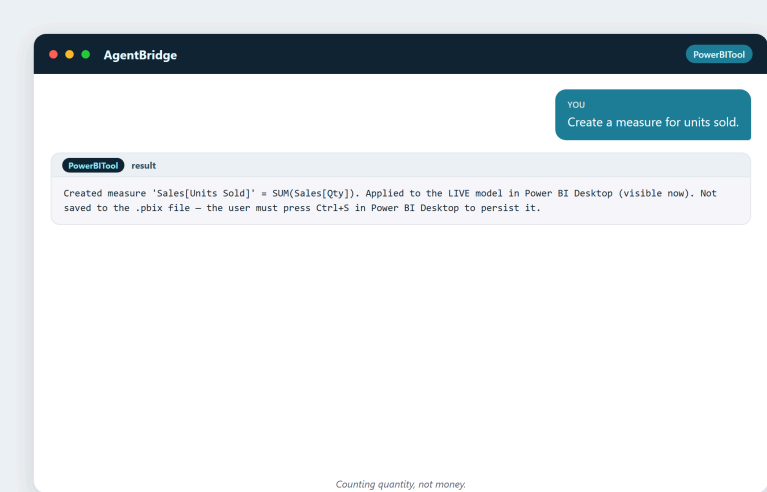
The assistant builds each of these from a plain request. Watch a set appear:

“Create a Total Sales measure with a euro format.”



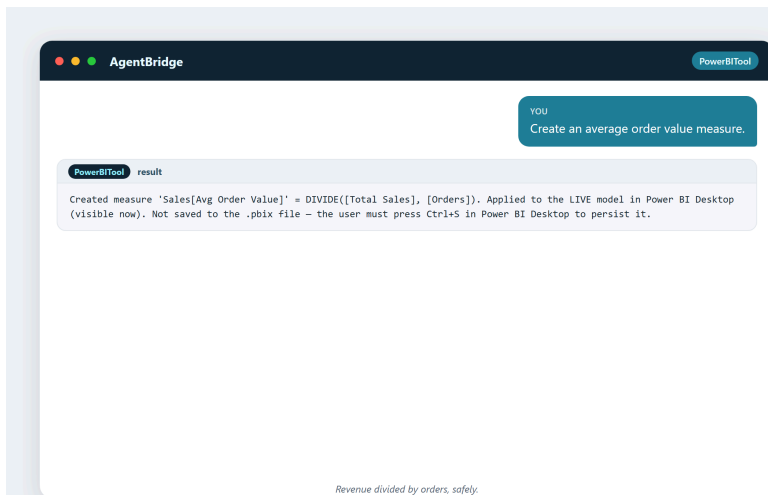
Total Sales measure

“Create a measure for units sold.”



Units Sold measure

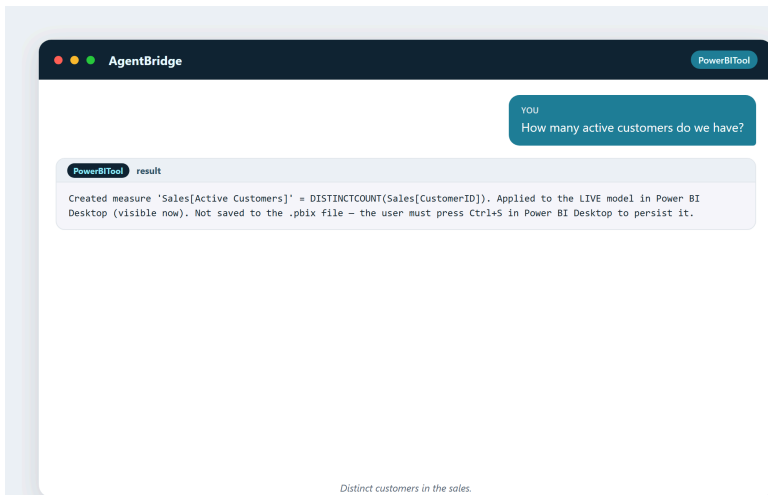
“Create an average order value measure.”



Average order value

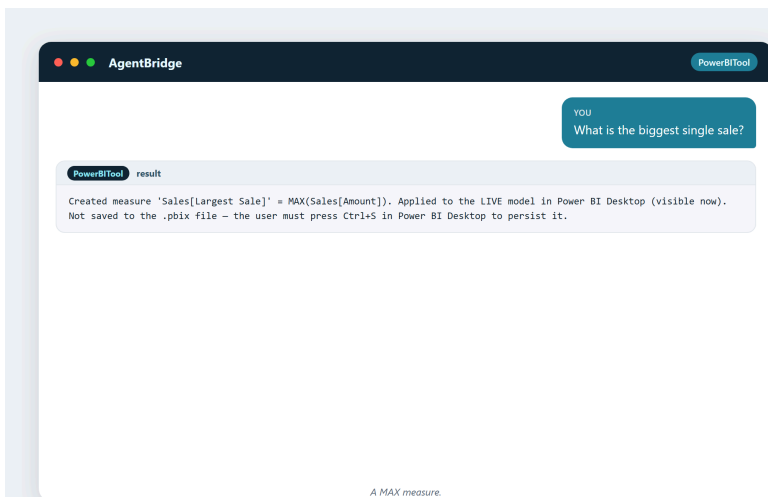
Notice the average order value uses `DIVIDE`, not a slash. That is deliberate — `DIVIDE` handles the case where the bottom is zero without crashing. A small safety habit that saves you from `#DIV/0!` errors later.

“How many active customers do we have?”



Active customers

“What is the biggest single sale?”



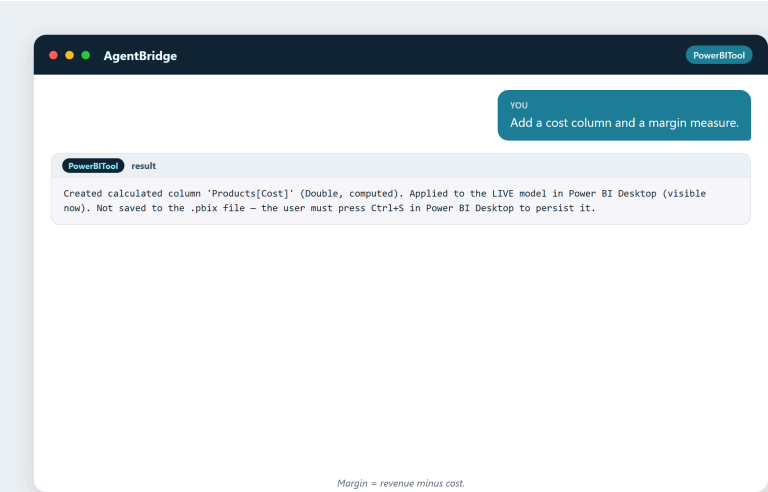
Largest sale

In a handful of sentences, the whole core KPI set exists, live in the model.

Money metrics: margin and share

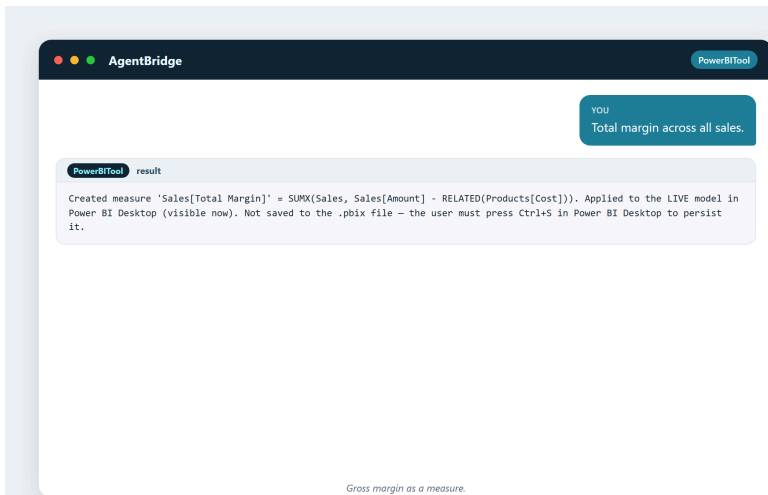
Revenue is vanity; profit is sanity. To know what you *keep*, you need cost:

“Add a cost column and a margin measure.”



Cost column

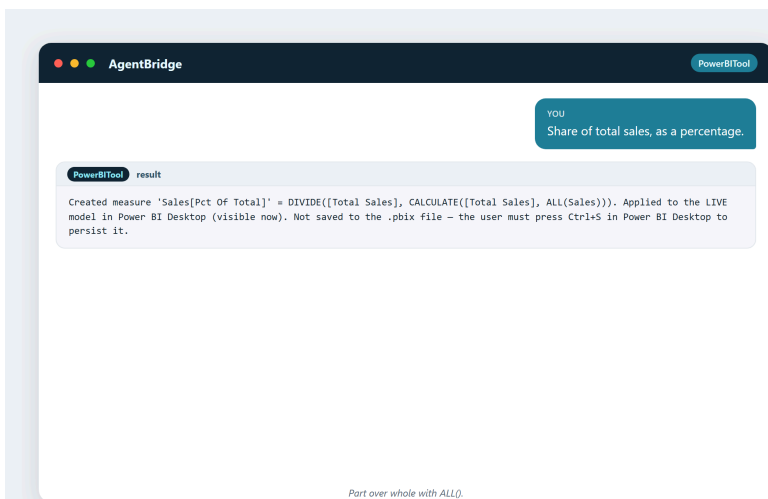
“Total margin across all sales.”



Total margin

And to see how a slice compares to the whole:

“Share of total sales, as a percentage.”

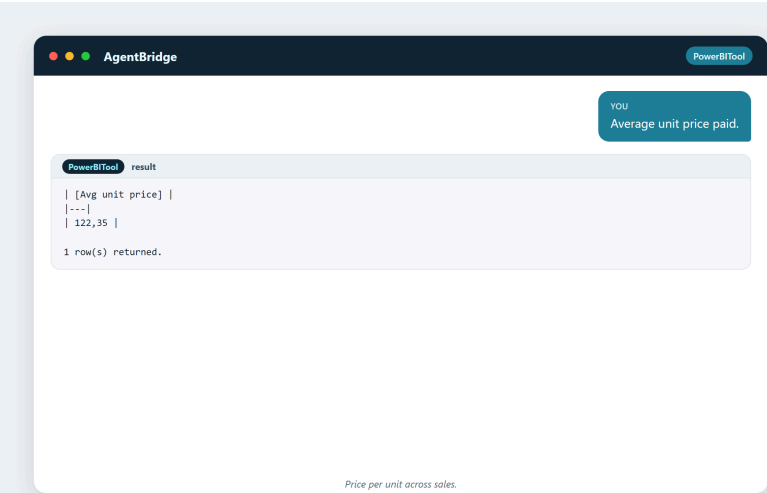


Percent of total

A percentage-of-total is one of the most-used metrics in reporting — it turns any number into “how big is this compared to everything?”

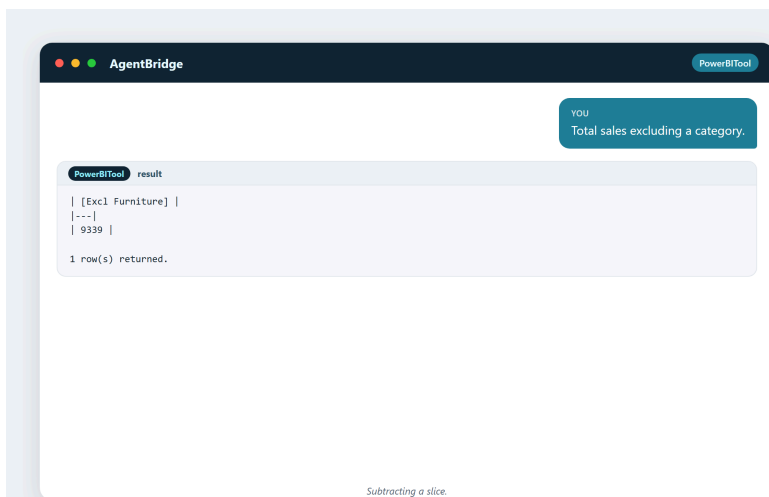
A few more quick ones

“Average unit price paid.”



Average unit price

“Total sales excluding a category.”



Sales excluding a category

Each is a plain sentence, each is a real measure in the live model.

A curiosity: the metric that backfired

When the Soviet Union measured nail production by **quantity**, factories made tiny useless nails by the million. When they switched to measuring by **weight**, they made a few enormous nails. Same goal, different metric, different absurdity. The lesson every analyst must learn: **you get what you measure**, so measure carefully — ideally a metric that can only improve if the business truly improves.

What you'll carry from this chapter

- Choose metrics that move with the business, that you can act on, that can't be gamed.
- Avoid vanity metrics (ever-growing totals).

- The core sales set: revenue, units, orders, average order, active customers.
- Margin and share-of-total turn revenue into meaning.
- You get what you measure — measure wisely.

Next: the four kinds of analysis, from “what happened” all the way to “what should we do.”

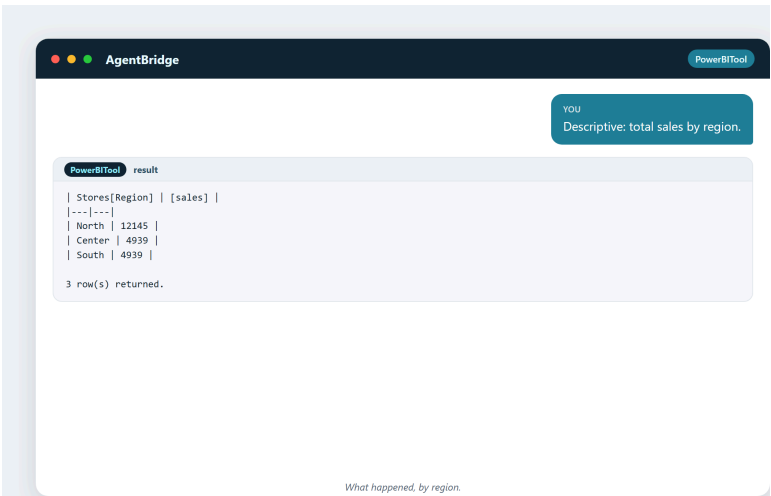
12. Four Kinds of Analysis

Every analysis you will ever do falls into one of four kinds, arranged by how much they ask of the data. They climb a ladder: from looking back, to explaining why, to guessing forward, to recommending what to do. Knowing which kind you are doing tells you how hard to push and how much to trust the answer.

1. Descriptive — what happened?

The simplest and most common. You describe the past. “Sales were €22,023. The North did €12,145.” No explanation, no prediction — just the facts, clearly.

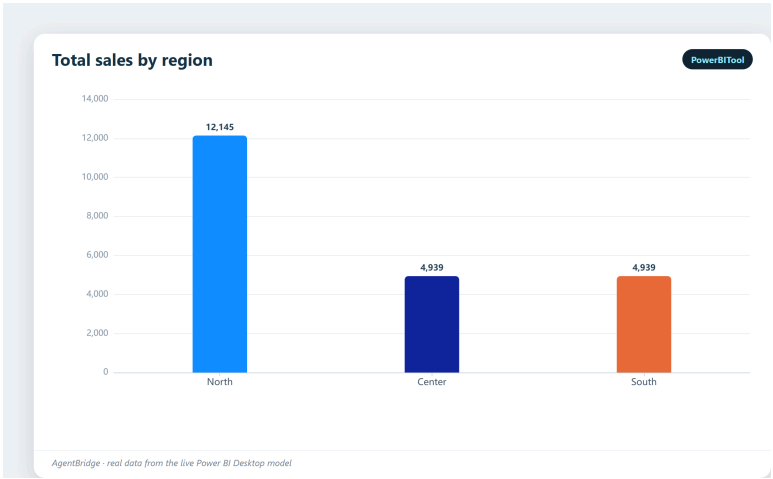
“Descriptive: total sales by region.”



Descriptive by region

Most dashboards are descriptive. They answer “how are we doing?” and they are the foundation everything else stands on.

The regional breakdown, as a chart:

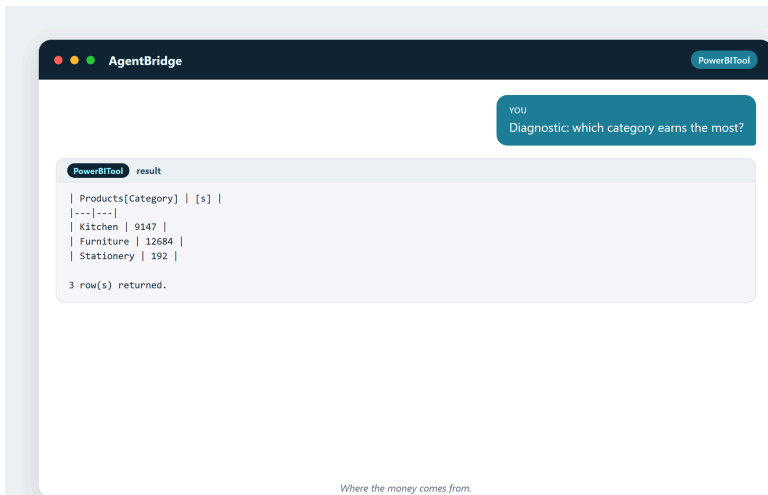


Sales by region — bar chart

2. Diagnostic — why did it happen?

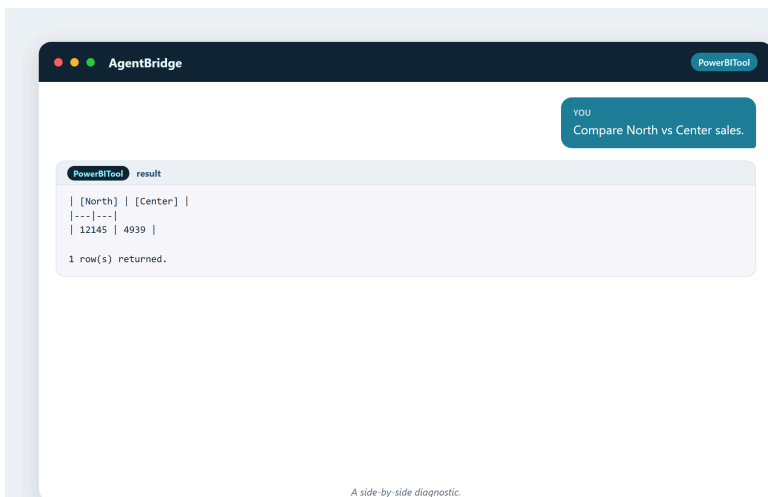
Now you dig. Something changed, and you want the cause. You slice, compare, and cross-reference until the reason surfaces.

“Diagnostic: which category earns the most?”



Diagnostic by category

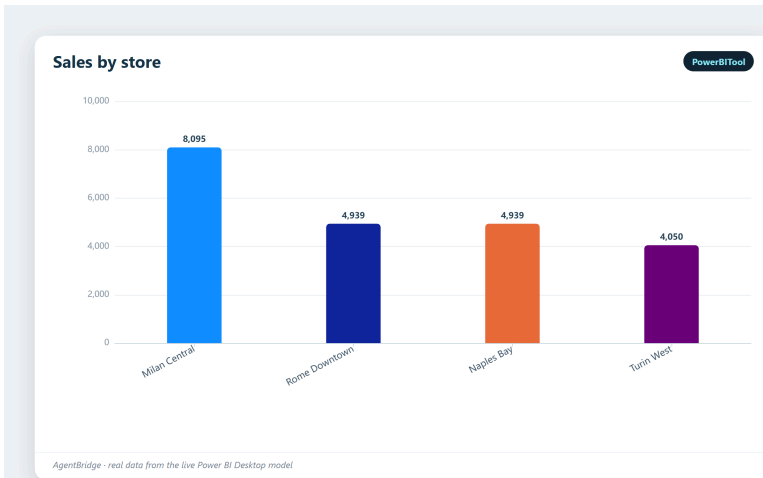
“Compare North vs Center sales.”



North vs Center

Diagnostic work is where the analyst earns their keep. Descriptive tells you the patient has a fever; diagnostic finds the infection.

The same diagnostic eye turned on stores:

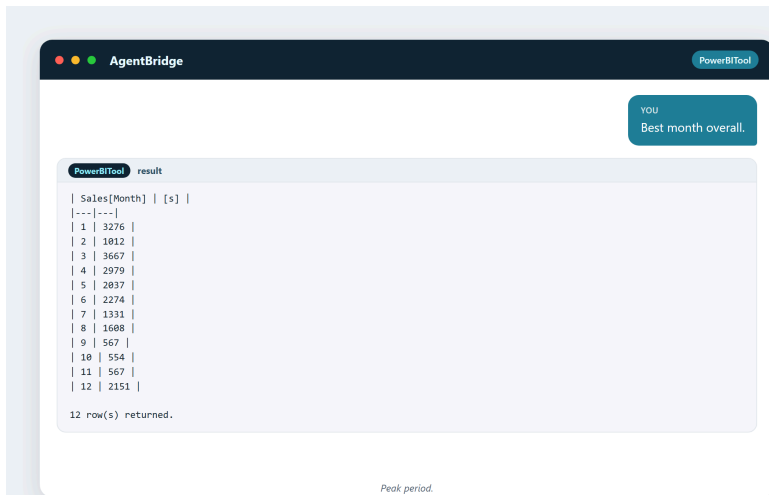


Sales by store — bar chart

3. Predictive — what will happen?

You use the past to guess the future. Demand next quarter, churn next month, sales by year-end. This usually needs statistics or machine learning, and it comes with a confidence range — a good prediction says “about 11,000, give or take.”

“Best month overall.”



Best month

A single-period view like this is the raw material for prediction: you see the pattern, then you project it forward.

4. Prescriptive — what should we do?

The top of the ladder. Given the prediction and the constraints, what action maximises the goal? Which price, which promotion, which stock level. Prescriptive analysis is the rarest and hardest, and it usually sits on top of the other three.

The ladder in one picture

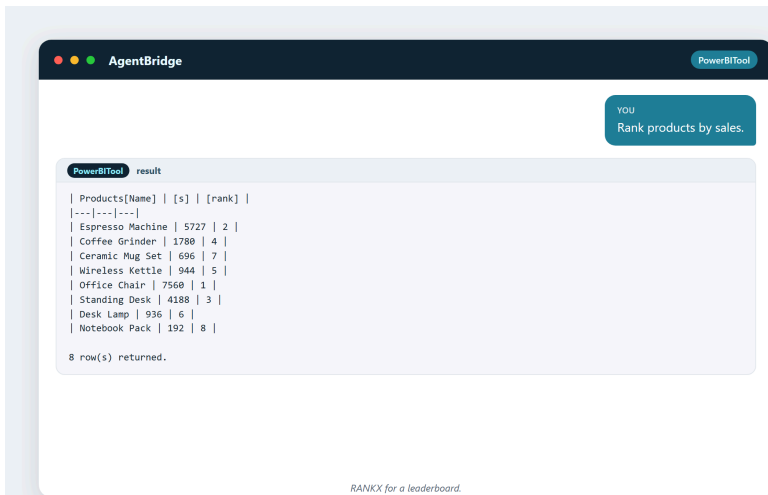
Kind	Question	Effort	Trust needed
Descriptive	What happened?	Low	High (it's just facts)
Diagnostic	Why?	Medium	Medium (watch for false causes)
Predictive	What next?	High	Lower (it's a guess with a range)
Prescriptive	What to do?	Highest	Lowest (it's a recommendation)

Notice the pattern: the higher you climb, the more value you add — and the less certain you are. A good analyst is honest about that trade-off.

Ranking: the analyst's favourite move

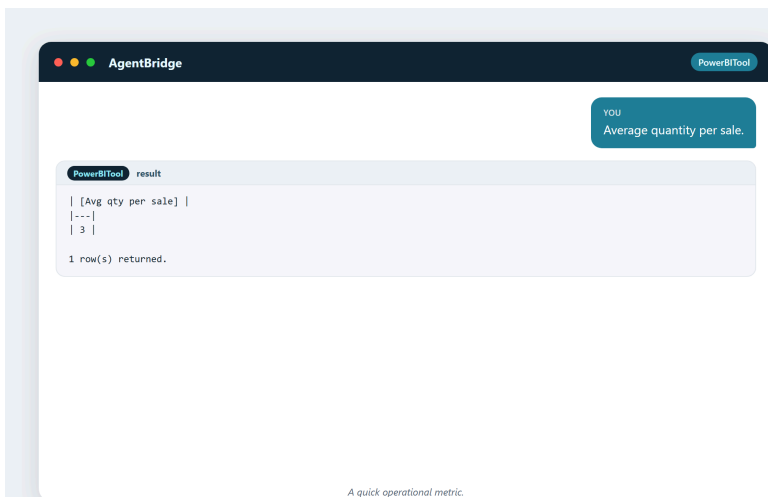
Ranking turns a flat list into a story. Who's first, who's last, who's improving.

“Rank products by sales.”



Rank products

“Average quantity per sale.”



Average quantity per sale

Ranking is descriptive, but it points the diagnostic work: the bottom of the list is where you look first for a problem.

A curiosity: the analytics maturity myth

Consultants love to sell a “maturity model” where you must climb from descriptive to prescriptive or you’re a laggard. In reality, **most businesses would be transformed just by getting descriptive and diagnostic right**. Don’t be ashamed of a good “what happened and why” — it is where 90% of the value lives. The predictive and prescriptive layers are the cherry on top, not the cake.

What you’ll carry from this chapter

- Four kinds: descriptive, diagnostic, predictive, prescriptive.
- Value and uncertainty both rise as you climb.
- Most value lives in descriptive + diagnostic.
- Ranking is the simplest way to find where to look.
- Be honest about how much to trust each kind.

Next: the customer — the most important subject of analysis there is.

13. Knowing Your Customers

If you analyse one thing in your business, analyse the customers. They are where the money comes from and where it leaks away. A little analysis of who buys, how much, and who is leaving is worth more than a thousand charts about products.

The customer questions

Every business secretly wants to know:

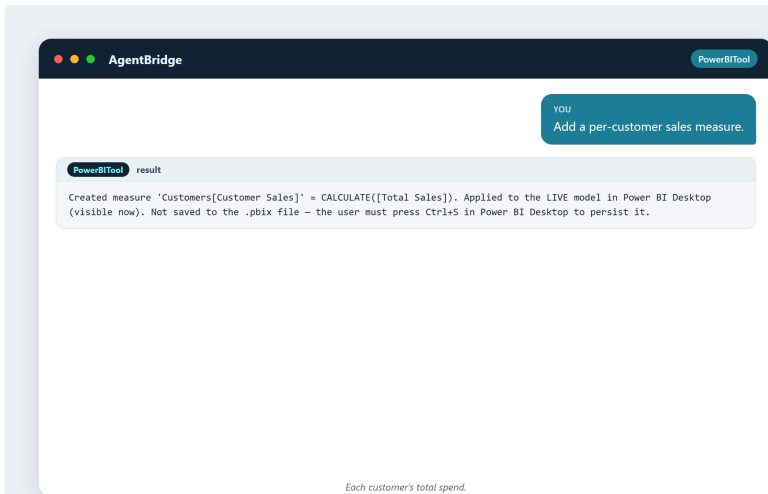
- Who are my best customers?
- Which customers are about to leave?
- Which ones are worth winning back?
- How much does each type of customer spend?
- Where do my customers come from?

Answer these and you can focus your money and effort where it pays.

A per-customer measure: the foundation

To analyse customers, you need a measure that rolls up each customer's spend:

“Add a per-customer sales measure.”

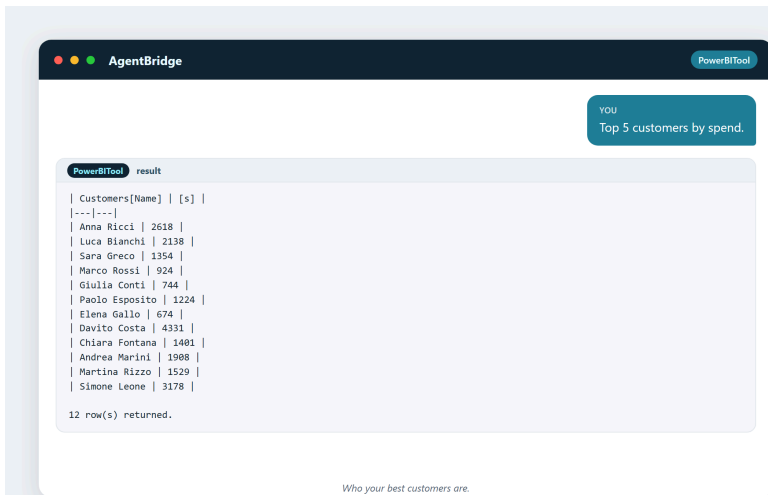


Customer sales measure

Now every customer has a “Customer Sales” number, and you can rank, segment, and compare them all day.

Who are the best customers?

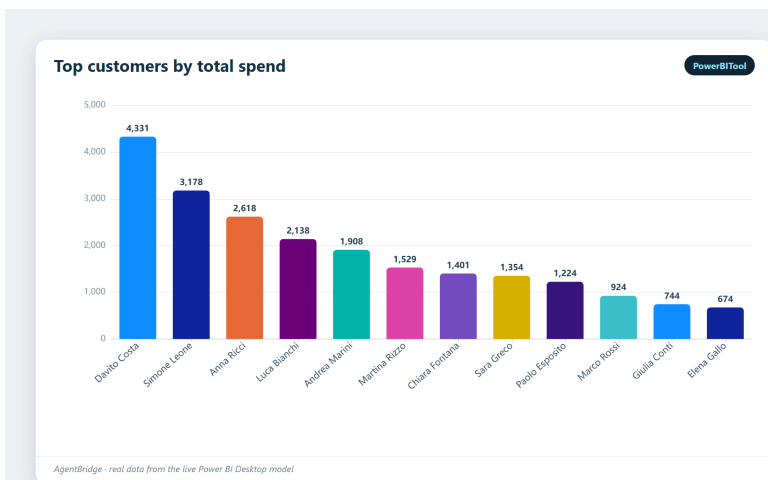
“Top 5 customers by spend.”



Top customers

Instantly you see who matters. In this data, a few customers carry a large share of revenue — which is normal and important. It means: **take care of these people**. Losing one of them hurts.

The same ranking, drawn as a chart:

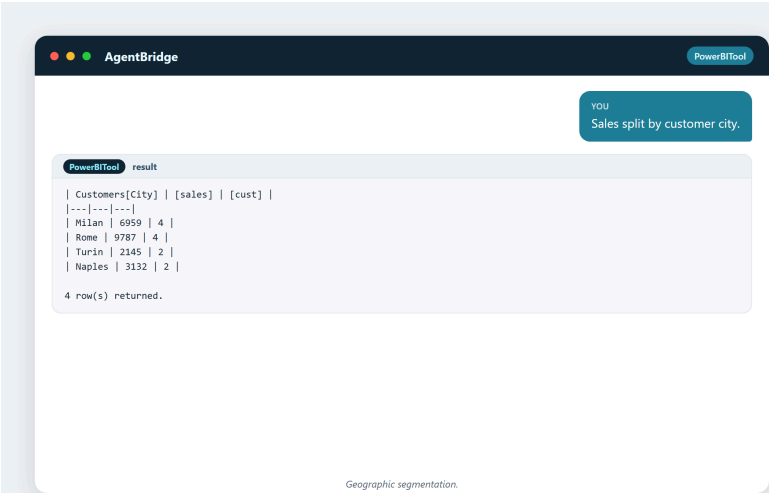


Top customers by spend — bar chart

Segmenting: slicing customers into groups

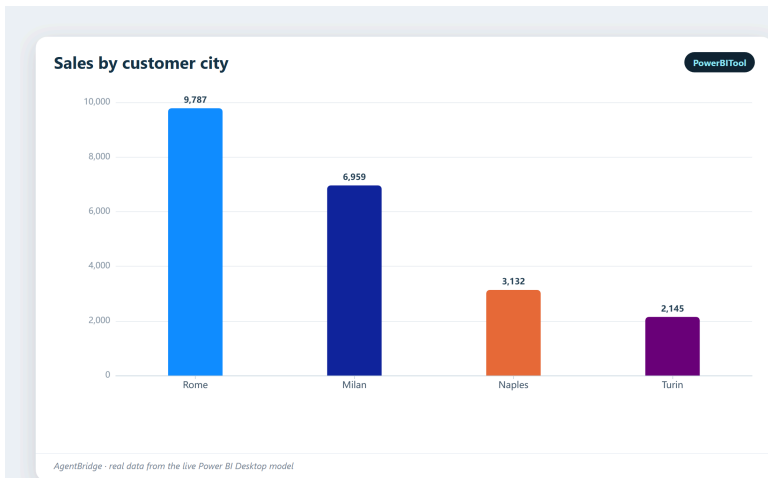
A **segment** is a group of customers who behave alike. You can slice by anything:

“Sales split by customer city.”



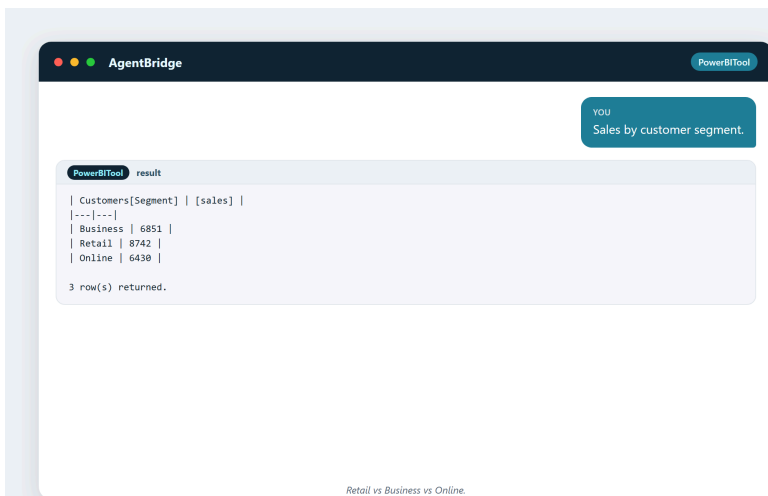
Sales by city

The city totals, as a chart:



Sales by city — bar chart

“Sales by customer segment.”

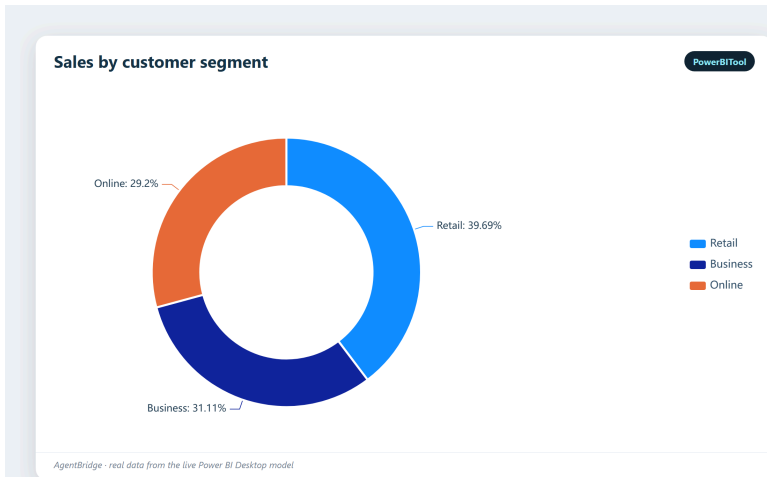


Sales by segment

By city, by segment (Retail / Business / Online), by how much they spend — each slice reveals a different opportunity. Maybe the Business

segment spends more per head and deserves a special offer. Maybe one city is underserved and could grow.

The segment split, as a donut:



Sales by segment — donut chart

A real framework: RFM

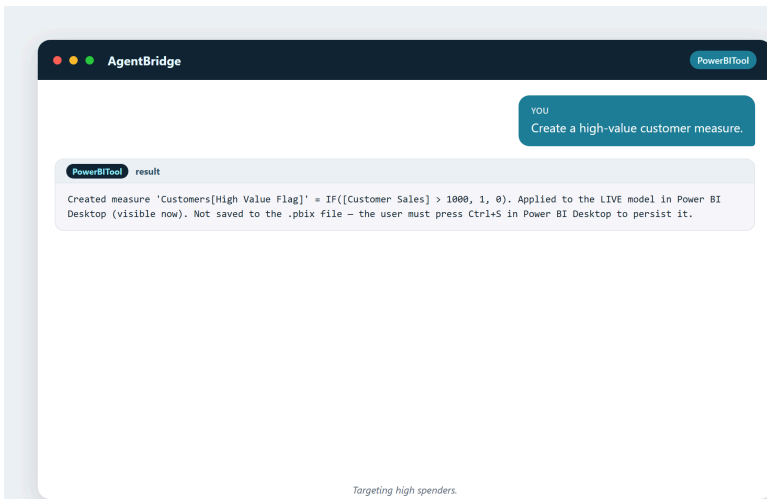
The most famous customer-analysis method is **RFM** — three letters:

- **Recency** — how recently did they buy?
- **Frequency** — how often do they buy?
- **Monetary** — how much do they spend?

Score each customer on all three and you can sort them into groups: the loyal champions, the occasional buyers, the ones drifting away. RFM is powerful because it is simple and it works: customers who bought recently, often, and a lot are your gold; customers who haven't bought in a while are the ones to win back before they're gone.

Finding the high-value ones

“Create a high-value customer measure.”

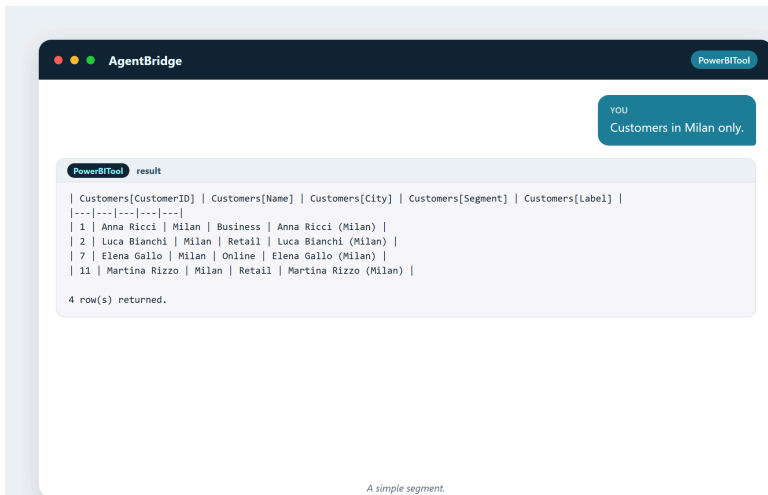


High value flag

A flag that marks customers above a spend threshold — the ones to protect, reward, and never lose.

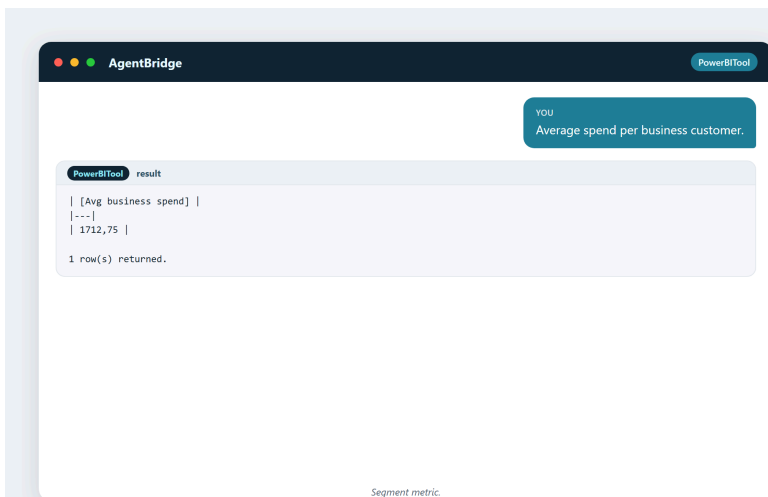
A couple more cuts

“Customers in Milan only.”



Milan customers

“Average spend per business customer.”



Average business spend

Each is a plain question, each is a real answer from the live model.

A curiosity: the churn you can't see

The most expensive customers to lose are the ones who leave **quietly** — they never complain, they just stop buying. By the time you notice, they're gone and you never learned why. A simple “customers who haven't bought in 90 days” report catches them early, while they can still be won back. Silent churn is the leak you can't afford to miss.

What you'll carry from this chapter

- Customers are the highest-value thing to analyse.
- A per-customer measure unlocks ranking and segmentation.
- RFM (Recency, Frequency, Monetary) is the classic, works-everywhere method.
- Segment by anything: city, type, spend.
- Watch for silent churn — customers who leave without complaining.

Next: time — the dimension that turns a snapshot into a story of change.

14. Trends, Time, and Seasonality

A number is a snapshot. Add time, and it becomes a story. Sales of €11,000 means little until you know whether that's up or down, and whether it's normal for this time of year. Time is the dimension that turns a photo into a movie, and almost every important business question lives in it.

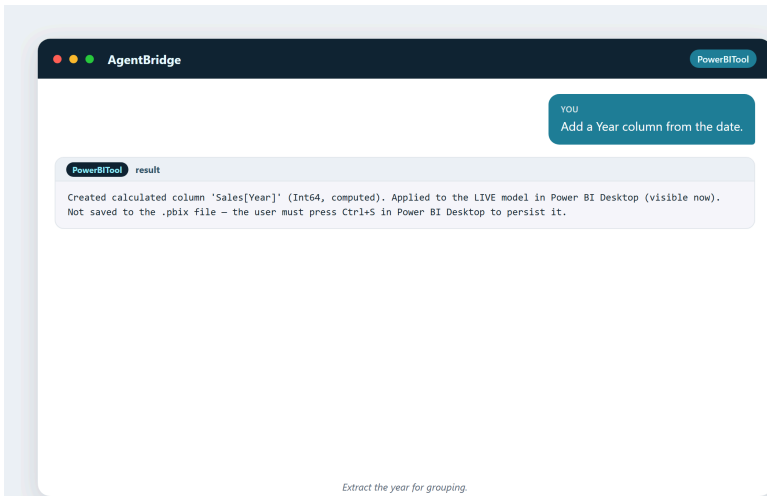
Why time is special

Time is the one dimension you cannot avoid. Every sale, every click, every record happens *at* a moment. And time has a property other dimensions don't: **things repeat**. Ice cream sells in summer. Retail spikes at Christmas. Tax software roars in April. This repetition is **seasonality**, and spotting it stops you from panicking over a “dip” that happens every single January.

Getting the date into usable pieces

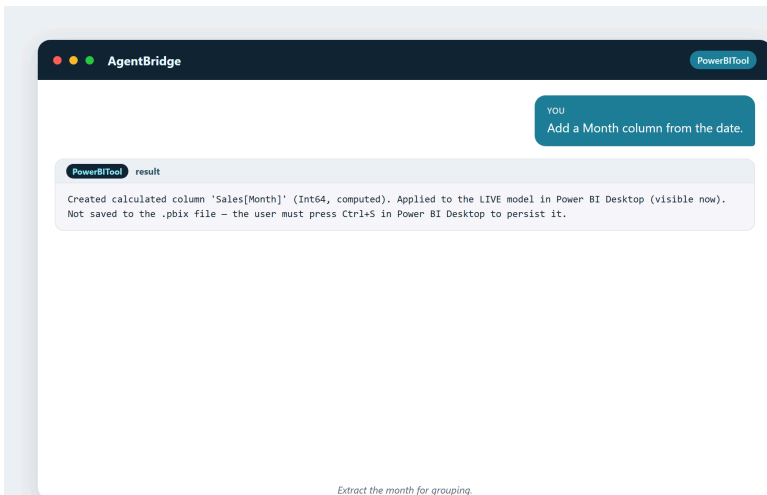
Raw dates are awkward. To analyse time, you break the date into pieces — year, month, day — as columns you can group by:

“Add a Year column from the date.”



Year column

“Add a Month column from the date.”

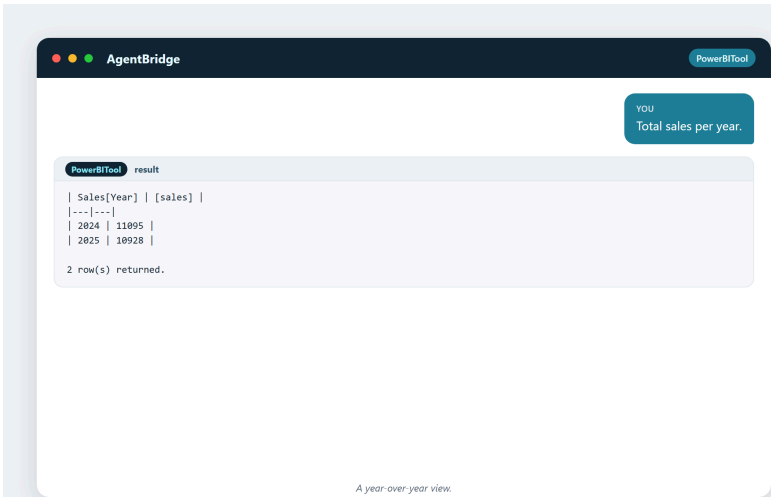


Month column

Now you can group by year or month and see the shape of time.

The year-over-year view

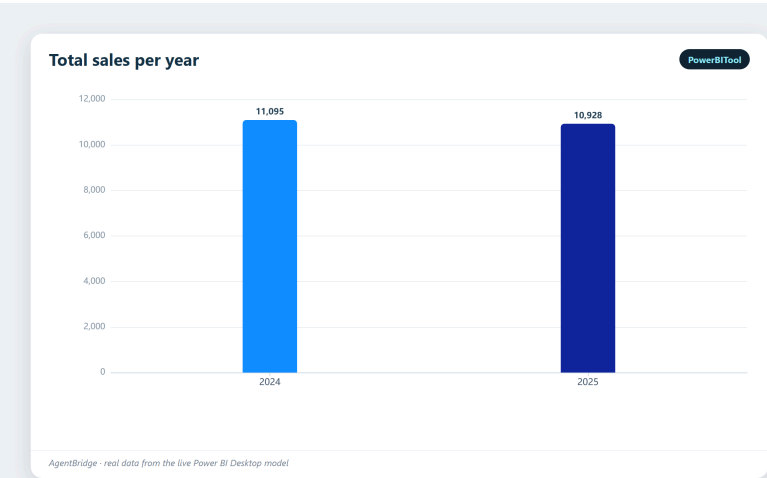
“Total sales per year.”



Sales per year

Two years, side by side. Is 2025 better than 2024? The comparison is the whole point — a single year tells you nothing, but two years tell you the direction.

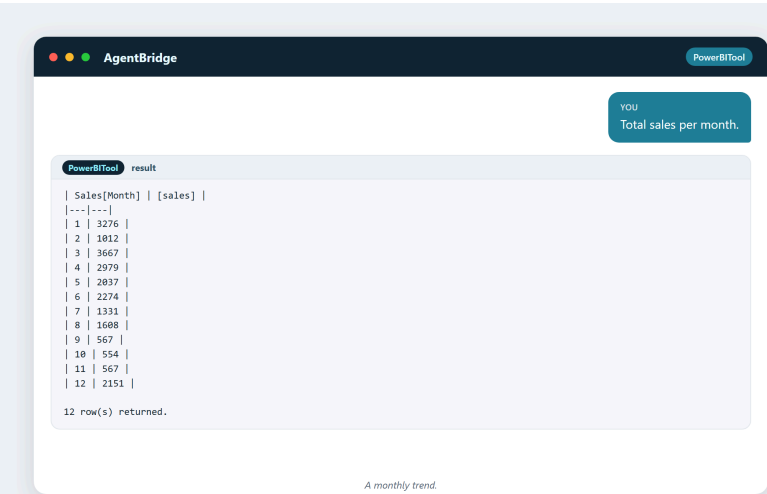
The year-over-year comparison, as a chart:



Sales per year — bar chart

The monthly trend

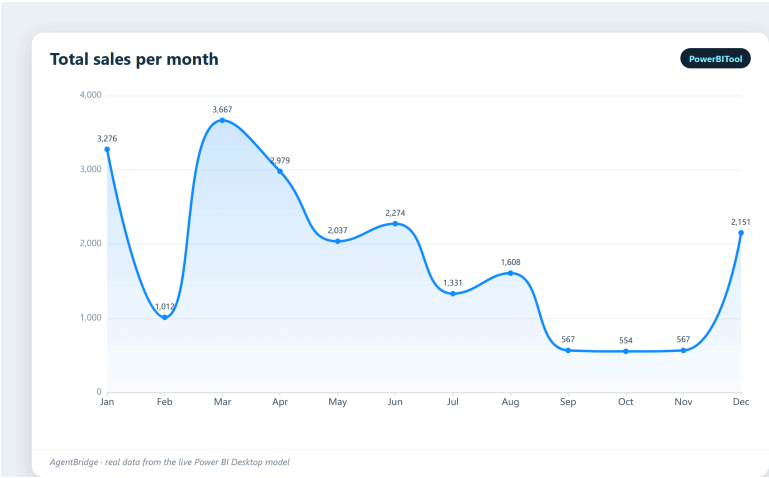
“Total sales per month.”



Sales per month

Twelve months of data. You can see the peaks and valleys — the busy months and the quiet ones. This is the raw shape of your business’s heartbeat.

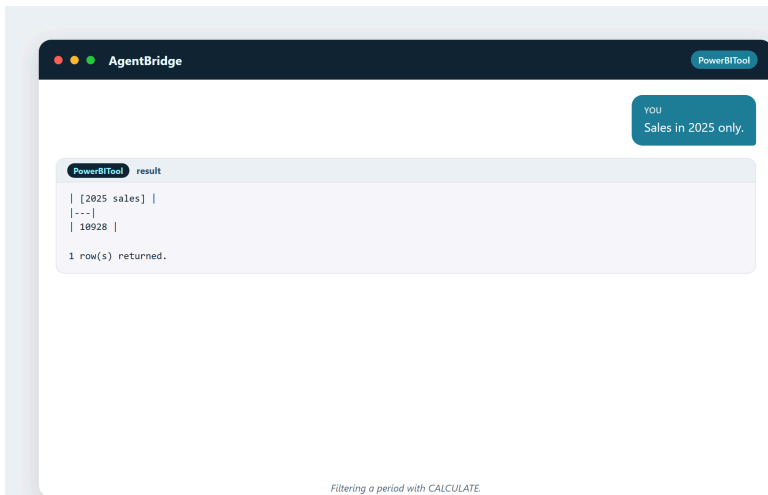
The monthly heartbeat, drawn as a line:



Sales per month — line chart

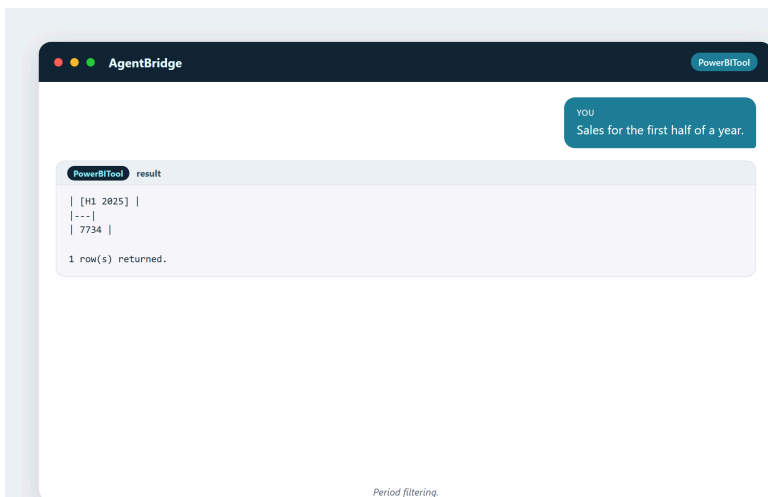
Filtering a period

“Sales in 2025 only.”



2025 sales

“Sales for the first half of a year.”

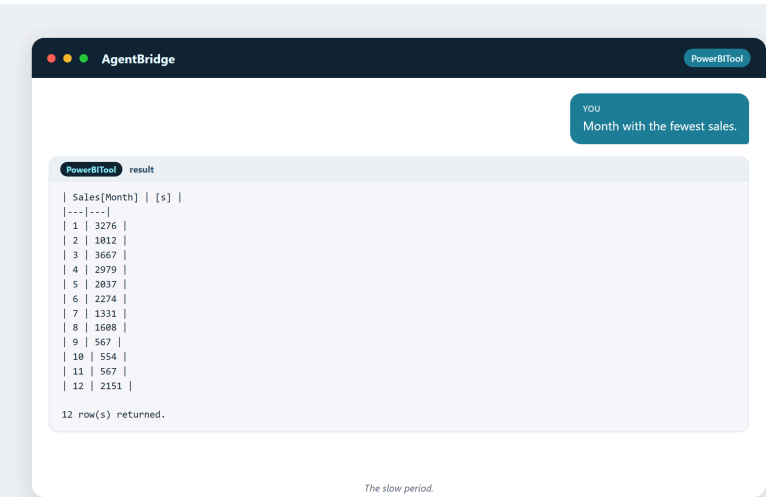


First half of 2025

Slicing a specific window of time is how you answer “how did we do last quarter?” in one sentence.

Finding the slow period

“Month with the fewest sales.”



Fewest sales month

Knowing your slowest month is as useful as knowing your busiest — it’s when you plan promotions, schedule maintenance, or brace for a quiet spell.

Moving averages: smoothing the noise

Monthly numbers are bumpy. A **moving average** (say, the average of the last 3 months) smooths the bumps so the underlying trend shows through. It’s the difference between watching a shaky handheld camera and a smooth steadicam shot. The trend is what you want to see; the moving average reveals it.

A curiosity: the “January effect” that isn’t

A manager sees January sales down 30% and calls an emergency meeting. But January is *always* down after December’s holiday rush. Without comparing to last January, the drop is meaningless — it’s the season, not a problem. This is why analysts compare **year-over-year** (this January vs last January) rather than **month-over-month** (January vs December). The right comparison turns a false alarm into a non-event.

What you’ll carry from this chapter

- Time turns a snapshot into a story.
- Break dates into year/month/day to group and trend.
- Seasonality means things repeat — don’t panic at the expected dip.
- Compare year-over-year, not just month-over-month.
- Moving averages smooth the noise to reveal the trend.

Next: how do you know a difference is real and not just luck? A gentle tour of testing and chance.

15. Testing, Chance, and Being Right

You changed the website and conversions went up 2%. Did your change work, or was it just luck? This is the question that separates real analysis from wishful thinking, and the answer lives in the unglamorous world of testing and chance. Don't worry — we'll keep it painless.

The problem: was it the change or the luck?

Any number can bounce around by chance. If you flip a coin 10 times and get 7 heads, you don't conclude the coin is rigged. Same with business: if a new ad gets a few more clicks, it might be better, or it might be noise. The question is: **how confident can you be that the difference is real?**

The idea of a sample

You almost never see the whole population — you see a **sample**. 1,000 visitors to your site, not all the people who could ever visit. A sample is a small taste of a much bigger pot. The trick is that a small taste can tell you about the whole pot — *if* it's big enough and unbiased.

Big sample + random selection = trustworthy. Tiny sample or cherry-picked = dangerous. A/B tests work because they split visitors randomly into two groups and compare.

A/B testing: the honest experiment

The gold standard for “does this work?”:

1. Split your audience **randomly** into two groups.
2. Group A sees the old version; Group B sees the new version.
3. Measure the outcome in both.
4. Compare. If B beats A by more than chance explains, the change is real.

Randomness is the whole trick. It makes the two groups identical except for the one thing you changed, so any difference must be the change.

Significance: is the difference real?

Statisticians use a **p-value** to answer “could this be chance?” A p-value below 0.05 is the usual bar: it means “if there were really no difference, we’d see something this extreme less than 5% of the time.” Below the bar, you call it **statistically significant** — likely real. Above it, you shrug and say “not enough evidence.”

You don’t need to compute p-values by hand. You need the instinct: **a small difference on a small sample is probably noise; a clear difference on a big sample is probably real.**

The two ways to be wrong

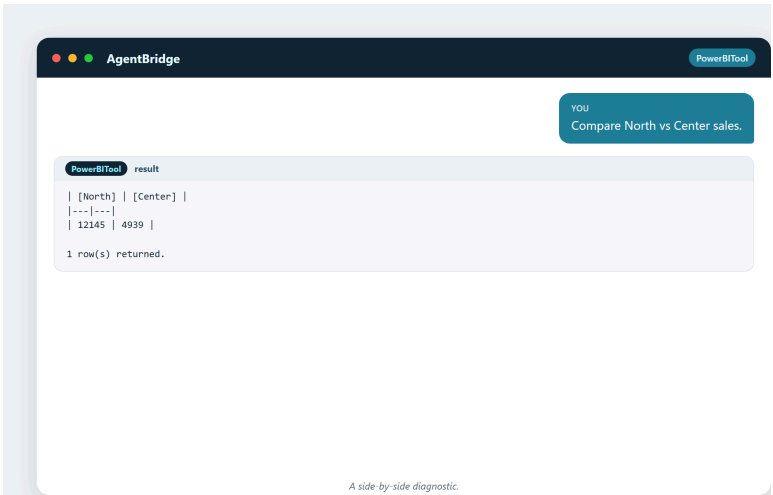
- **Type I error (false positive):** you say the change worked when it didn’t. You ship a useless change. The 5% bar controls this.
- **Type II error (false negative):** you say the change didn’t work when it did. You throw away a good idea. Usually caused by too small a sample.

Both happen. Good testing balances them: enough data to catch real effects, a strict enough bar to avoid chasing ghosts.

Comparing two groups, live

You don’t need a lab to see the shape of a comparison. The assistant can put two groups side by side in one query:

“Compare North vs Center sales.”



A side-by-side diagnostic.

Compare two groups

Scale this up with random assignment and a big sample, and you have an A/B test. The logic is identical: two groups, one difference, measure and compare.

A curiosity: the cookie that fooled everyone

A company ran an A/B test, saw a big lift, and celebrated. The catch: the two groups weren’t actually random — a bug put all the mobile users in one group. The “win” was really just mobile users behaving differently. The test was sound in theory and broken in practice.

Randomisation is everything. A test is only as good as the split behind it.

When you don't need a formal test

Not every decision needs a p-value. If you change the price of one item and watch one week of sales, you're not running an experiment — you're observing. Formal testing is for the decisions that matter and can be run properly. For everything else, be honest that you're guessing, and keep the decision reversible.

What you'll carry from this chapter

- A difference can be luck; ask how confident you are.
- Samples let a small taste tell you about the whole pot — if they're big and random.
- A/B testing = random split, change one thing, compare.
- “Significant” means “unlikely to be pure chance.”
- Randomisation is everything; a bad split fakes a win.

Part III is done — you can build metrics, tell the kinds of analysis apart, know your customers, read trends, and tell real from noise. Next we make it all visible: Power BI and the art of showing your data.

16. Power BI in Plain English

You have heard the name. This chapter strips Power BI down to what it actually is, without the marketing fog — and shows how the assistant talks to it directly.

What Power BI actually is

Power BI is Microsoft's tool for turning data into **dashboards and reports** that people can look at, click on, and explore. It has three main parts:

- **Power BI Desktop** — the free program on your PC where you build the model and the report. This is where the assistant works.
- **Power BI Service** — the online place where you publish dashboards so others can see them in a browser or on their phone.
- **Power BI Mobile** — the app for checking dashboards on the go.

You build in Desktop. You share through the Service. That's the whole picture.

The three layers inside

Every Power BI project has three layers, and it helps to know their names:

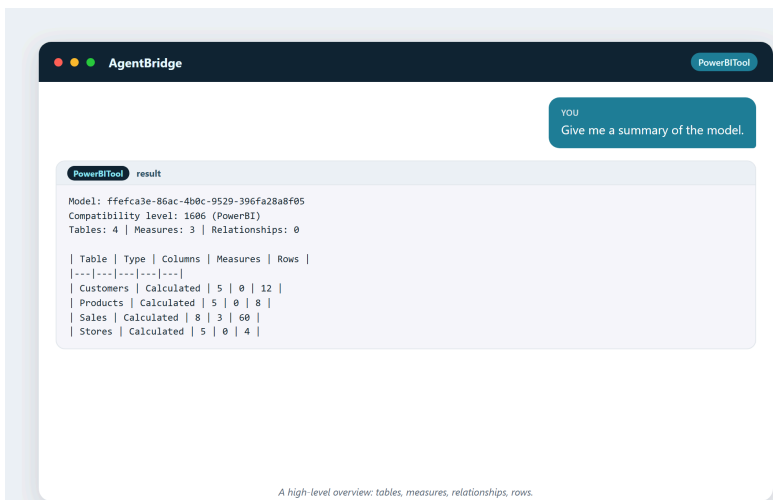
1. **Data** — what you connect to (a database, a file, a web source).
2. **Model** — the tables, relationships, and measures you build on top of the data.
3. **Report** — the visual pages people actually look at.

The assistant works almost entirely in the **model** layer — the tables, measures, and relationships. The report layer (the pretty visuals) is where a human arranges things on the canvas. The model is the engine; the report is the dashboard.

Seeing the model, live

The assistant can read the whole model and report it back:

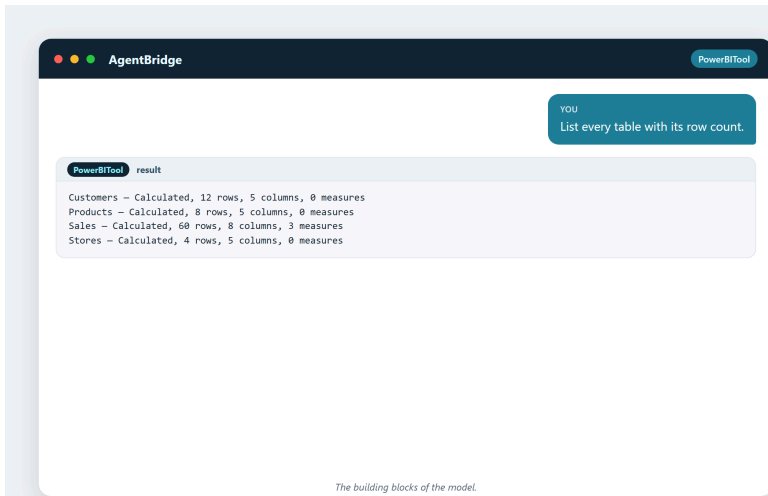
“Give me a summary of the model.”



Model summary

Tables, measures, relationships, row counts — the whole engine in one view. This is the first thing you do when you open any Power BI project: understand the model.

“List every table with its row count.”



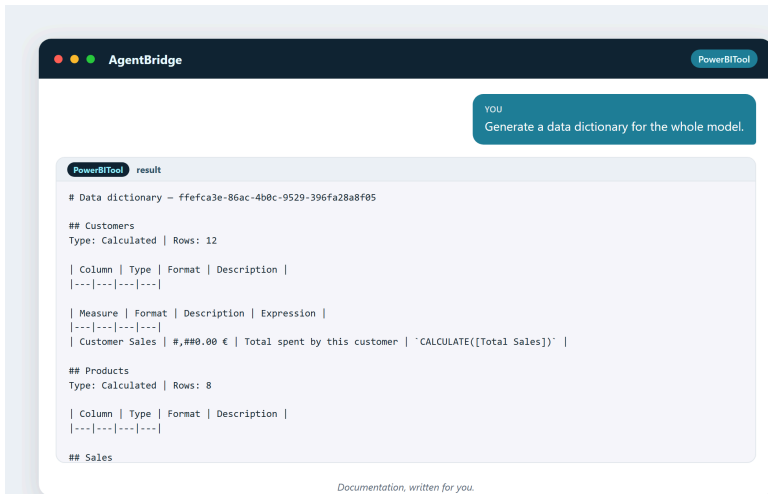
List tables

The building blocks, counted and ready.

The data dictionary: documentation for free

One of the assistant's most useful tricks is writing a **data dictionary** — a document that lists every table, column, and measure with what it means.

“Generate a data dictionary for the whole model.”

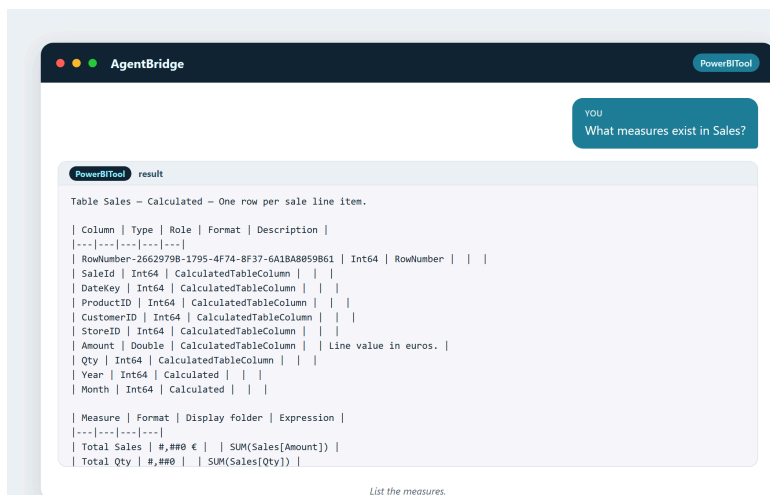


Data dictionary

Documentation that would take an analyst an afternoon appears in a second. This is a big deal: good documentation is the difference between a model a team can trust and a model only one person understands.

Seeing the measures

“What measures exist in Sales?”



Measures in Sales

Every measure with its formula and format. When someone asks “how is Total Sales calculated?”, the answer is right there.

A curiosity: Power BI’s improbable rise

Power BI started in 2015 as a small add-on and raced to the top of the analytics world, largely because Microsoft bundled it with tools companies already had and priced it low enough that almost anyone could try it. Its quiet superpower is that it sits inside the Microsoft ecosystem — Excel, Azure, Office — so for millions of businesses it was the path of least resistance. The best tool is often not the best tool; it’s the one that’s already there.

Why the assistant matters here

Power BI is powerful but has a learning curve — DAX, the model view, the ribbon of buttons. The assistant removes that curve for the model work: you describe what you want, it edits the model live. You still

arrange the visuals yourself, but the hard part — the measures and the wiring — becomes a conversation.

What you'll carry from this chapter

- Power BI = Desktop (build), Service (share), Mobile (view).
- Three layers: data, model, report.
- The assistant works in the model layer.
- It can summarise, list, and document the model on demand.
- The assistant removes the learning curve for the hard part.

Next: how the assistant finds and connects to your Power BI Desktop — the moment the two meet.

17. Connecting to Your Data

Before the assistant can do anything with Power BI, it has to connect to it. This chapter is about that handshake — how the assistant finds your open report, connects to the live model, and knows exactly what it’s talking to.

The local connection

Here is the key thing to understand: Power BI Desktop, when you open a report, starts a small **analysis engine** on your own machine (a program called `msmdsrv`). The assistant connects to *that* engine, on *your* machine.

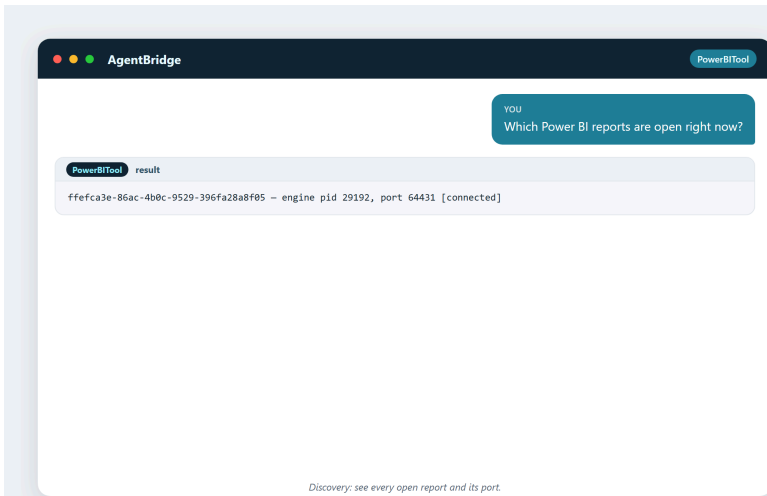
```
You → AgentBridge → PowerBITool → the engine inside your  
Power BI Desktop
```

No cloud. No upload. The data never leaves your computer. The assistant is simply talking to the same engine that Power BI itself uses, through a local door.

Finding what’s open

The assistant can see every Power BI report you have open, each with its own engine and port:

“Which Power BI reports are open right now?”



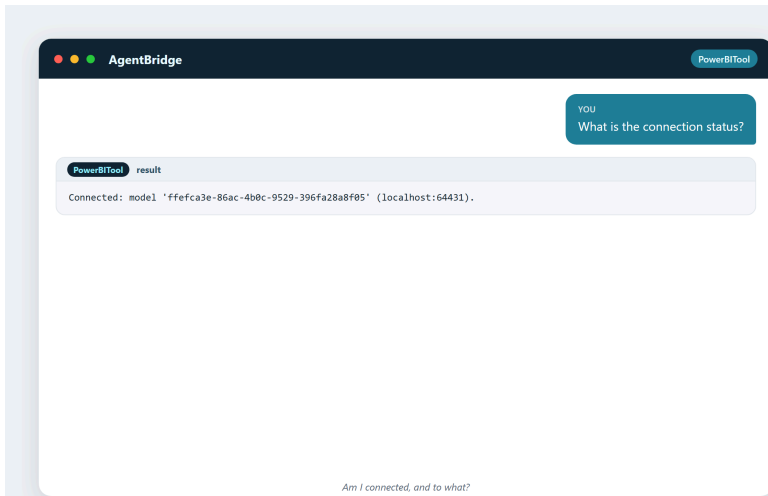
Open reports

If you have one report open, it connects to it directly. If you have several, you tell it which one by name. This is how it stays pointed at the right thing.

Confirming the connection

Once connected, you can always check the status:

“What is the connection status?”



Connection status

It tells you which model it's on and which local port. This matters because every later change goes to *this* live model. Knowing exactly what you're connected to is the first rule of safe editing.

What “live” really means

When the assistant changes the model, the change happens in the **live, in-memory model** inside Power BI Desktop. You see it immediately — that's the visual feedback loop. But there is one important catch the tool always reminds you of:

The change is live but **not saved to the file**. To keep it, you press **Ctrl+S** in Power BI Desktop.

This is a safety feature, not a bug. It means every change is reversible until you choose to save. You can experiment freely; nothing is permanent until you decide.

A curiosity: the port is a secret door

Each Power BI Desktop instance picks a random local network port for its engine — that number in the connection string (like `localhost:64431`). The assistant discovers this port automatically by finding the running Power BI process and its child engine. You never have to know the number; the tool figures it out. It's the same door Power BI uses internally — the assistant just learned how to knock.

Reconnecting and safety

If you close the report and open another, the assistant notices the engine changed and asks you to reconnect — it won't blindly write to the wrong model. This session-safety is what makes live editing trustworthy: the tool checks that the engine behind the connection is still the one it connected to before it lets a change through.

What you'll carry from this chapter

- The assistant connects to the local engine inside your Power BI Desktop.
- No cloud, no upload — everything stays on your machine.
- It discovers open reports and their ports automatically.
- Changes are live but not saved until you press Ctrl+S.
- The tool guards against writing to the wrong model.

Next: modelling in Power BI — the assistant as a careful, well-documented modeler.

18. Modelling Data in Power BI

Modelling is where the analysis is won or lost. A good model makes every question easy; a bad model makes every question a fight. This chapter shows the assistant as a careful modeller — one that not only builds the model but documents it and checks it against best practices.

What a good model looks like

You met the star schema in Chapter 8. In Power BI, a good model means:

- A clean **fact table** (the numbers: sales, transactions).
- Tidy **dimension tables** (the descriptions: products, customers, dates).
- **Relationships** wired correctly (many-to-one, no ambiguity).
- **Measures** with clear names, formats, and descriptions.
- **Documentation** so the next person (or you, in six months) understands it.

The assistant helps with all of these, live.

Documenting as you go

Good models are documented models. The assistant can add descriptions to tables and columns on request:

“Add a description to the Sales table.”

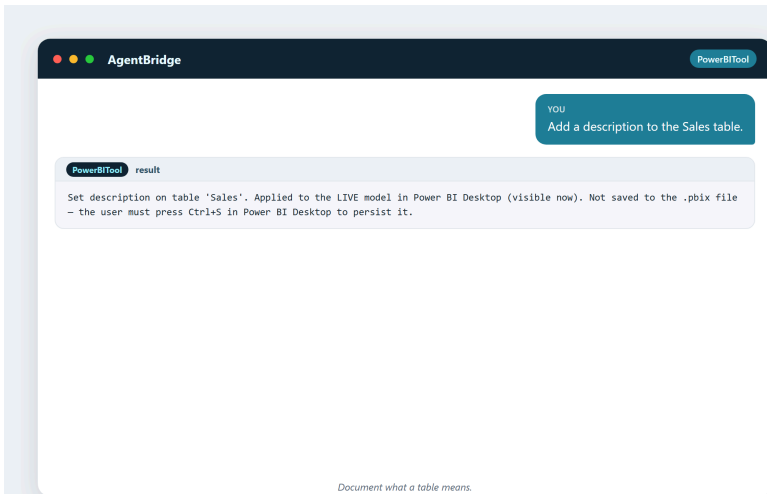
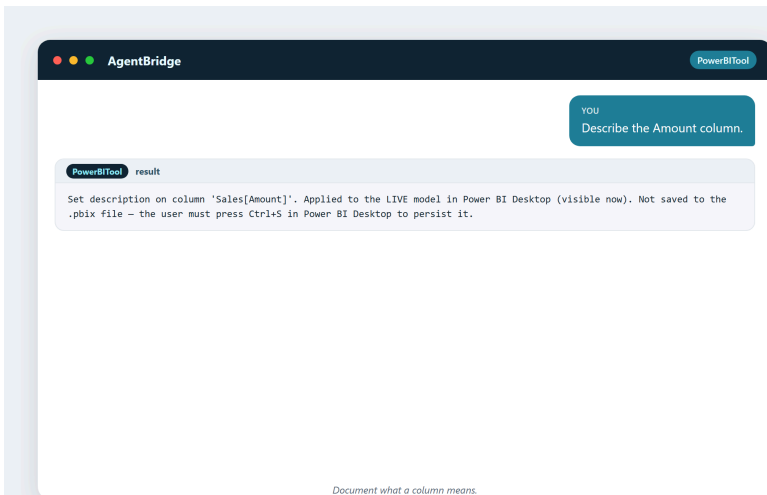


Table description

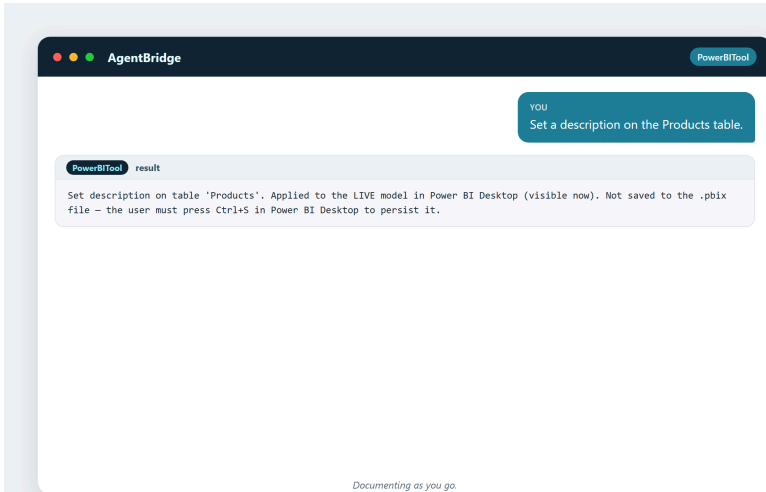
“Describe the Amount column.”



Column description

These small notes show up in the model and in the data dictionary. They are the difference between a model that’s a black box and one that’s a shared asset.

“Set a description on the Products table.”

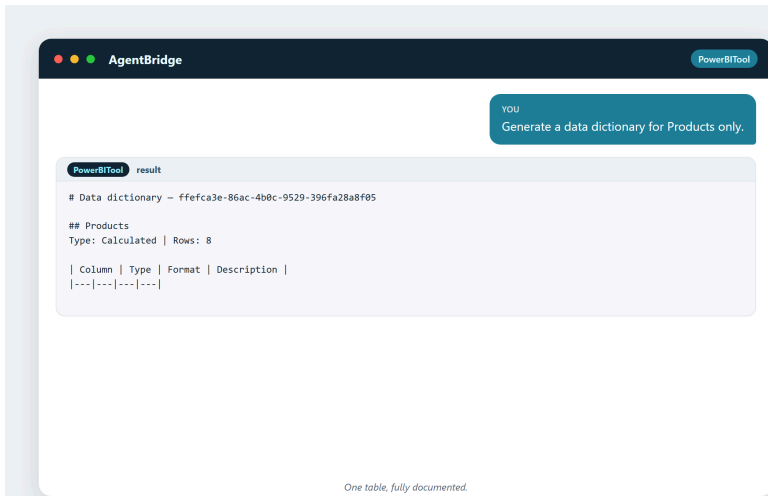


Products description

The data dictionary, table by table

You can document the whole model or a single table:

“Generate a data dictionary for Products only.”



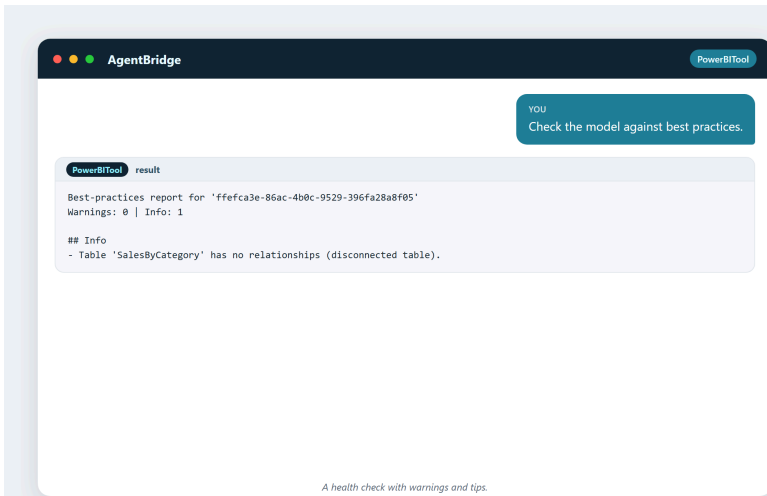
Products dictionary

A focused dictionary for one table — handy when you’re handing a piece of the model to a colleague.

The health check: best practices

This is one of the assistant’s most valuable moves. It scans the whole model and reports problems and tips:

“Check the model against best practices.”



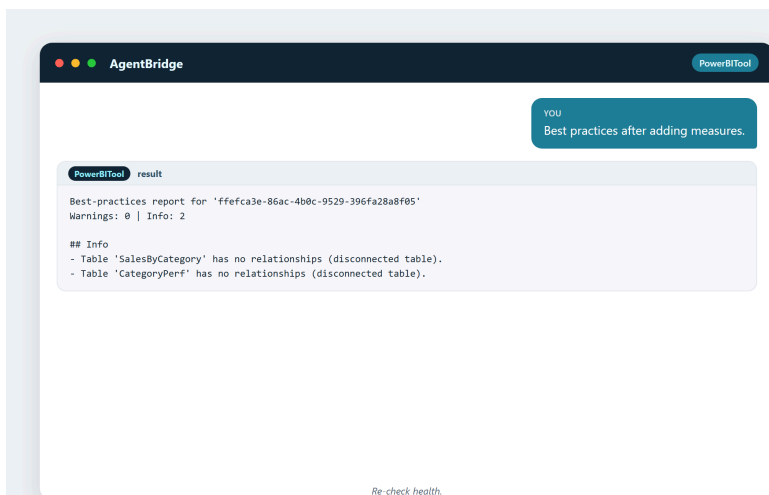
Best practices report

It flags measures with no format string, tables with no description, disconnected tables — the small sins that make a model hard to use. This is like a linter for your data model: it won't stop you working, but it tells you where the model is messy before the mess bites you.

Re-checking after changes

As you build, the model drifts. Re-running the check keeps it honest:

“Best practices after adding measures.”



Best practices after changes

A quick re-scan shows what your latest changes introduced. Build, check, fix, repeat — the rhythm of a clean model.

A curiosity: the bus factor

There's a metric in software teams called the **bus factor**: how many people would have to be "hit by a bus" before a project is stuck because only one person understands it. A model with no documentation has a bus factor of one — terrifying. Every description and dictionary entry the assistant writes raises that number. You're not just tidying; you're making the model survivable.

Why the assistant is a good modeller

A human modeller under deadline pressure skips documentation and best practices. The assistant doesn't get tired, doesn't skip steps, and checks everything. Pair a human's judgement about *what* to model with

the assistant's diligence about *documenting and checking* it, and you get models that stay clean.

What you'll carry from this chapter

- A good model: clean fact + dimensions, wired right, documented.
- Add descriptions to tables, columns, and measures as you go.
- Generate data dictionaries to document the whole model or one table.
- Run the best-practices check like a linter — often.
- Documentation raises the bus factor; it makes the model survivable.

Next: DAX, the language behind the numbers — and how you don't have to write it.

19. DAX: The Language Behind the Numbers

DAX is the calculation language inside Power BI. It has a reputation for being scary. This chapter is about why it matters, and why — with the assistant — you can use it without ever fighting it.

What DAX is for

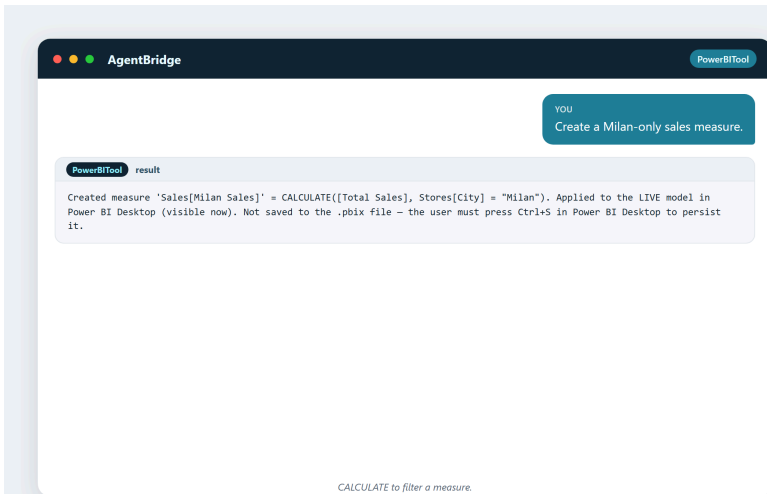
DAX (Data Analysis Expressions) computes the numbers in your reports: totals, averages, percentages, year-over-year, running totals, rankings. Every measure you see on a Power BI dashboard is DAX under the hood.

The core functions are simple: SUM, AVERAGE, COUNT, MIN, MAX, and the mighty CALCULATE, which lets you compute a number *under a specific filter*.

The assistant writes it; you read it

You don't type DAX. You describe the number you want, and the assistant writes the DAX and creates the measure live. But you should be able to *read* what it made, so you trust it.

“Create a Milan-only sales measure.”



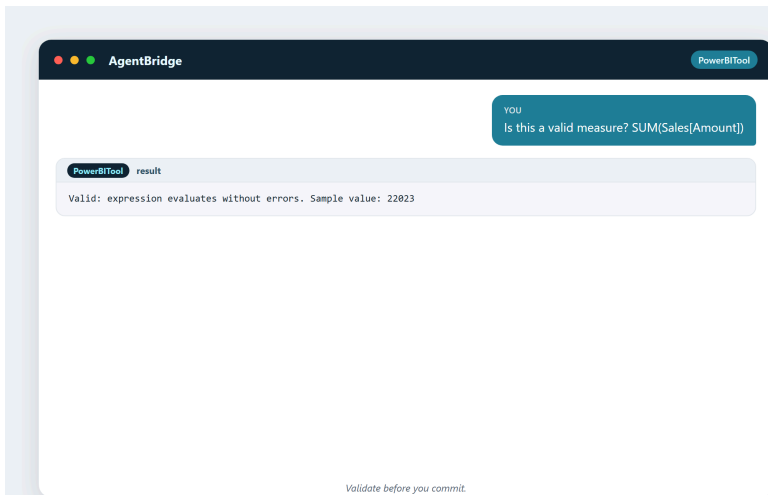
Milan sales measure

Under the hood that's `CALCULATE([Total Sales], Stores[City] = "Milan")` — the total sales, but only where the city is Milan. Once you see the pattern, DAX stops being magic.

Validate before you trust

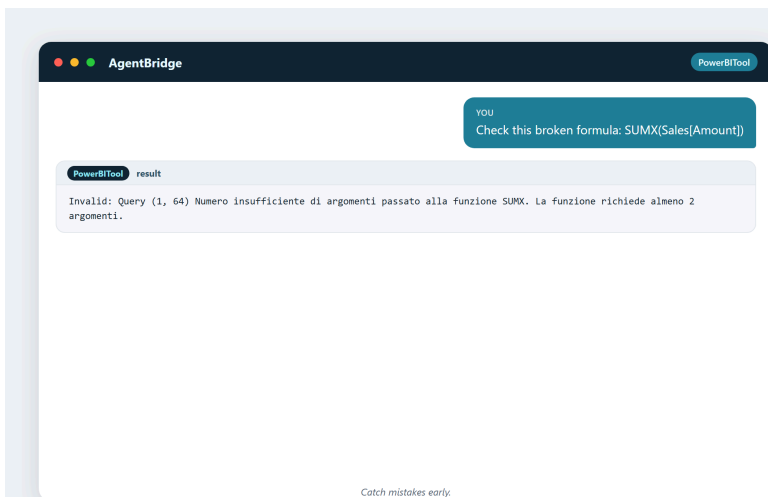
The assistant can test a formula without creating anything:

“Is this a valid measure? `SUM(Sales[Amount])`”



Validate good measure

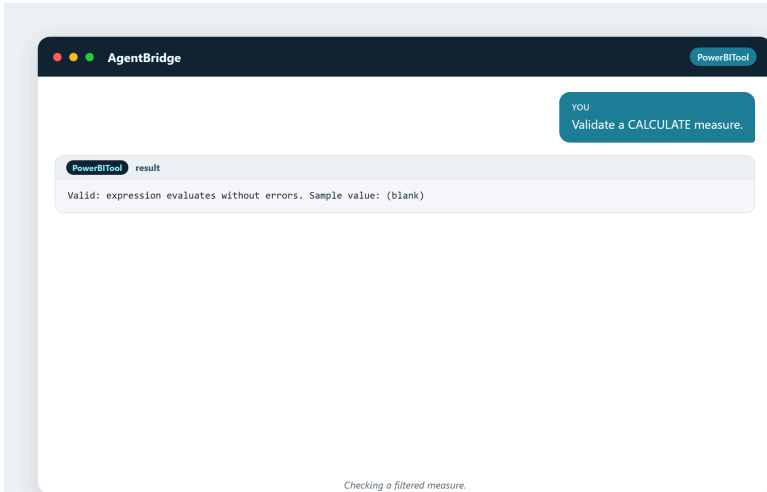
“Check this broken formula: SUMX(Sales[Amount])”



Validate broken measure

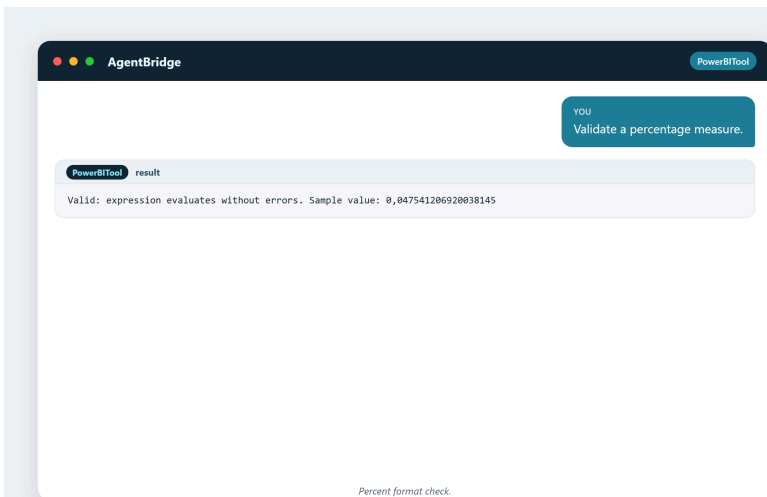
One passes, one fails — and you learn what’s wrong *before* it becomes a broken measure in the model. This “check first” habit saves hours of debugging.

“Validate a CALCULATE measure.”



Validate CALCULATE

“Validate a percentage measure.”

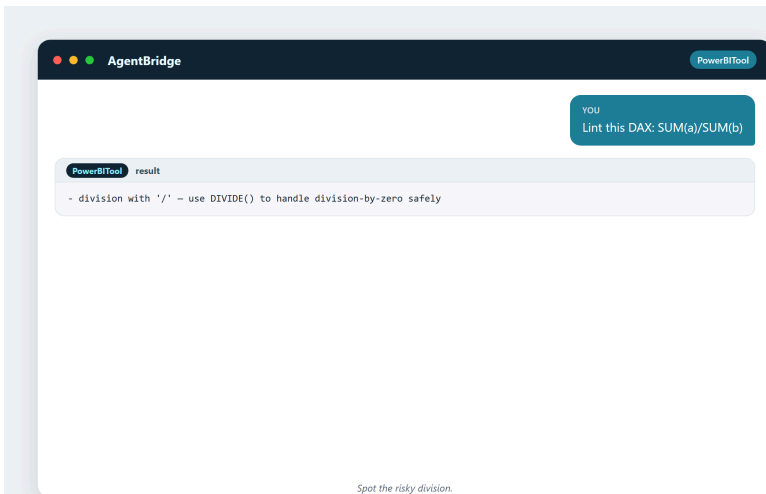


Validate percentage

Linting: the style cop for DAX

Beyond “does it run?”, the assistant can check “is it *well-written*?” — a process called **linting**. It spots common mistakes and risky patterns.

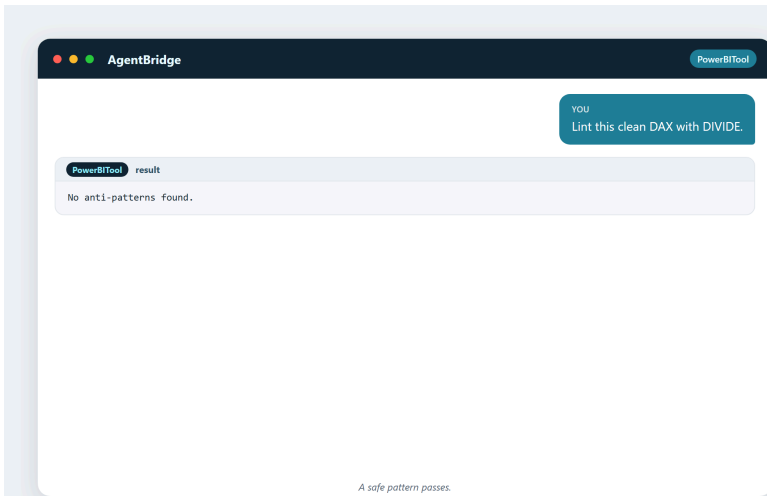
“Lint this DAX: SUM(a)/SUM(b)”



Lint slash division

It warns: don’t use a plain / — use `DIVIDE`, which handles division by zero safely. A small nudge that prevents a whole class of `#DIV/0!` errors.

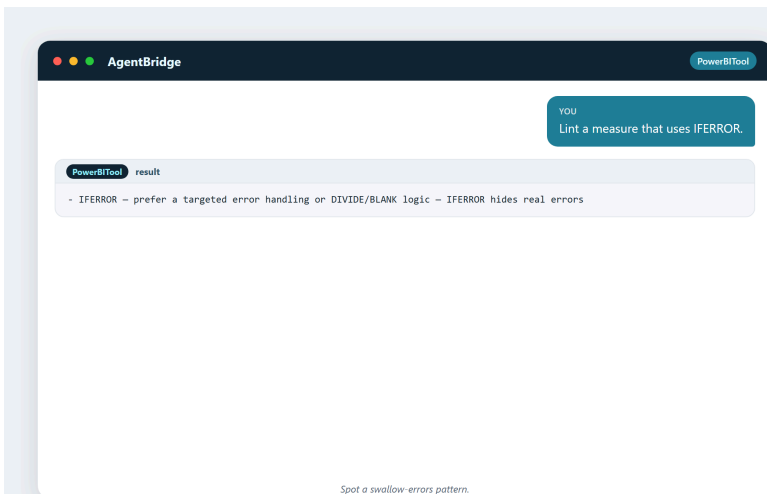
“Lint this clean DAX with `DIVIDE`.”



Lint clean DAX

The clean version passes. You learn the good pattern by seeing it rewarded.

“Lint a measure that uses IFERROR.”



Lint IFERROR

It flags IFERROR as a smell — wrapping errors can hide real bugs instead of fixing them. The linter teaches good habits one warning at a time.

Editing measures

Measures evolve. The assistant can update and delete them:

“Change the format of Total Sales to whole euros.”

AgentBridge

PowerBITool

YOU

Change the format of Total Sales to whole euros.

PowerBITool

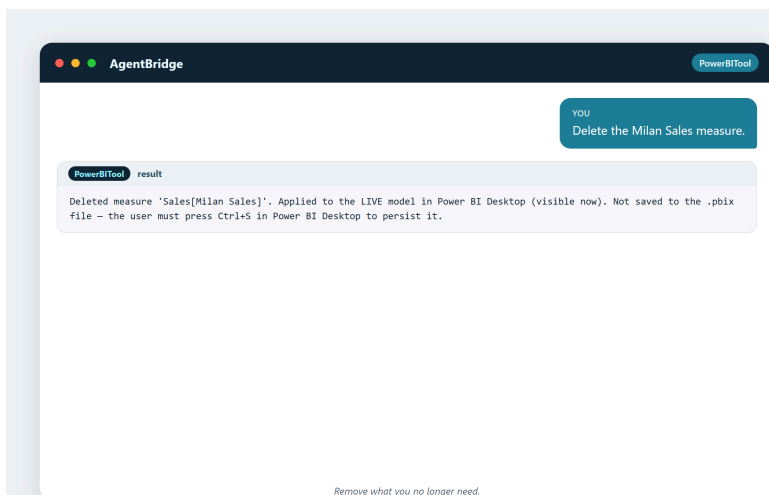
result

Updated measure 'Sales[Total Sales]': format. Applied to the LIVE model in Power BI Desktop (visible now). Not saved to the .pbix file – the user must press Ctrl+S in Power BI Desktop to persist it.

Update a measure's format.

Update format

“Delete the Milan Sales measure.”



Delete measure

Rename, reformat, remove — all live, all reversible until you save.

A curiosity: the filter context beast

The reason DAX is called hard is one concept: **filter context** — the invisible set of filters that a calculation sees at any moment (the current row, the current slicer selection, the current visual). Master it and DAX is your friend; misunderstand it and numbers look wrong in ways that are hard to trace. Here is the liberating truth of this book: **you describe the answer, and the assistant handles the filter context.** The beast becomes the tool's problem, not yours.

What you'll carry from this chapter

- DAX computes the numbers; CALCULATE is its most powerful word.
- You describe the answer; the assistant writes the DAX.
- Validate a formula before you create it.

- Lint to catch bad patterns (plain /, IFERROR hiding bugs).
- The hard part — filter context — is now the tool's job.

Next: seeing is believing — how to choose the right chart and not lie with visuals.

20. Choosing the Right Chart

A chart is not decoration. It is a tool for making a truth visible. The right chart makes an insight obvious in a second; the wrong chart hides it or, worse, lies. This chapter is about picking the right picture for the truth you want to show.

The one rule

There is one rule that covers most of it:

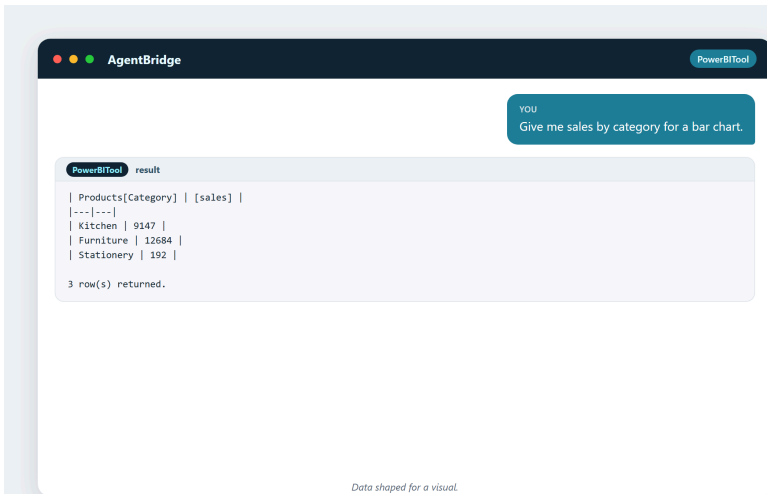
Match the chart to the question, not to what looks cool.

Comparing categories? Bars. Change over time? A line. Part of a whole? A pie (a small one). Relationship between two numbers? A scatter. Pick the chart that answers the question, and the answer shows itself.

Comparing categories: use bars

When you want to compare “how much for each thing” — sales by category, by region, by product — use a **bar chart** (or column chart). Bars are easy for the eye to rank. The assistant can hand you the data shaped for exactly this:

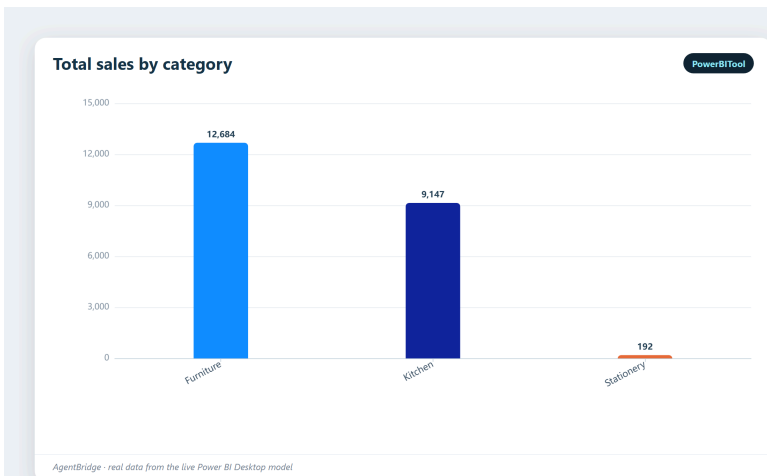
“Give me sales by category for a bar chart.”



Sales by category for a bar chart

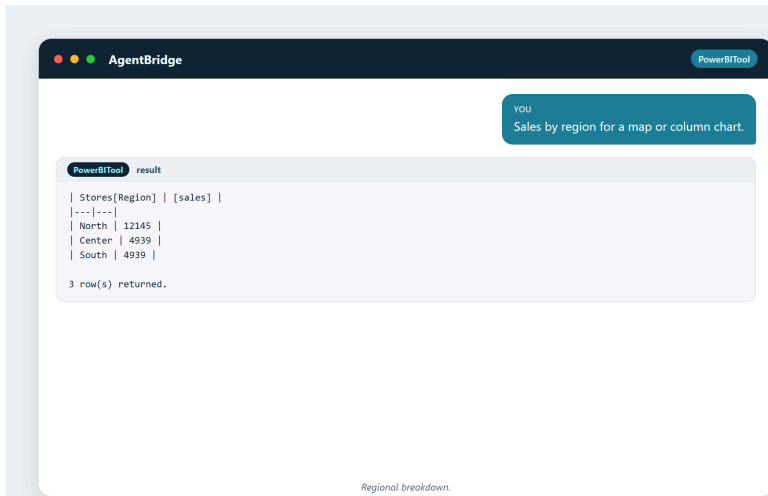
Three categories, their totals, ready to become a bar chart. The eye instantly sees Furniture on top, Stationery at the bottom.

And here is that same real data rendered as the chart:



Sales by category — bar chart

“Sales by region for a map or column chart.”

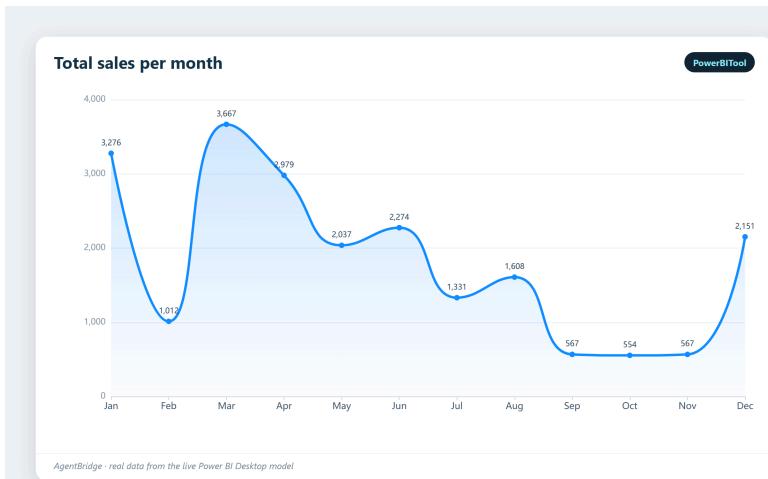


Sales by region

Regional totals, ready for a column chart or a map.

Change over time: use a line

When the question is “how did this move over time?”, a **line chart** shows the shape of the trend — the rises, the dips, the season — better than any table.



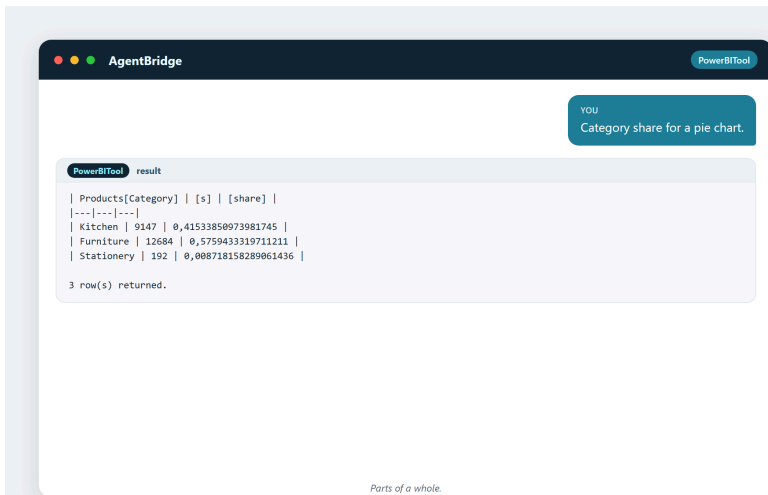
Total sales per month — line chart

Twelve months of real sales, one line. You can see the March peak and the autumn trough at a glance — the kind of pattern a table of numbers hides.

Part of a whole: use a pie (carefully)

A pie chart shows how a total splits into parts. It works with **three or four slices**. It fails badly with ten.

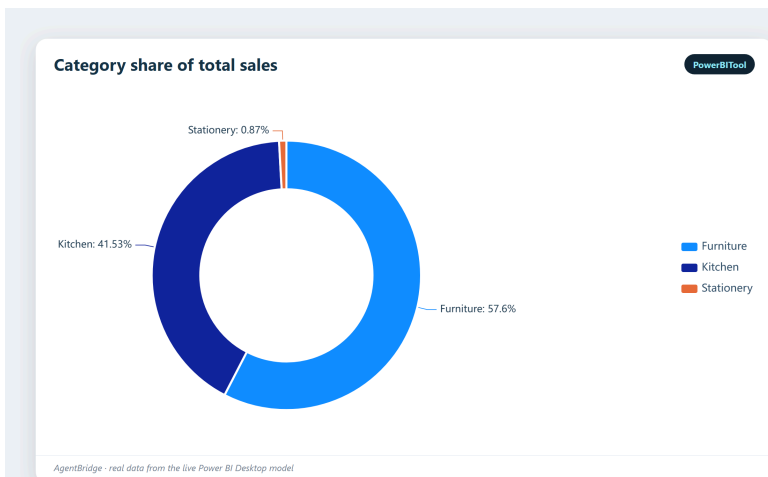
“Category share for a pie chart.”



Category share

Kitchen, Furniture, Stationery as shares of the whole — a clean pie. Add a dozen categories and the same chart becomes unreadable confetti.

The same share, rendered as a donut with the values and percentages:



Category share — donut chart

The charts to avoid

- **3D charts** — they distort the data. A 3D pie tilts slices and lies about their size. Never.
- **Dual-axis charts** — two y-axes can make unrelated things look related. Use with extreme care, or not at all.
- **Pie charts with many slices** — unreadable. Use a bar chart instead.
- **Truncated axes** — a bar chart whose axis starts at 90% instead of 0 makes a tiny difference look huge. Start the axis at zero unless you have a strong reason.

A curiosity: the chart that lied to a nation

In the 2012 U.S. election, a widely-shared chart showed President Obama winning “98% of the vote” — because it was a map of *county* wins, and rural counties are huge in area but tiny in population. The map showed land, not people, and misled millions. The lesson: a chart can be technically accurate and completely misleading. The analyst’s job is to choose the view that shows the *truth*, not just *a* truth.

Colour with purpose

Colour is powerful and easily wasted:

- Use colour to **highlight**, not to decorate.
- Reserve red for “bad / below target”, green for “good” — and don’t overuse either.
- Design for **colour-blind** readers: don’t rely on red/green alone; add labels or shapes.
- Fewer colours = clearer message.

A dashboard is a story, not a paint palette

Every visual on a page should earn its place by advancing the story. If a chart doesn't help the reader understand or decide, cut it. A clean page with three good charts beats a busy page with twelve pretty ones.

What you'll carry from this chapter

- Match the chart to the question, not to what looks cool.
- Bars for comparing; lines for time; small pies for parts of a whole.
- Avoid 3D, dual-axis tricks, many-slice pies, and truncated axes.
- A chart can be accurate and still misleading — choose the honest view.
- Use colour to highlight, and design for colour-blind readers.

Next: putting it together — reports and dashboards that people actually use.

21. Reports and Dashboards People Actually Use

A dashboard that nobody opens is a failure, no matter how beautiful it is. This chapter is about building reports that people actually look at, trust, and act on — and how the assistant prepares the numbers that go on them.

Dashboard vs. report — they’re not the same

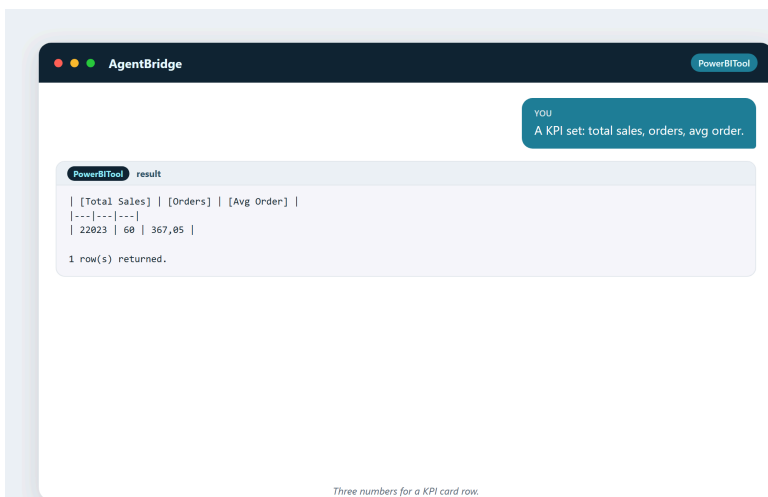
- A **dashboard** is a single page of the most important numbers, designed to be read at a glance. Think: a car’s speedometer and fuel gauge.
- A **report** is a deeper, multi-page exploration you can drill into. Think: the owner’s manual you consult when something’s wrong.

Both have a place. The dashboard answers “how are we doing right now?”; the report answers “let’s dig into why.”

The KPI row: the top of every good dashboard

Most great dashboards open with a row of big numbers — the handful of KPIs that matter most. The assistant can build that row in one ask:

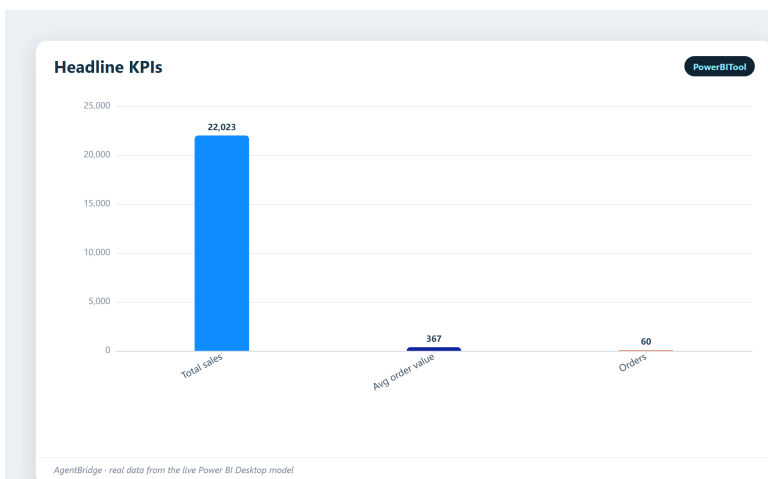
“A KPI set: total sales, orders, avg order.”



KPI set

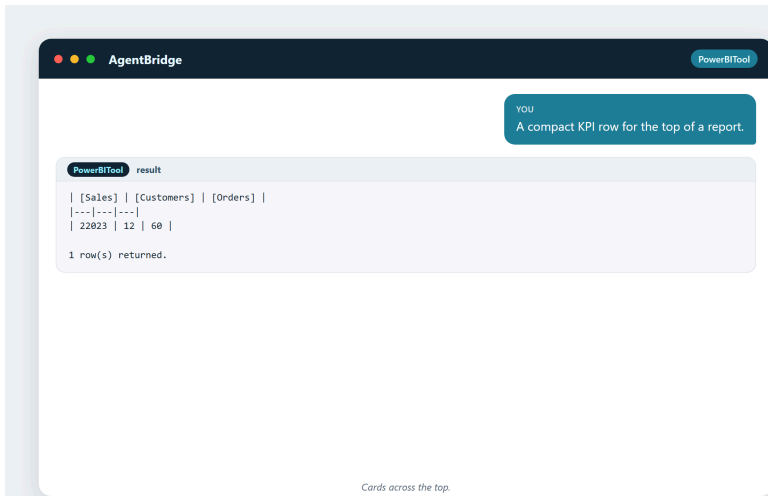
Three numbers, ready for three KPI cards across the top. This is the first thing a busy executive reads, so it must be the right three numbers.

The same KPIs, drawn as a comparison chart:



Headline KPIs — chart

“A compact KPI row for the top of a report.”



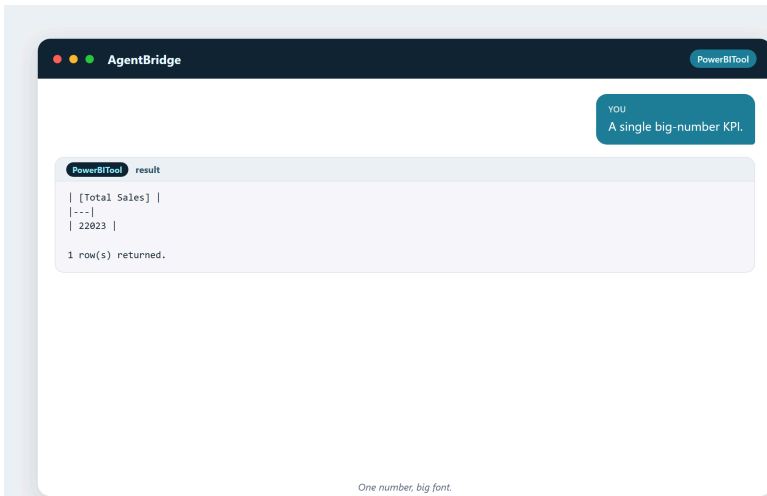
Compact KPI row

Same idea, a different set — whatever your top-line metrics are.

The single big number

Sometimes one number is the whole story:

“A single big-number KPI.”



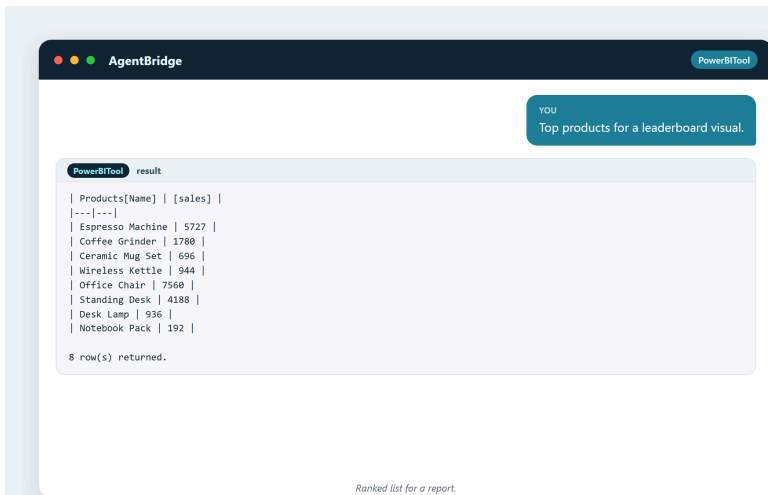
Big number KPI

A big-number card is the dashboard equivalent of a headline. Use it for the one metric everyone cares about.

The leaderboard

People love a ranking. A top-N list drives action and a little friendly competition:

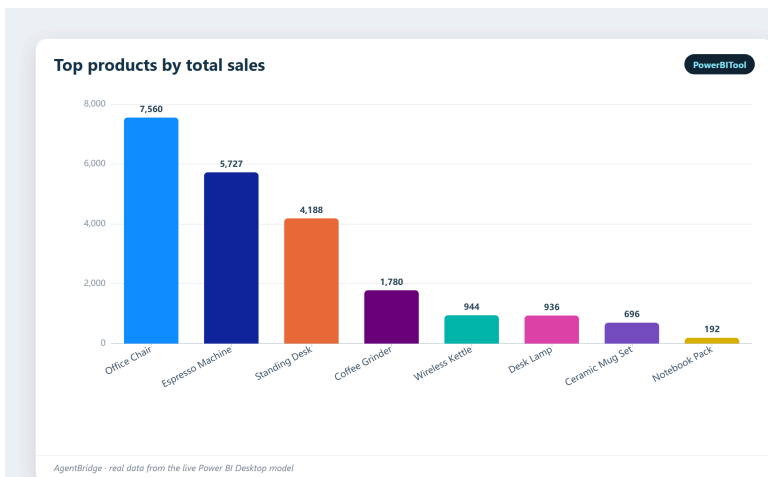
“Top products for a leaderboard visual.”



Top products leaderboard

A ranked list, ready for a leaderboard visual. The bottom of the list is where the problems are; the top is where to double down.

The leaderboard, drawn as a chart:



Top products — leaderboard chart

What makes a dashboard usable

- **Few numbers, big and clear.** A glance should tell the story.
- **The right numbers.** The ones that move with the business (Chapter 11).
- **Consistent.** Same metrics, same colours, same layout every time — so people learn to read it.
- **Live.** Updated automatically, so it's always current.
- **Honest.** No truncated axes, no vanity metrics.

The drill-down principle

A good dashboard shows the summary and lets you **drill down** when you need detail. You see “North is down” on the dashboard, click it, and the report shows you which stores, which products, which days. Summary first, detail on demand — never bury the reader in detail upfront.

A curiosity: the cockpit rule

Pilots don't want more instruments; they want the *right* instruments in the *right* place. The same rule governs dashboards. A famous design principle says a good dashboard answers your most important question **without you having to ask** — the number you need is already there, big, current, and honest. If you have to hunt or click to find the thing you check every morning, the dashboard isn't done.

The assistant's role in the dashboard

The assistant builds the **numbers** — the measures and KPIs that feed every card and chart. You arrange them on the canvas. This division of

labour is the sweet spot: the assistant handles the calculation and the checking; you handle the story and the layout.

What you'll carry from this chapter

- A dashboard is a glance; a report is a drill-down.
- Open with a KPI row of the few numbers that matter.
- Use big-number cards and leaderboards for focus.
- Usable = few, right, consistent, live, honest.
- The assistant builds the numbers; you build the story.

Part IV is done — you can connect, model, calculate, and visualise. Now the final act: the agentic revolution that ties it all together, and your place in it.

22. The Rise of the AI Assistant

Everything in this book has been building to this moment: a tool that listens to a plain-English question and does the whole chain of analytical work for you. This chapter is about what changed, and why it matters more than it first sounds.

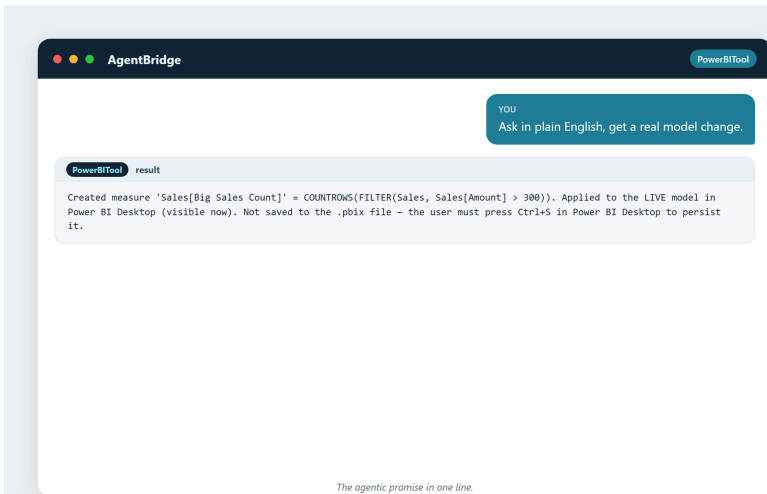
The old way: a chain of manual steps

For decades, answering a business question in Power BI meant a chain of manual steps: open the tool, find the table, write the DAX, check for errors, create the measure, format it, add it to a visual, repeat. Each step needed a skill, and the chain needed time. The bottleneck was never the question — it was the doing.

The new way: describe the answer

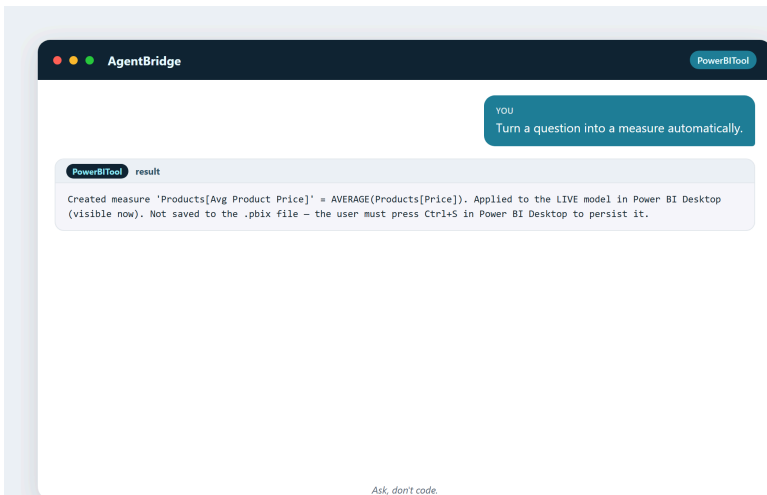
Now you describe the answer you want, and the assistant does the chain:

“Ask in plain English, get a real model change.”



Plain English to a real change

“Turn a question into a measure automatically.”



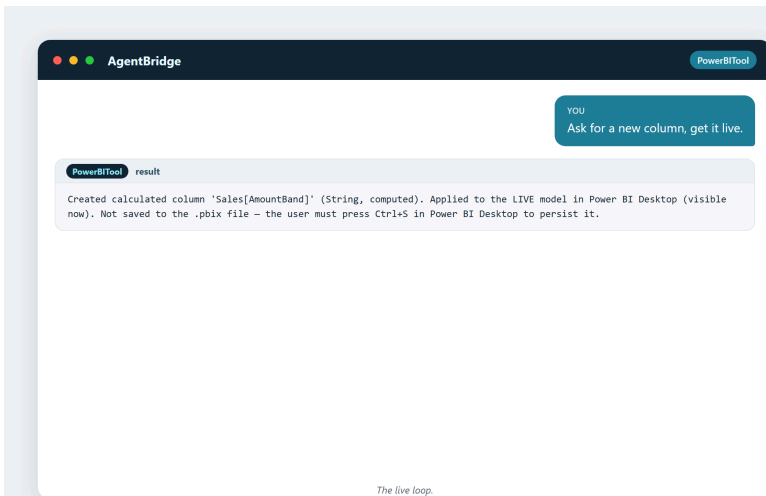
Question to measure

The chain that used to take a skilled analyst a while now happens in a sentence. The skill moves from *doing the steps* to *asking the right question*.

Why this is a bigger deal than “autocomplete”

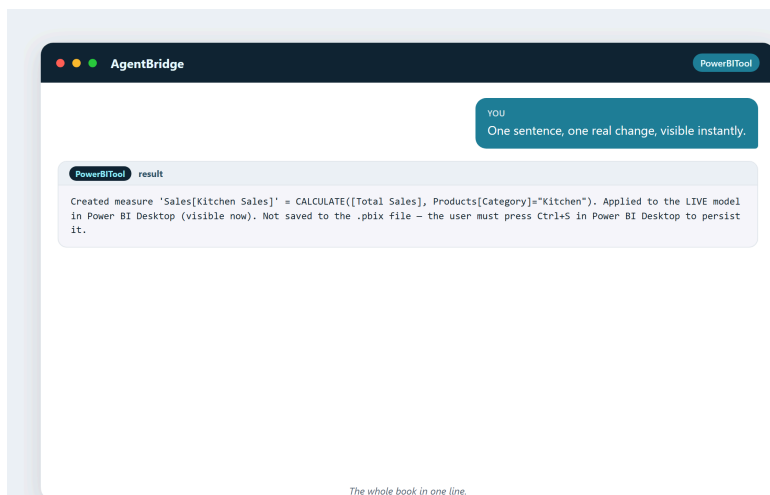
This is not autocomplete. Autocomplete helps you type faster. The assistant here **understands the intent and operates the tool** — it connects to the model, writes correct DAX, applies the change live, and reports what it did. That is the difference between a faster keyboard and a capable colleague.

“Ask for a new column, get it live.”



New column live

“One sentence, one real change, visible instantly.”



One sentence, one change

The agentic shift

The word for this is **agentic**: the software takes *actions* toward a goal, not just answers questions. You give it a goal (“add a kitchen-sales measure”), and it acts — connecting, calculating, applying, confirming. An agent closes the loop between intent and result.

This is why the examples in this book feel different from a tutorial. Each one is a real action on a real model, done by an agent, visible on a real screen.

What it means for the analyst

The fear is that the assistant replaces the analyst. The reality is more interesting: it **removes the drudgery and elevates the judgement**. The analyst stops spending hours on syntax and starts spending time on the questions that matter — which questions to ask,

which answers to trust, what to do next. The assistant is a force multiplier for a good analyst, and a doorway for a curious non-analyst.

A curiosity: the Jevons paradox of analysis

When something gets cheaper, we use more of it, not less. Economists call this the **Jevons paradox**. When analysis becomes this easy, we won't do less analysis — we'll do vastly more. Questions that were “not worth the effort” become free. The world won't need fewer analysts; it will need analysts who can ask better questions of a tool that answers instantly.

The honest caveat

An assistant is only as good as its checks. This one validates DAX, guards against dangerous operations, and reports exactly what it changed — because an agent that acts without checking is a liability, not a helper. Trust the assistant the way you trust a careful colleague: completely, but with the receipts always visible.

What you'll carry from this chapter

- The old way was a long manual chain; the new way is one described answer.
- This is action, not autocomplete — the agent operates the tool.
- “Agentic” means it closes the loop from intent to result.
- It removes drudgery and elevates judgement.
- Cheaper analysis means more analysis, not less (Jevons).

Next: meet the two tools behind every example in this book.

23. Meet AgentBridge and PowerBITool

Every example in this book was made with two tools working together. This chapter introduces them properly — what each one is, how they fit together, and what they can do.

AgentBridge: the assistant

AgentBridge is an AI assistant that runs on your own computer. You talk to it in plain words, and it does work across your tools. It is not a chatbot that only talks — it acts. It can write documents, build spreadsheets, send email, research the web, and, with the right plugin, operate Power BI.

The key properties:

- **Local.** It runs on your machine. Your data stays with you.
- **Plain-language.** You describe what you want; you don't write code.
- **Extensible.** Plugins give it new abilities. PowerBITool is one of them.

PowerBITool: the hands inside Power BI

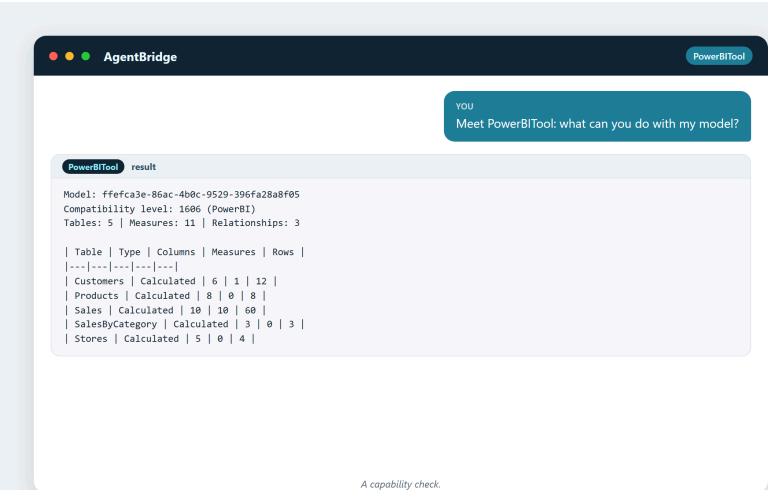
PowerBITool is the plugin that gives AgentBridge hands inside Microsoft Power BI Desktop. Through it, the assistant can:

- **Connect** to the live model of an open report.

- **Inspect** — model summary, tables, schema, measures, relationships.
- **Edit the model** — create and delete tables, columns, measures, relationships; set descriptions.
- **Run and validate DAX** — with a safety guard that blocks anything dangerous.
- **Profile and document** — profile tables, generate a data dictionary, lint for best practices.

Here is the assistant introducing itself to a model:

“Meet PowerBITool: what can you do with my model?”



Meet PowerBITool

The summary it returns is the whole surface: tables, measures, relationships, row counts — everything it can see and work with.

How they fit together

You → AgentBridge (the brain) → PowerBITool (the hands) →
Power BI Desktop (the model)

AgentBridge understands your words and plans the action.

PowerBITool carries out that action on the live Power BI model. You see the result immediately in Power BI Desktop. The loop is: ask → think → act → see.

The safety guard

One thing worth calling out: PowerBITool runs DAX through a **fail-closed guard**. It allows read-only queries (EVALUATE, system views) and blocks anything that could modify the model through the query door — no DROP, no INSERT, no DELETE, no multi-statement tricks. If a query isn't clearly safe, it's rejected. This is what makes it responsible to let an AI touch a live model.

Free, and supported by real people

Both tools are free. PowerBITool is open on GitHub, and — as the preface promised — the support is free too: open an issue and a real technician answers within about 24 hours with a real fix. That combination (free tool, free human support, fast answers) is the promise behind every example you've seen.

A curiosity: the plugin model

PowerBITool is not compiled into AgentBridge. It's a **plugin** dropped into a `Tools` folder, discovered at startup. That means the assistant's abilities can grow without changing the core — today Power BI, tomorrow other tools. The plugin model is why the assistant can keep gaining new “hands” without getting bloated.

What you can do with them, together

Everything in this book, and more: connect to a report, understand the model, clean data with calculated columns, build measures, wire relationships, validate and lint DAX, generate documentation, and check best practices — all by talking.

What you'll carry from this chapter

- AgentBridge is the local, plain-language assistant (the brain).
- PowerBITool is the plugin that operates Power BI Desktop (the hands).
- The loop: ask → think → act → see, all local.
- A fail-closed guard keeps the live model safe.
- Free tool, free support, real humans, fast answers.

Next: a whole day of the analyst's work, automated.

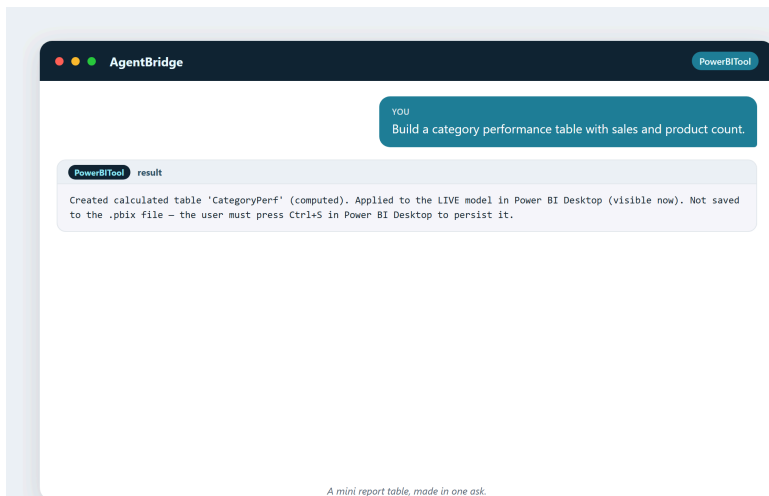
24. Automating the Analyst's Day

Let's spend a day inside the analyst's work and watch the assistant handle it. This chapter strings together the examples the way a real working day goes: build, validate, document, check, finish. Each image is a real action on a live model.

Morning: build the reporting pieces

The day starts by turning raw tables into reporting pieces. Instead of clicking for an hour, you ask for what the dashboard needs.

“Build a category performance table with sales and product count.”

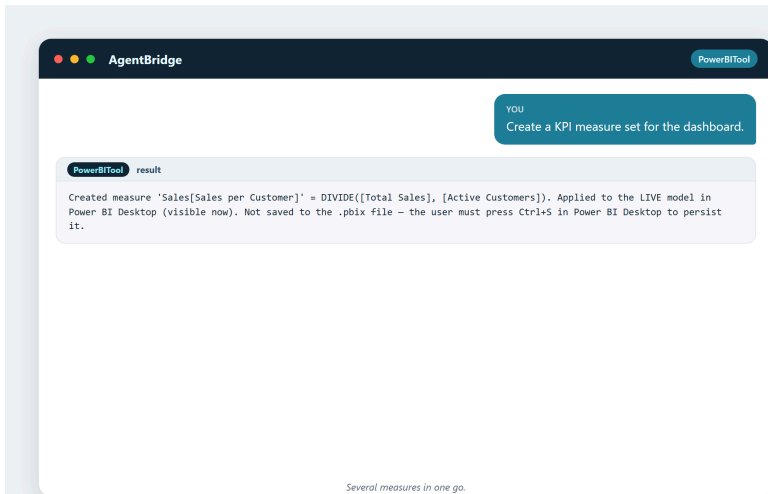


Category performance table

One ask, one new table — sales and product count per category, computed and live.

Then the KPIs the dashboard needs:

“Create a KPI measure set for the dashboard.”



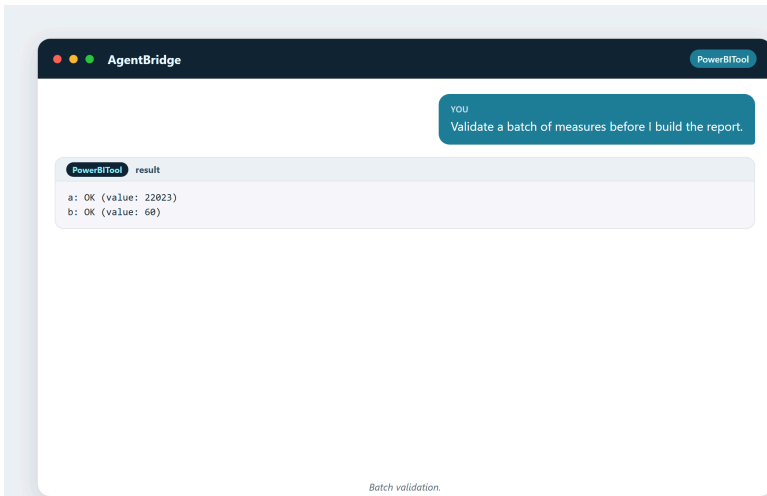
KPI measure set

A measure for revenue per customer, created and applied. The kind of small metric that used to take a careful minute each now arrives in a sentence.

Before the meeting: validate everything

Before you build the report, you check that the numbers are right. The assistant validates a whole batch at once:

“Validate a batch of measures before I build the report.”



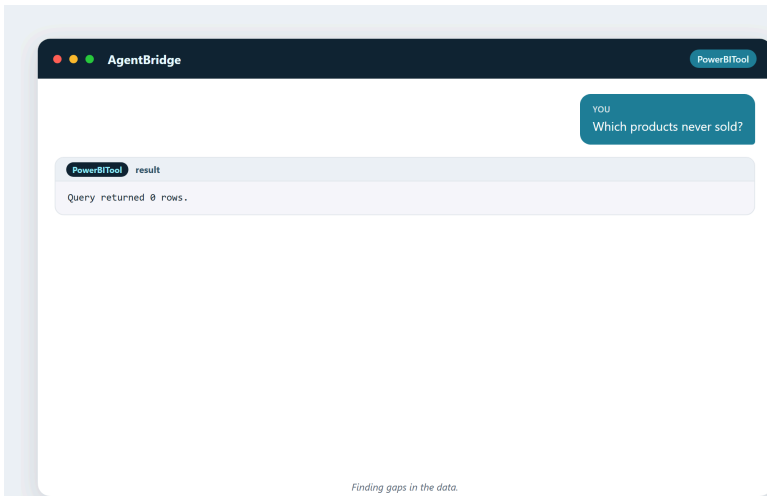
Batch validation

Each measure returns OK with its value. No surprises in front of the boss.

Mid-morning: catch the gaps

A good analyst looks for what's *missing*, not just what's present:

“Which products never sold?”



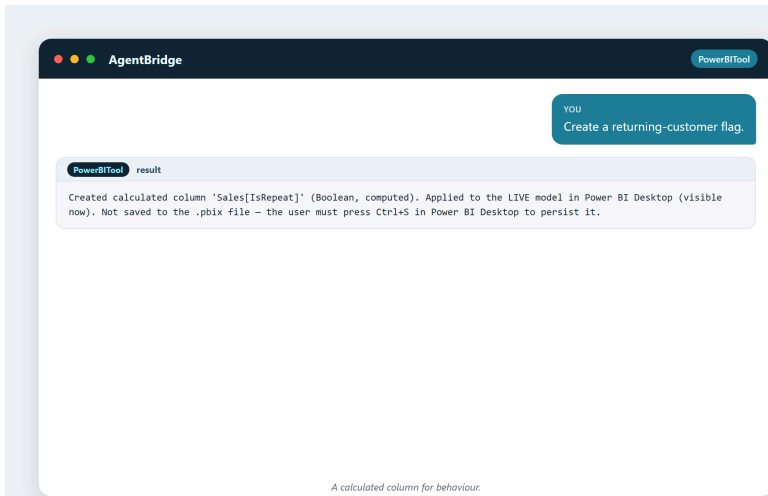
Products that never sold

The query runs and returns zero rows — every product sold at least once. That’s a useful answer too: no dead stock hiding in the catalogue.

Late morning: model behaviour

You want to flag repeat behaviour without hand-tagging rows:

“Create a returning-customer flag.”

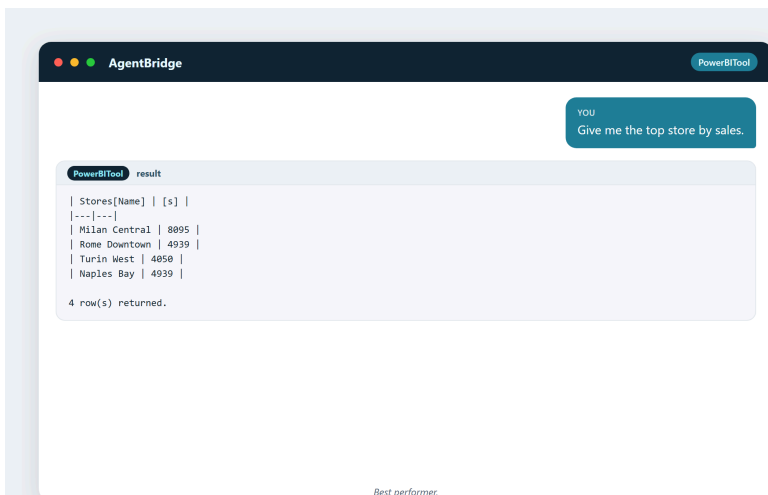


Returning-customer flag

A Boolean column that marks each sale as repeat or not — computed across the whole table in one go.

And the best performer:

“Give me the top store by sales.”



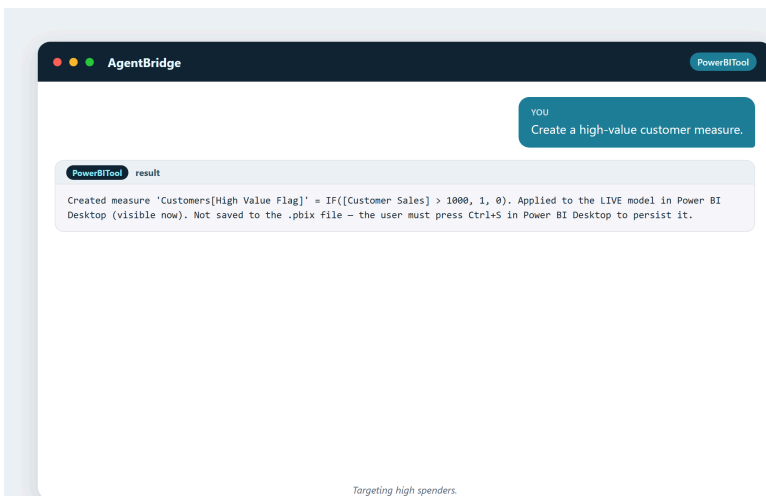
Top store by sales

Milan Central leads. The ranking that used to need a pivot and a sort is now a single question.

Afternoon: targeting

The marketing team wants high-value customers:

“Create a high-value customer measure.”

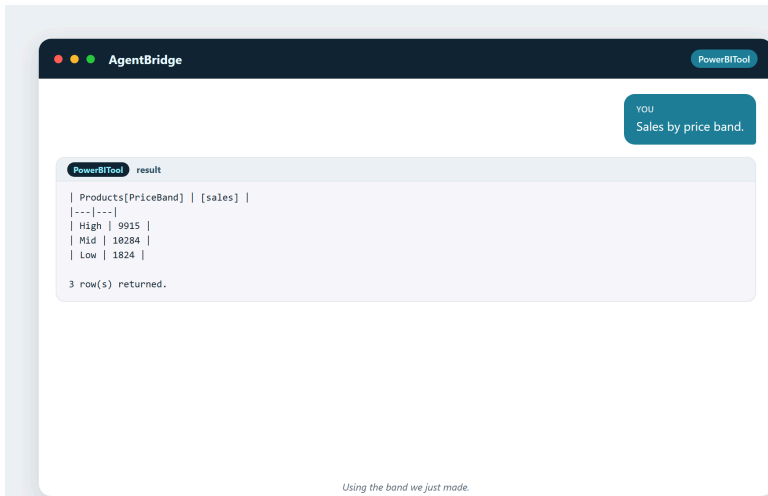


High-value customer measure

A flag for customers over a spend threshold, live in the model, ready to filter on.

And to see how the price bands we made earlier perform:

“Sales by price band.”



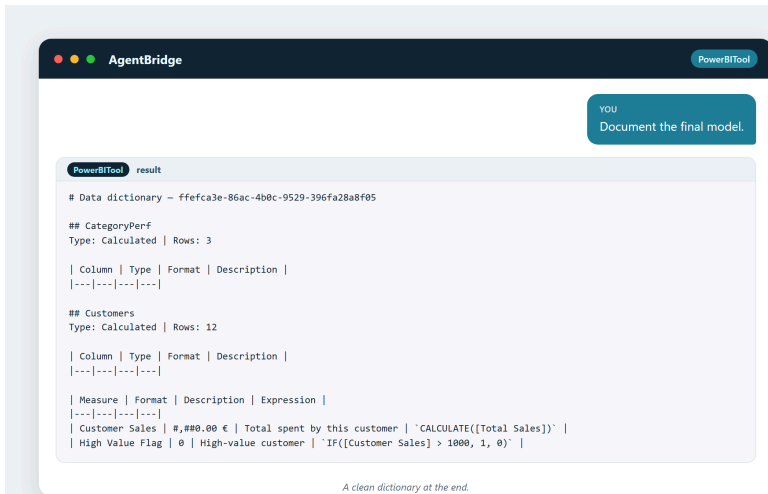
Sales by price band

High, Mid, Low — the band column from Chapter 7 now drives a real breakdown. This is the payoff of building small pieces: they combine later.

End of day: document and check

Before you close, you document the work and check its health. The assistant writes the dictionary for the whole model:

“Document the final model.”

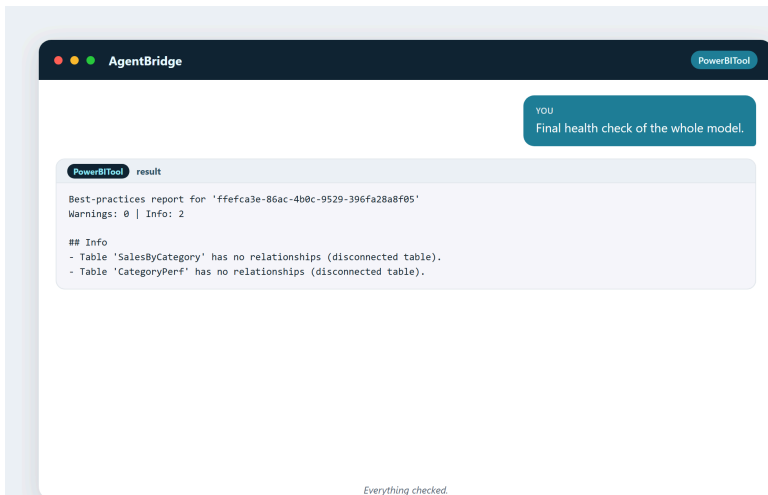


Final data dictionary

Every table, every measure, with its format and expression — documentation you would never have written by hand, done for you.

Then the health check:

“Final health check of the whole model.”



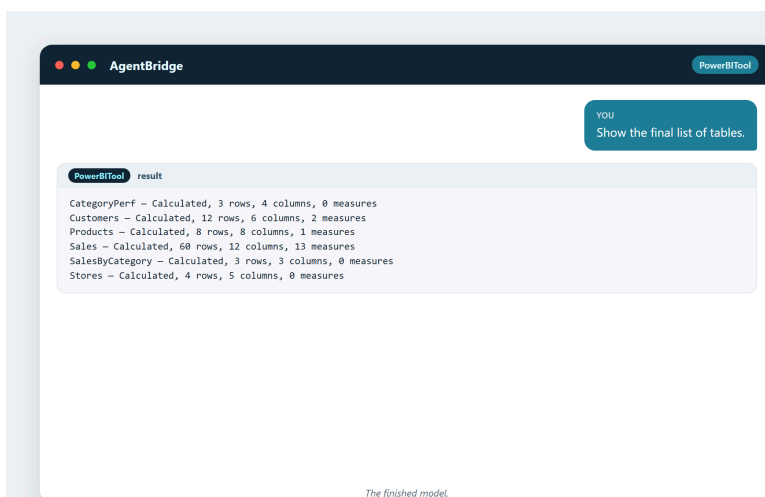
Final health check

Zero warnings. The two “info” notes are just disconnected summary tables, which is expected. The model is clean.

Close: the finished model

At the end of the day, you look at what you built:

“Show the final list of tables.”



Final list of tables

Six tables, fourteen measures, three relationships — a working analytical model, built and documented in a single day of plain-language asks.

What the day shows

A whole working day — build, validate, catch gaps, model behaviour, target, document, check — done by describing each step. The analyst’s

hands did none of the clicking. The analyst's head did all the deciding.

That's the trade this book keeps making: **you keep the judgement, the tool takes the drudgery.**

What you'll carry from this chapter

- A full day of analyst work maps to a sequence of plain-language asks.
- Build pieces (tables, measures), validate them, catch gaps, document, check.
- Small pieces combine later (the price band drives a breakdown).
- The model ends clean, documented, and ready — with none of the manual clicking.

Next: storytelling, ethics, and governance — the part the tool can't do for you.

25. Storytelling, Ethics, and Governance

Here is the part the tool cannot do for you. It can build the measure, validate the DAX, and document the model. It cannot decide what the story means, whether the number is honest, or whether the model is safe to trust. That is yours. This chapter is about the three things that stay human.

Storytelling: the number is not the point

A dashboard full of correct numbers that tells no story is a wall of noise. The value of analysis is the decision it drives. So the analyst's real job is to turn numbers into a story someone can act on.

A simple structure works:

- **What happened.** The fact, plainly. “Kitchen sales fell this quarter.”
- **Why it matters.** The stakes. “Kitchen is 40% of our revenue.”
- **What to do.** The action. “Review the supplier price on the top SKU.”

The assistant gives you the first line instantly. The second and third lines are judgement — context the tool doesn't have.

A curiosity: the word “data” comes from the Latin *dare*, “to give.” Data is meant to be given, not hoarded. A number that never reaches a decision was never really given at all.

The honesty of a chart

The same data can tell opposite stories depending on how you draw it. A truncated axis makes a small change look huge. A cherry-picked time window makes a dip look like a trend. This is not new — it is as old as charts themselves — but a tool that makes charts effortless also makes misleading ones effortless.

The rule is simple: **draw the honest chart, then tell the honest story.** If you would feel uneasy explaining why you cut the axis at that point, don't cut it.

A famous caution: “There are three kinds of lies: lies, damned lies, and statistics.” The joke endures because a number carries an air of truth that words don't. That air is a responsibility, not a trick.

Ethics: three questions to ask before you publish

Before any analysis reaches a decision-maker, ask:

1. **Is it true?** Does the number actually mean what the label claims? (The assistant's validation helps here — but you own the meaning.)
2. **Is it fair?** Could this analysis be used to hurt someone unfairly — to single out a person, penalise a group, or hide an uncomfortable truth?
3. **Is it private?** Does the data include personal information that should be protected or aggregated?

A tool that makes analysis fast also makes it easy to skip these questions. Don't. Speed is no excuse for a careless or harmful conclusion.

Governance: the model is an asset

A model that drives decisions is a business asset, and assets need governance:

- **Who owns it?** Someone must be accountable for the numbers.
- **Where did it come from?** The data source and the transformations must be traceable.
- **Is it documented?** A model nobody understands is a model nobody can trust — or safely change.
- **Is it checked?** Regular best-practice and quality checks keep drift out.

This is where the assistant quietly shines. Every measure it creates can carry a description. Every model can get a generated data dictionary and a best-practices report. Governance is usually the thing teams skip because it's tedious — and tedious is exactly what an assistant removes.

A curiosity: the term “governance” in data comes from the same root as “government.” It is not bureaucracy for its own sake — it is the rule of law for your numbers. Without it, the data is a failed state.

The human edge, restated

The assistant can do the *how*. You own the *what*, the *why*, and the *should*. That division is not a limitation — it is the whole reason a human is still in the loop. The analyst of the future is not replaced by the tool; the analyst is the one who asks the tool the right questions and answers the ethical ones the tool can't.

A short checklist for the human

Before you publish anything the assistant helped build:

- ☐ The chart is honest (no misleading axis, no cherry-picked window).
- ☐ The number means what the label says.
- ☐ The story answers “so what?” and “now what?”
- ☐ The model is documented and has an owner.
- ☐ The analysis could not be used to harm unfairly.
- ☐ Personal data is protected or aggregated.

Six boxes. The tool did the hours of work; these six checks are the human part that keeps it trustworthy.

What you’ll carry from this chapter

- The number is not the point — the decision is.
- Draw the honest chart, tell the honest story.
- Ask: is it true, fair, private?
- The model is an asset: owner, traceability, documentation, checks.
- The tool does the *how*; you own the *what*, *why*, and *should*.

Next: your career as a data analyst in the age of the assistant.

26. Your Career as a Data Analyst in the Age of the Assistant

The book ends where your career actually begins: what does it mean to be a data analyst now, when a tool can do much of the doing? This chapter is a practical answer.

The job description is changing

For years, a data analyst's job description was a list of tools: Excel, SQL, Power BI, a bit of Python. That list is becoming a floor, not a ceiling. The new job is less “operate the tools” and more “ask the questions and own the answers.”

What's changing:

- **From syntax to intent.** You used to be paid to know the DAX. Now you're paid to know the question.
- **From building to judging.** The tool builds; you decide what's right.
- **From producing to interpreting.** Anyone can make a chart. You make it mean something.

What the assistant does *not* take away

It's tempting to think a capable tool shrinks the analyst's role. The opposite is closer to true. The assistant removes the parts of the job that were tedious and low-value, and pushes you up the value chain:

- **Curiosity** is worth more, not less. The better your questions, the more the tool gives you.
- **Context** is yours alone. The tool doesn't know your business, your customers, your politics. You do.
- **Trust** is built by the human. People act on analysis because they trust the person behind it, not the software that made it.

The skills to double down on

If you want to thrive as an analyst now, invest in these:

1. **Asking good questions.** The bottleneck moved here. Learn to frame a business problem as a data question.
2. **Data literacy.** Understand what data can and cannot tell you. Spot a bad assumption before it becomes a bad decision.
3. **Storytelling.** Turn a result into a decision. This is the most career-proof skill in the field.
4. **Judgement and ethics.** Know when a number is misleading, when an analysis is unfair, when to say “not yet.”
5. **Fluency with the assistant.** Learn to talk to the tool. The analysts who win are not the ones who resist the assistant — they're the ones who use it best.

A curiosity: the calculator analyst

When electronic spreadsheets arrived in the 1980s, people predicted the end of the financial analyst. The spreadsheet did the arithmetic instantly, so the arithmetic-based job would vanish. What actually happened: the number of analysts grew, because analysis got cheap enough to do everywhere. The job shifted from *computing* to *thinking*. The same shift is happening now, with a bigger tool.

The portfolio beats the certificate

In a world where the tool can produce the artifact, what proves you're good? Not a certificate — a portfolio. Show that you can:

- Turn a messy question into a clean model.
- Validate and document your work.
- Tell a story that drove a decision.
- Use the assistant well, and know its limits.

The appendices of this book give you a portfolio project model and guided exercises for exactly this. Build a few, show them, and the tool becomes part of your story, not a threat to it.

The honest warning

There is a real risk: the assistant makes it easy to produce an answer you don't understand. A junior who leans on the tool without judgement becomes a button, not an analyst. The way to avoid that is to use the assistant to *learn faster*, not to *think less*. Ask it to explain the DAX it wrote. Check its work. Understand every number before you publish it. Used that way, the assistant makes you better. Used lazily, it makes you dependent.

Where this leaves you

You are not the person who operates the tool. You are the person who decides what the tool should do, checks that it did it right, and turns the result into something that matters. That is a more interesting job than the one it replaced — and it's the one this book has been quietly preparing you for, one plain-English example at a time.

What you'll carry from this chapter

- The job moved from operating tools to asking questions and owning answers.
- Curiosity, context, and trust are worth more, not less.
- Double down on questions, literacy, storytelling, judgement, and fluency with the assistant.
- A portfolio beats a certificate.
- Use the assistant to learn faster, not to think less.

Next: a short conclusion, then the appendices.

Conclusion: The Analyst Who Stopped Pretending

You started with a title about pretending. Let's close the loop on what that means.

The pretending is over

For a while, a lot of analysis was theatre. Long reports nobody read. Dashboards built to look busy. Hours spent wrangling spreadsheets to produce a number that answered a question nobody really asked. That's the "pretending to work" the title pokes at — motion without movement.

The assistant ends that. When a plain sentence can build a measure, validate it, and document the model, there's no excuse for busywork. What's left is the real work: the questions worth asking and the decisions worth making. The pretending falls away, and what remains is the part that actually matters.

What this book tried to do

This book had three jobs:

1. **Show the craft.** What a data analyst actually does, from history to metrics to Power BI, in plain English.
2. **Show the tool.** Every example was made with AgentBridge driving PowerBITool on a live Power BI model — real actions, real results, no mock-ups.

3. **Show the future.** What the job becomes when the doing is automated: more judgement, more curiosity, more responsibility.

If you take one thing from it, take this: **the tool handles the how; you own the what, the why, and the should.**

The promise behind every example

Every image in this book came from a real connection between AgentBridge and PowerBITool. The tool is free. The support is free. Open an issue on the GitHub repository and a real technician answers within about 24 hours with a real fix. That is not a marketing line — it's the same tool the authors use, and the same support they stand behind.

If you've read this far, you already know more about agentic data analysis than most people in the field. The next step is small: install the tool, open a report, and ask it one question. See what happens. That's how every example here started.

A final curiosity

The oldest known accounting artifact is a clay tablet from Mesopotamia, about five thousand years old: a record of sheep and grain. Someone long ago decided it was worth writing down the numbers so a decision could be made later. The tools have changed from clay to DAX. The instinct is identical — and now, for the first time, the tool can do almost all of the writing, so the human can do all of the thinking.

Thank you for reading. Go ask a good question.

Appendix A — Glossary of Terms

Plain-English definitions of the terms used in this book.

Agent / agentic. Software that takes *actions* toward a goal, not just answers questions. An agent closes the loop from intent to result.

AgentBridge. The local, plain-language AI assistant that plans and acts across tools. The “brain” in this book.

PowerBITool. The AgentBridge plugin that operates Microsoft Power BI Desktop. The “hands” in this book.

Power BI Desktop. Microsoft’s tool for building data models and reports.

Model. The set of tables, columns, measures, and relationships that Power BI uses to answer questions.

Table. A set of rows and columns. The basic container for data.

Column. A single field in a table, with a data type (text, number, date).

Measure. A calculated value (usually an aggregate like a sum or average) that responds to filters in a report.

Calculated column. A new column added to a table, computed with a formula for every row.

Calculated table. A new table created from a formula, computed on the fly.

Relationship. A link between two tables (e.g. Sales → Products) so data flows between them.

Cardinality. The “one-to-many” or “many-to-one” nature of a relationship.

DAX. Data Analysis Expressions — the formula language of Power BI.

SQL. Structured Query Language — the standard language for querying databases.

SUM / AVERAGE / COUNT. Basic aggregates: total, mean, and count.

DISTINCTCOUNT. Count of unique values.

CALCULATE. A DAX function that changes the filter context of a measure.

FILTER. A DAX function that keeps rows matching a condition.

RELATED. A DAX function that pulls a value from a related table.

DIVIDE. A safe division function that handles divide-by-zero.

ALL. A DAX function that removes filters (often used for “% of total”).

RANKX. A DAX function that ranks rows by a value.

TOPN. A DAX function that returns the top N rows.

Filter context. The set of filters currently applied when a measure computes.

Row context. The “current row” when a calculated column or iterator computes.

Iterator. A DAX function (SUMX, AVERAGEX) that evaluates row by row.

Data dictionary. Documentation of every table, column, and measure in a model.

Profiling. Inspecting a table's distinct values, blanks, min/max, and samples.

Linting. Static checks that flag risky patterns (e.g. / instead of `DIVIDE`).

Best practices. A health check of the model against known good patterns.

Fail-closed guard. A safety rule that blocks anything not clearly allowed.

Data quality. How clean, complete, and trustworthy the data is.

Segmentation. Splitting customers or data into groups for analysis.

Seasonality. Regular, repeating patterns over time.

KPI. Key Performance Indicator — a metric that matters to the business.

Dashboard. A view of key KPIs, usually on one screen.

Report. A detailed, interactive set of visuals built on a model.

Data governance. The rules, ownership, and controls around a data asset.

Jevons paradox. When something gets cheaper, we use more of it, not less.

Appendix B — Data Quality Checklist

Use this before trusting any analysis. Each item can be checked with the assistant.

Completeness

- ☐ No unexpected blank values in key columns. (*Profile the table; look at the Blanks column.*)
- ☐ Every expected row is present (no missing periods, regions, or products).
- ☐ Row counts match the source system.

Accuracy

- ☐ Totals reconcile with the source of truth.
- ☐ Numbers are in the right unit (euros vs cents, units vs cases).
- ☐ No obviously wrong values (negative quantities, dates in the future).

Consistency

- ☐ Text is standardised (no “Milan” vs “milan” vs “MILAN”). (*Add an UPPER/LOWER column to check.*)
- ☐ Same entity has the same name everywhere.
- ☐ Codes match across tables (every ProductID in Sales exists in Products).

Uniqueness

- ☐ Key columns are unique where they should be (one row per SaleId).
- ☐ No duplicate customers, products, or stores.

Validity

- ☐ Values fall within expected ranges (price > 0, dates valid).
- ☐ Categories come from an allowed list.
- ☐ Formats are correct (dates are dates, not text).

Timeliness

- ☐ Data is current enough for the decision.
- ☐ The refresh happened when it was supposed to.

Integrity

- ☐ Relationships are wired correctly (many-to-one, active).
- ☐ No orphan rows (sales pointing to a missing product).
- ☐ The model passes the best-practices check.

How the assistant helps

- **Profile** each table to see distinct values, blanks, min/max, samples.
- **Validate** formulas before you save them.
- **Lint** DAX to catch risky patterns.
- **Best-practices report** to check the whole model at once.

A clean model is not a nice-to-have. Every number downstream inherits the quality of the data upstream. Check it once, trust it everywhere.

Appendix C — Dashboard Checklist

Before you ship a dashboard, walk through this list.

Purpose

- ☐ The dashboard answers a clear question (or set of questions).
- ☐ The audience is known (exec, ops, analyst).
- ☐ The decision it supports is known.

Layout

- ☐ The most important number is the biggest and top-left.
- ☐ Related metrics sit together.
- ☐ Nothing competes for attention.
- ☐ It fits one screen without scrolling (for the main view).

Metrics

- ☐ Every KPI is defined and understood.
- ☐ The KPIs are the *right* ones, not just the easy ones.
- ☐ Units and formats are clear (€, %, counts).
- ☐ Each number can be traced back to the model.

Visuals

- ☐ The chart type fits the data (bars for comparison, line for trend, etc.).
- ☐ No misleading axis, no cherry-picked window.
- ☐ Colors mean something (not decoration).

- ☐ Labels are readable at a glance.

Interactivity

- ☐ Filters work and make sense.
- ☐ Drill-down goes where the user expects.
- ☐ Cross-highlighting behaves correctly.

Trust

- ☐ The numbers reconcile with the source.
- ☐ The model passed the best-practices check.
- ☐ The dashboard is documented (what each KPI means).
- ☐ Someone owns it and keeps it fresh.

Honesty

- ☐ The story the dashboard tells is the honest one.
- ☐ A hard question about a number can be answered without embarrassment.

A good dashboard is not the prettiest one. It's the one people trust enough to act on. Build for trust first, beauty second.

Appendix D — Real Cases by Sector

How the same agentic pattern — ask in plain English, get a real model change — shows up across different kinds of business. Each case is a small story of a question and how the assistant answers it.

Retail

The question: “Which products are eating our margin?” **The ask:** create a margin measure and rank products by it. **The result:** a leaderboard showing that a few high-discount SKUs sell well but lose money. The buyer cuts the discount on the worst one. Margin recovers within a quarter.

The question: “Which stores are underperforming?” **The ask:** total sales by store, then by region. **The result:** two stores lag. A visit finds a stock-out problem, not a demand problem.

E-commerce

The question: “Who are our high-value customers?” **The ask:** a high-value flag measure over a spend threshold. **The result:** a segment of 8% of customers driving 40% of revenue. A targeted email campaign lifts repeat purchases.

The question: “Which category is growing fastest?” **The ask:** sales by category by month. **The result:** one category climbs while others flatline. Marketing shifts budget to ride the trend.

Manufacturing

The question: “Which line has the most defects?” **The ask:** defect count by production line, ranked. **The result:** one line stands out. Maintenance finds a worn part before it fails.

The question: “Are we hitting our output target?” **The ask:** actual vs target as a percentage measure. **The result:** a KPI card that turns red when output slips.

Finance

The question: “Where is spend concentrated?” **The ask:** spend by department, share of total. **The result:** one department is 35% of spend. A budget review follows.

The question: “Which accounts are overdue?” **The ask:** a flag for invoices past due. **The result:** a collection list that clears cash faster.

Healthcare

The question: “Which patients are at risk of readmission?” **The ask:** a risk flag based on prior visits. **The result:** a follow-up list for the care team.

The question: “How is bed occupancy trending?” **The ask:** occupancy by week. **The result:** a line chart that warns of a coming surge.

The common thread

Every sector asks the same shape of question: *compare, rank, flag, trend*. The assistant answers that shape instantly, whatever the data. The domain changes; the pattern doesn't. That's why one tool travels so far.

Appendix E — Guided Exercises

Practice the agentic pattern. Each exercise gives you a goal and a hint. Try it yourself first; the solutions are in Appendix F.

Exercise 1 — Read the model

Goal: Connect to a Power BI report and list every table with its row count. **Hint:** Ask for the tables and their rows in one sentence. **You'll learn:** connection and discovery.

Exercise 2 — Profile a table

Goal: Find out how many distinct categories exist and whether any column has blanks. **Hint:** Profile the table and read the Distinct and Blanks columns. **You'll learn:** data quality at a glance.

Exercise 3 — Clean text

Goal: Add a column that puts a text field in capital letters so it groups cleanly. **Hint:** Ask for an UPPER-style calculated column. **You'll learn:** standardising messy data.

Exercise 4 — Bucket a number

Goal: Turn a price column into High / Mid / Low bands. **Hint:** Ask for a calculated column with a threshold rule. **You'll learn:** turning a number into a usable category.

Exercise 5 — Wire a relationship

Goal: Connect a sales table to a product table on the shared ID. **Hint:** Ask for a many-to-one relationship on the matching column. **You'll learn:** the wiring that lets data flow.

Exercise 6 — Build a measure

Goal: Create a total-sales measure with a euro format. **Hint:** Ask for a SUM measure with a currency format. **You'll learn:** the most basic, most important measure.

Exercise 7 — Filter a measure

Goal: Create a measure that counts only sales above a threshold. **Hint:** Use CALCULATE with a filter condition. **You'll learn:** conditional aggregation.

Exercise 8 — Share of total

Goal: Make a measure showing each category's share of total sales. **Hint:** Divide the filtered total by the ALL() total. **You'll learn:** part-over-whole.

Exercise 9 — Rank

Goal: Produce a leaderboard of products by sales. **Hint:** Ask for a ranking with RANKX. **You'll learn:** ordering by a metric.

Exercise 10 — Validate before you save

Goal: Check that a formula is valid before creating the measure. **Hint:** Validate the DAX first; create it second. **You'll learn:** the safe order of operations.

Exercise 11 — Lint

Goal: Find a risky division in a formula. **Hint:** Lint a formula that uses / instead of DIVIDE. **You'll learn:** catching anti-patterns.

Exercise 12 — Document

Goal: Generate a data dictionary for the whole model. **Hint:** Ask for the dictionary and read what it returns. **You'll learn:** documentation as a one-sentence task.

Work through these in order. Each one is a real action on a live model — the same kind you saw in the chapters. When you can do all twelve, you can do the job.

Appendix F — Exercise Solutions

Solutions to the guided exercises in Appendix E. Each shows the plain-English ask and the DAX the assistant produces.

Exercise 1 — Read the model

Ask: “List every table with its row count.” **What happens:** the assistant reads the live model and returns each table with its type, row count, and column count. No DAX needed — it’s a discovery call.

Exercise 2 — Profile a table

Ask: “Profile the Products table.” **What happens:** the assistant returns a profile table with Distinct, Blanks, Min, Max, and Top values per column. The distinct count of Category is 3; blanks show per column.

Exercise 3 — Clean text

Ask: “Add a column with the category in capital letters.” **DAX:** `UPPER(Products[Category])` **Result:** a new calculated column `Products[CategoryUpper]`.

Exercise 4 — Bucket a number

Ask: “Bucket products into High / Mid / Low by price.” **DAX:**


```
SWITCH(TRUE(),  
  Products[Price] >= 200, "High",  
  Products[Price] >= 50, "Mid",  
  "Low")
```

Result: a new calculated column Products[PriceBand].

Exercise 5 — Wire a relationship

Ask: “Connect Sales to Products on ProductID.” **Result:** a many-to-one, single-direction, active relationship Sales[ProductID] → Products[ProductID].

Exercise 6 — Build a measure

Ask: “Create a Total Sales measure with a euro format.” **DAX:** SUM(Sales[Amount]) with format #,##0.00 €. **Result:** a new measure Sales[Total Sales].

Exercise 7 — Filter a measure

Ask: “Count only sales above 300.” **DAX:** COUNTROWS(FILTER(Sales, Sales[Amount] > 300)) **Result:** a new measure Sales[Big Sales Count].

Exercise 8 — Share of total

Ask: “Each category’s share of total sales.” **DAX:** DIVIDE([Total Sales], CALCULATE([Total Sales], ALL(Sales))) **Result:** a new measure Sales[Pct Of Total] with a percent format.

Exercise 9 — Rank

Ask: “Rank products by sales.” **DAX:** `RANKX(ALL(Products), [Total Sales])` **Result:** a leaderboard with each product’s rank.

Exercise 10 — Validate before you save

Ask: “Is this a valid measure? `SUM(Sales[Amount])`” **What happens:** the assistant validates and returns “Valid” with a sample value (22023). Only then do you create the measure.

Exercise 11 — Lint

Ask: “Lint this DAX: `SUM(a)/SUM(b)`” **What happens:** the linter flags the `/` and suggests `DIVIDE()` to handle divide-by-zero safely.

Exercise 12 — Document

Ask: “Generate a data dictionary for the whole model.” **What happens:** the assistant returns a markdown dictionary listing every table, its type and row count, and every measure with its format and expression.

The pattern in every solution

Ask plainly → the assistant writes correct DAX → it applies the change live → it reports exactly what it did. That loop is the whole skill. Once it feels natural, you’ve internalised the book.

Appendix G — A Portfolio Project Model

A portfolio proves you can do the job. This model gives you a project to build, document, and show. Do one or two of these and you have something to point at in an interview.

The project: a sales analysis dashboard

Build a small end-to-end analysis on a sample sales dataset (the same shape used throughout this book: Sales, Products, Customers, Stores).

Step 1 — Understand the data

- Profile each table.
- Note distinct values, blanks, and ranges.
- Write one sentence per table: what it holds.

Step 2 — Clean and model

- Standardise messy text (UPPER/LOWER columns).
- Bucket numbers into bands (price bands).
- Wire the relationships (Sales → Products, Customers, Stores).

Step 3 — Build the metrics

- Total Sales, Total Qty, Orders.
- Average Order Value, Sales per Customer.
- A margin measure (revenue minus cost).

- A share-of-total measure.

Step 4 — Segment

- Top customers by spend.
- Sales by segment (Retail / Business / Online).
- Sales by region and by month.

Step 5 — Validate and document

- Validate every measure before saving.
- Lint the DAX for anti-patterns.
- Generate the data dictionary.
- Run the best-practices report.

Step 6 — Visualise

- A KPI card row (total sales, orders, avg order).
- A bar chart of sales by category.
- A line chart of the monthly trend.
- A leaderboard of top products.

What to show in the portfolio

For each project, present:

1. **The question.** What business problem you were solving.
2. **The model.** A screenshot of the tables and relationships.
3. **The metrics.** The measures you built, with their DAX.
4. **The dashboard.** The final visuals.
5. **The story.** What you found and what you'd do about it.

6. **The tooling.** A note that you built it with AgentBridge + PowerBITool, and how the assistant helped (validation, documentation, best-practices).

Why this works

An interviewer doesn't care that the tool made it fast. They care that you can: frame a problem, build a clean model, validate your work, document it, and tell a story. This project exercises all six. The tool is a bonus that shows you're current — not a shortcut that replaces the thinking.

Make it yours

Swap the sample data for a dataset you care about — a hobby, a public dataset, a side project. The more you care about the subject, the better the questions you'll ask, and the better the portfolio looks. The pattern is the same; the data is yours to choose.

Appendix H — Interview FAQs

Common data analyst interview questions, with short answers that show you understand both the craft and the modern tooling.

“What does a data analyst actually do?”

Turns business questions into data answers. Finds and cleans data, models it, computes metrics, and tells a story that drives a decision. The tool handles the doing; the analyst owns the question and the judgement.

“SQL or DAX?”

Both. SQL for querying databases; DAX for modelling and measures in Power BI. They complement each other. Knowing when to use which is the real skill.

“What’s the difference between a calculated column and a measure?”

A calculated column is computed once per row and stored. A measure is computed at query time, responding to filters. Use a column for row-level attributes; use a measure for aggregations that must react to the report’s filters.

“Explain CALCULATE.”

CALCULATE changes the filter context of a measure. It’s the most powerful function in DAX because it lets you compute a value under a

specific set of filters — for example, sales for one category, or excluding a region.

“How do you handle divide-by-zero?”

Use DIVIDE instead of /. DIVIDE returns a safe result (blank or a fallback) when the denominator is zero. Never use raw / in a measure.

“How do you check data quality?”

Profile the tables: distinct values, blanks, min/max, duplicates. Reconcile totals with the source. Run a best-practices check on the model. A clean model is the foundation of every trustworthy number.

“What’s a star schema?”

A central fact table (e.g. Sales) connected to dimension tables (Products, Customers, Stores) by many-to-one relationships. It’s the standard, efficient shape for analytical models.

“How do you deal with a misleading chart?”

Redraw it honestly. Check the axis, the time window, and the aggregation. If a chart can be read two ways, the analyst’s job is to make the honest reading the obvious one.

“What do you do when the data contradicts the expected answer?”

Trust the data, then investigate why. A surprising result is often the most valuable one. Check the source, the filters, and the definitions

before concluding.

“How do you use AI tools in your workflow?”

As an accelerator, not a replacement. I use an assistant (AgentBridge + PowerBITool) to build and validate measures, document the model, and run best-practices checks — so I spend my time on the questions and the story, not the syntax. I check every number before I publish it. The tool does the how; I own the what and the why.

“Tell me about a project you built.”

Use the portfolio project from Appendix G: the question, the model, the metrics, the dashboard, the story, and how the assistant helped. Show that you can do the whole chain and that you understand every step.

The meta-answer

Almost every good answer comes back to the same idea: **the tool makes the work fast; the analyst makes it right**. Show that you know both halves and you stand out from the people who only know one.